Scale — Decision Records

The register's "named decisions" (`docs/technical-debt-register.typ`), recorded so that Wave 1 can proceed on settled answers rather than guesses. Each record states the decision, the options considered, the recommendation, and what it unblocks.

Status policy. Decision records are historical. Their suite counts, kernel composition, and ABI details are snapshots at decision time, not current status. Live status is in `docs/commitments-ledger.typ`; do not rewrite an old record to the current numbers.

Index. `docs/decision-index.typ` lists every record by number and title. Find the record there, then read that one section; do not read this file end to end.

Record format

Each record is one decision, appended to the register. The heading is `== D<n> --- <short title>`; `E1` is the one pre-numbered error-semantics record. Numbers are taken at append time and are never reused or reordered.

A current record has these parts:

1. **Decision.** The normative choice, in one short paragraph. A reader should be able to implement from this part alone.
2. **Why.** The reason and the trade-off that decided it. This is the part a future reader uses to judge whether the context still holds.
3. **Boundary.** What the decision covers and, where useful, what it does not touch: kernel/Prelude, ABI, image, or runtime.
4. **Status.** `recorded` (documentation only), `contract recorded` (design fixed, implementation next), `landed`/`implemented` (code and tests), or `applied` (an earlier decision now in force). A status may carry the suite counts at that moment.
5. **Non-goals.** Explicit exclusions, so a later reader does not infer a promise the record never made.

Older records may also use **Options**, **Recommendation**, and **Unblocks** when the decision was still being chosen. Keep that shape when editing an old record; use the current shape for new records.

Rules:

1. One semantic change gets one record. A test-only or documentation-only change gets a record only when it fixes a durable rule.
2. Records are append-only. A later record supersedes an earlier one and names it; do not rewrite the old record's decision.
3. Counts and implementation snapshots in an old `Status` are historical. Never update them to current numbers; update `docs/commitments-ledger.typ` instead.
4. A record is not the specification. Link the normative document it affects (`docs/specification.typ`, `docs/term-ir.typ`, or the relevant design doc).
5. Every record ends with a short separator line so the register stays scannable.

E1 — Error semantics (decided, applied)

`executeWord` returns `Result<EvaluationState, EvaluationError>`; `run` attaches source position and returns `Result<EvaluationState, Diagnostic>`. Defects are values, not exceptions. A **non-event** (ChannelEvents would-block) is explicitly *not* modelled as an error — it is a distinct outcome, so blocking never fights ChannelEvents.

Status: **applied** (`types.fs`, `evaluator.fs`).

D1 — Ownership lifetime

Decision: is linear-ownership state *data* (serialized with the image) or *execution state* (like `Location`, not serialized)?

- **Option A — data.** Ownership lives in the image and round-trips through persistence. Consequence: persistence gains an ownership section; the image is no longer purely "values" — ownership becomes a computable property of a saved image.
- **Option B — execution state.** Ownership is per-run, like `Reductions`; not serialized. Consequence: a saved image carries no ownership; a resumed run starts with a fresh ownership state.

Recommendation: B (execution state), for now. Ownership is a property of a running process, not of the stored world; treating it as data would make the image's meaning depend on execution history — the same trap as `nor`. If a workflow later needs to snapshot a *partially consumed* heap, revisit as A.

Unblocks: whether C7–C9 need a new serialization section (under B, they do not).

D2 — Dictionary encoding (decided, applied)

Decision: how are the association environments (`UserWords`, `Bindings`, `Models`) represented as image entries?

- **Option A — vocabulary-vector precedent.** An environment entry is a tensor of `EntryIdentity` pairs `[nameId; valueId; …]`; lookup resolves the name through the text index (`string → EntryIdentity`) then scans. Reuses the existing pattern.
- **Option B — linked structure.** Explicit cons-cells in the image, with a head pointer. More faithful to a pure heap, more machinery.

Recommendation: A. The vocabulary already demonstrates this encoding; it is image-resident, persists for free, and needs no new node kinds. Name resolution remains a host text index — which is itself register row C6, already converted.

Note: a "mutation" is a new image entry (the image is immutable), so environments grow as they do today.

Unblocks: C7/C8/C9, and gives ownership state its home if D1 chooses A.

Status: **applied** (`evaluator.fs`, `types.fs`). Environments are image-resident vectors of `EntryIdentity` pairs behind roots (`UserWordsRoot`, `BindingsRoot`, `SetsRoot`). Lookups are **latest-write-wins**: the association vector is appended to, and both `findPair` and `findModel` scan it and return the most recent match, so re-binding a name replaces the earlier one. A regression test proves bindings survive a persistence round-trip. One discipline rule surfaced: interning (adding text) must thread the updated image — discarding it made later literals allocate different identities.

D3 — Value typing and `nor`'s universe

Decision: how are sorts represented, so `nor` complements over a declared, closed domain rather than the whole lexicon?

The defect: `nor` currently complements over `state.Image.Lexicon` — every tensor, including vocabulary entries, text, and bookmarks. It is category-wrong and history-dependent (`addTensor` on any operation changes the result).

- **Option A — per-entry tag.** Each lexicon entry carries an optional `ValueTag`. Simple, but "all entries tagged T" is still history-dependent, and the tagging relation lives in the host unless it is also image-resident.
- **Option B — declared domain model.** `model` creates a domain; members are admitted to it; `nor A B` complements within the shared domain.
- **Option C — sorts + typed models (recommended).** A *sort* is an `EntryIdentity` naming a type. A model is a subset of a sort: `{ Type : EntryIdentity; Members : EntryIdentity list }`. `nor A B` requires `A.Type = B.Type`, else a diagnostic, and returns `Type \ (A ∪ B)`.

Recommendation: a ternary `nor` over an explicit universe (adopted).

Rather than wait for a full sort system, `nor` now takes its universe as an explicit operand: `U A B nor` yields `U \ (A ∪ B)`. The universe is a model, so the complement is taken over a *declared, closed* domain — no category error (the old code complemented over the whole lexicon, including vocabulary and text tensors) and no dependence on unrelated image contents. `and`/`or` remain derivable given a universe, so `nor` stays the primitive.

Implemented (`evaluator.fs`, `popModel` + `norWord`), with tests in `development/scale/tests.fs`. This is the minimal correct fix. It leaves the door open to the fuller option C (sorts + typed models) that Unification will eventually want, at which point the universe could default to a sort rather than being passed explicitly. **Update (W3, D14):** option C's registry is landed. The ternary `nor` is unchanged; the default-universe form is the unary `complement` over a declared sort. A binary `nor` would be ambiguous on a stack, so the arity did not change.

```
// A sort is an image entry naming a type (e.g. the token "player").
// A model is a subset of a sort.
type Model =
    { Type    : EntryIdentity
      Members : EntryIdentity list }

// nor A B : requires A.Type = B.Type, else Error (ValueSetMismatch …)
//           result = A.Type's declared domain minus (A.Members ∪ B.Members)
```

Open sub-decision (resolved): how a model is given its sort at the surface. Adopted for now: the universe is passed explicitly (ternary `nor`), sidestepping the arity question entirely. If the fuller sort system (option C) is later adopted, the surface choice is (i) `model <sort> <name>`, (ii) keep `model <name>` and treat the name as its own sort, or (iii) a separate `sort` word. That decision stays open for the Unification work.

Unblocks: the `nor` fix (Tier 1, now applied) and C9 (Wave 1's `Models`). It is the substrate `nor` and Unification share (both need typed terms + resolution).

C4 — Stack (decided, applied)

Reclassified from host-type leak to **category error**: the stack is control state, not image data. Resolved by representing it as `EntryIdentity list` rather than a `Tensor` of doubles-as-indices.

Status: **applied** (`stack.fs`, `types.fs`).

D4 — Program classification (Wave 2)

Decision: is `EvaluationState.Program : string list` data (convert to `EntryIdentity`s) or control state (re-scope)?

Control state. Every *persistent* form of program text is already image-resident — user-word bodies (D2), quotations, and bookmark programs are text entries. The live `Program` list is only the interpreter's pending-token work queue, consumed as it executes and never persisted (bookmarks persist the remaining program as text when needed). Converting it would force interning every token and decoding it again at dispatch, with no semantic gain. C10 is therefore closed as re-scoped, with the same latitude as C4. The future event-stream commitment is the proper home for execution history, and remains deferred.

D5 — Property-test framework (W1)

Decision: emitted properties are framework-free predicates (`: bool`) composed with a generated `(==>)` implication. Wiring a random-generation harness — Hedgehog as the design target, FsCheck as the possible bootstrap — is deferred until a package can be added.

Rationale. Zero dependencies: the emitted module compiles and runs anywhere, and the predicate is the reusable core a harness wraps. Trade-offs, recorded honestly: no random generation yet, and the verb's domain function is a generated default (`true`), so the sample property is vacuous until a host domain module supplies implementations. The pipeline's contract at this stage is *shape*: known-compiling properties that a harness can adopt unchanged.

Verification (golden file). The emitter's output for the sample sentence ("Every player that is alive can jump.") is compiled in this project as `generated/sample-properties.fs`. A test pins the emitter to exactly that known-compiling output and invokes the compiled property directly.

Amendment (harness wired). Hedgehog 1.0.0 was added (Q2 answered: Hedgehog; the design intent holds). The harness lives in `properties.fs` — fourteen properties over generated inputs, covering the level model (an `EntryIdentity` projects injectively into a rank-0 tensor), the append-only image text interning, persistence, Railway totality and determinism, the `nor` family as set algebra, image-resident bindings and latest-write-wins, tokenizer activity, emitter purity, vocabulary lineage, and stack order (last in, first out). Each property is folded into the suite totals individually (`runAll` returns a `{ Passed; Failed }` tally), so a regression names how many properties broke and the summary counts them alongside the example tests (121 checks when green); a `GaveUp` outcome — a generator that discarded too many samples — prints its discard count and an explicit note that there is no counterexample or seed to recheck, rather than a bare FAIL. Two API notes for the next agent: the F# API lives under `Hedgehog.FSharp` (types are `Hedgehog.Gen<'a>` / `Hedgehog.Property<'a>`; `property { ... }` needs a `let!` before `return` or it degrades to `Property<unit>`), and `Property.reportBool` returns a `Report` whose `Status` is `OK`/`Failed`/ `GaveUp` — Hedgehog swallows exceptions inside property bodies, so a thrown exception surfaces only as `Failed` (wrap and print if you need the cause). Also `List.zip` errors on unequal lengths rather than truncating.

D6 — Reproducibility of the verification suite

Decision: the suite serves scientific-computing hygiene, not just pass/fail:

1. **A failing suite fails the process.** `bases/scale-cli/entry.fs` exits non-zero when any test or property fails — CI and reproducibility rely on the exit code, not on a human reading the log.
2. **Counterexamples are reproducible.** On property failure the harness re-runs via `Property.checkBool` and prints the shrunk counterexample plus Hedgehog's recheck seed string; `Property.recheck` with that string reproduces the exact failure.
3. **No a-priori seed pinning** (verified empirically against Hedgehog 1.0.0): the F# API cannot pin a seed before a run, so each run samples fresh inputs — better falsification coverage over time, but a single green run is evidence, not proof. Reproduction happens *on failure*, via the recheck seed; a

failing property that cannot be reproduced from the seed string should be treated as a harness bug.

Findings the properties surfaced (recorded honestly):

- The first harness's `nor`/`and`/`or` probes pushed the result model's name as a bare token; unquoted, it reads as an unknown word, so membership silently defaulted to false. The properties now quote model names — a reminder that property harnesses are code too, and a silently weakened probe is worse than no probe.
- The `bindings survive persistence` property initially tested a *stronger* claim than the project makes: the binding **data** is image-resident and survives, but `BindingsRoot` is execution state per D4 and must be re-threaded by the caller after a restore (as `development/scale/tests.fs` always did). The ledger row's wording is now aligned with that contract.
- `List.zip` throws on unequal lengths (hit twice in harness code now); recorded in the roadmap Gotchas.

Amendment (counted honestly, and quiet by default). Three harness defects were found by inspection and fixed:

1. **Properties were collapsed into one failure.** `Tests.runAll` did `if not (Properties.runAll v) then failed <- failed + 1`, so any number of property failures reported as a single "TEST(S) FAILED", and the property passes were never counted at all (the "107" in the summary silently omitted the 14 properties). `Properties.runAll` now returns a `{ Passed; Failed }` tally and the caller folds both counts in, so the summary names how many checks broke and the green total reads 121.
2. **`showBookmark` was impurity leaking into the property run.** It `printfn`s its PC coordinates, and it sat in the generator's `coreWords`, so random programs printed bookmark lines mid-suite. Like `print`, its only effect is stdout — no state a property can falsify, and it drowns the log — so it is now excluded from generated programs, matching the existing exclusion of `print`. The word itself is unchanged (the demo still uses it).
3. **A `GaveUp` was a bare FAIL.** Hedgehog gives up when a generator discards too many samples; there is no counterexample and no seed worth rechecking. The harness now prints the discard count and says so explicitly, so a too-narrow generator is diagnosable instead of looking like a falsification.

Two smaller ergonomics fixes: the showcase's scratch image path is a unique file under the OS temp directory (was a fixed `/tmp/lia-image.bin`, which two concurrent runs would collide on), and `main` honors a `--tests` argument that runs the suite without the demo showcase (previously `argv` was ignored, so the whole showcase always ran).

D7 — The refinement loop (demonstration)

Decision: the project's central claim — that specifications grow when they are falsified — is demonstrated end to end in `components/demo/demo/`, rather than left as an aspiration. The loop is:

1. `components/demo/demo/player.scale` is the single editable source of truth: one specification sentence, plus machine-appended refinements, each carrying the counterexample that revealed it.
2. The demonstration compiler emits the compiled property modules (`components/demo/demo/generated/PlayerSpecificationPristine.fs` for the pristine document, `components/demo/demo/generated/PlayerSpecificationRefined.fs` for the refined one) and a byte-pinned SMT obligation (`components/demo/demo/generated/PlayerSpecificationPristine.smt2` / `PlayerSpecificationRefined.smt2`) from that document.

3. Hedgehog runs the property. On failure it yields a shrunk counterexample and a recheck seed; the demonstration consumes that diagnostic (parsing the `checkBool` report, the documented diagnostic surface of Hedgehog 1.0.0).
4. The refinement service (`refiner.fs`) proposes single-field guards from the domain's *enumerable universe* (deliberately tiny: forty-two players) and proves each candidate's closure with the SMT query `pre ∧ guard ∧ ¬post` unsatisfiable (W14), rejecting vacuous guards. The finite universe is the candidate source and the coverage ranking, no longer the soundness check. It selects the weakest proved guard (maximum coverage).
5. The chosen guard is appended to the specification with its provenance, both artifacts are re-emitted, and a fresh Hedgehog run confirms the fault class is closed.

In this demonstration, the specification says every player that is alive can jump, while the implementation silently also requires health above one. Hedgehog shrinks the counterexample to a player who is alive with health zero, and the refiner chooses `health > 1` (nine of the twenty-one inputs remain in scope). The full loop is deterministic: the specification file, the emitted F# modules, and the `.smt2` obligations are byte-identical across runs, so the repository stays clean after a run.

Honest limits, recorded up front:

- **The runtime check was an in-process mirror — now closed (D9/W12).** It once evaluated a predicate derived from the same document that the emitter renders, while the emitted F# text itself was not compiled and run. The existing golden test for the emitter (`generated/sample-properties.fs`) proved the emitter mechanism produces compiling code. Decision record D9 closed the last gap by having the loop run the compiled artifact: the initial and refined properties are compiled modules (`PlayerSpecificationPristine.fs`, `PlayerSpecificationRefined.fs`) invoked by a thin harness, and re-emission is pinned byte-identical to the checked-in artifacts.
- **Lean is retired (D10/W13).** The theorem obligation is the `.smt2` file; no Lean toolchain is wired into this environment. Lean remains possible as an optional export, not as the backstop.
- **Refinement candidates are single-field guards drawn from an enumerable universe.** Closure is now proved by SMT (W14), so the finite universe is a candidate source and a coverage heuristic, not a soundness boundary. A larger domain needs a richer candidate generator, not a sampling-based closure check.
- **The refinement is single-field, and positive in shape.** The parser now recognizes varied sentences by vocabulary category and marker order (W2, decision record D8), but this demonstration uses a positive Capability sentence, and its refinement service is written for that shape; extending the search to invariants and transitions is future work.
- **The specification file is reset before each run**, so the showcase is repeatable; in real use the service only ever appends.
- **A refinement admits two readings.** It can be a discovered requirement to be accepted into the specification, or an implementation fault to be fixed. The demonstration does not choose; the file is the source of truth, not an oracle.

Verification. `development/scale/tests.fs` pins the loop's outcome: the initial specification is falsified, the refiner selects `health > 1` (the unique weakest guard that closes the class), the refined property holds under a fresh Hedgehog run, and the emitted artifacts and the specification file carry the refinement. W12/D9 extends this: the loop runs the compiled predicates, and re-emission is byte-identical to each checked-in artifact. W14 extends it again: closure is the proved SMT query, and the selected refinement records the solver version and the Z3 witness model.

D8 — Parser generalization (W2)

Decision: sentence-pattern recognition no longer hardcodes specific words. It reads the categories the vocabulary assigns, and respects the order of the markers:

- a Conditional marker frames the whole sentence, so the pattern is a Transition;
- otherwise a Negation marker makes it a negated statement, an Invariant;
- otherwise a Quantifier before a Function Name is a Capability;
- otherwise the sentence is not recognized.

The pattern reaches the emitted code. An Invariant asserts that the verb never holds, so its postcondition is negated (`precondition ==> not (verb player)`). The Capability path is byte-for-byte unchanged, so the pinned golden output is untouched.

A corpus of sentences (`development/scale/tests.fs`) pins the recognizer across the surface: subject and condition variety (including condition order), several quantifiers, negation, and a conditional, plus two negative cases (a verb placed before its quantifier, and no quantifier at all).

Deferred, named honestly:

- **Quantifier polarity.** The vocabulary records "no" as an ordinary Quantifier, so "No player can jump." currently reads as a Capability. Distinguishing it needs a polarity field on the vocabulary entry, which would also change the image encoding; deferred as its own decision. **Resolved by D18 (W2.1):** the grammar reads `every` as a Capability and `no` as an Invariant.
- **Emission for a subject with no conditions.** The generated record would have no fields, which is not valid F#. Parsing such sentences works and is tested; emitting them is deferred. **Still open.**
- **Transition-specific emission.** A Transition currently renders like a Capability; a state-change rendering is deferred. **Superseded by D18:** the grammar refuses the `Conditional` marker (`when`) with `ConditionalNotAllowed`, so no surface sentence produces a `Transition`.

Superseded in part by D18 (W2.1). D18 replaces the category-and-order recognizer with an explicit grammar, resolves quantifier polarity for `every` and `no`, and removes the `Transition` surface by refusing `when`. D8's vocabulary rule stands: the parser still reads domain words from the image rather than from a hardcoded list.

Verification. `development/scale/tests.fs` includes the sentence corpus, the two order and quantifier negative cases, and an assertion that an Invariant renders a negated postcondition.

D9 — The loop executes the compiler's output (applied)

Decision: what the refinement loop *runs* is the emitter's compiled output, not a property hand-written beside it. D7's first honest limit — the runtime check was an in-process mirror of the emitted text — is closed by construction, not by assertion.

Three sub-decisions carry it.

1. One emitter, two domain bindings. `emitter.fs` becomes the single renderer of the F# and Lean artifacts; `demo.fs`'s private `emitFSharp` / `emitLean` are deleted, not kept in parallel (two renderings of one translation is the two-sources-of-truth defect the register already named). The emitted F# stays framework-free (D5): a `(==>)` and a `: bool` predicate. What varies is the **domain binding** — the subject type, the verb's F# name, and the field-access mapping — supplied as data, never hardcoded:

- **Self-contained** (the golden sample): the subject record is generated from the vocabulary `TypeName`/`FieldName`, and the verb's domain function is the documented default `true`. This is the "compiles and runs anywhere" shape; its vacuity is the stated placeholder of D5, not a defect.

- **Host-bound** (the demonstration): the subject type and the verb come from the host domain module (`DemoDomain`), so the property is non-vacuous — it runs the implementation under test.

Rejected: an always-self-contained emitter (a demonstration whose verb is `true` demonstrates nothing); keeping a second demonstration emitter.

2. Two compiled artifacts, one per document state. A run has exactly two document states — pristine, then refined — so it emits exactly two modules, both checked in and compiled into `bases/scale-cli/scale.fsproj`:

Artifact	Module	Emitted from	Role in the loop
`components/demo/demo/generated/PlayerSpecificationPristine.fs`	`Scale.Generated.PlayerSpecificationPristine`	the pristine document	the property that is *falsified*
`components/demo/demo/generated/PlayerSpecificationRefined.fs`	`Scale.Generated.PlayerSpecificationRefined`	the refined document	the property whose *confirmation* closes the class

Rejected: compiling mid-run (invokes the toolchain from inside the run, and breaks the "output is checked in and pinned" discipline); passing the guard as a runtime value instead of baking it into the emitted precondition (the refinement must be visible in the source text of the artifact — it is the grew-specification, not an input).

Module names are flat and distinct: one emitted module may not nest under another's name.

3. The harness is thin and honest. Generators are *inputs*, supplied by the host (`DemoDomain`); the property body calls the compiled `: bool` predicate. This is exactly the D5 contract — a framework-free predicate adopted by a harness — and what was a mirror becomes a two-line wrapper. The refiner, its diagnostic, and the recheck seed are unchanged; the diagnostic now comes from a run of the compiled artifact.

Verification (acceptance). Re-emission is byte-identical to each checked-in artifact for its document state (so a run leaves the repository clean); the loop's initial falsification and its refined confirmation both execute compiled predicates; the D7 outcome tests (weakest guard `health > 1`, provenance in the `.scale` file, recheck seed) still hold; the suite reports its totals individually.

Limits, named up front. Lean is still `sorry` and unwired (D7). **Note: the backstop choice itself is in flux — the owner reports the plan changed to include a verifier other than Lean (roadmap Q5); until that tool is named and recorded, `LeanBinding` and the `.lean` obligation are the current encoding of a backstop slot, not a commitment to Lean (planned decision record: D10).** The demonstration domain is still one sentence, one field, one guard shape; the verb is host-supplied by design (the emitted property binds to a domain module); and the refined artifact is a checked-in golden that matches the run *because the loop is deterministic* — a non-deterministic refiner would require recompilation, which is the price of "the artifact is compiled output".

Status: applied (W12). The wording in the ledger and the README marks the mirror as closed under D9; the **Landed (W12)** note below records the landed implementation.

Landed (W12). `emitter.fs` is the single renderer, parameterized by a `DomainBinding` supplied as data: `SelfContained` (the golden sample) is built by `Emitter.selfContained`; the demonstration's `HostBound` binding lives in `demo.fs` and binds to `DemoDomain`. `demo.fs`'s private `emitFSharp` / `emitLean` and its `propertyFor` mirror are deleted; the loop's flip to the compiled predicate is a thin harness wrapping `Scale.Generated.PlayerSpecificationPristine.``player canJump`` (falsified) and` Scale.Generated.PlayerSpecificationRefined.`player canJump` (confirmed), both

compiled into `bases/scale-cli/scale.fsproj`. `scale-file.fs` moved above `emitter.fs` so a binding can bake the document's refinements into the emitted precondition. One correction the landing surfaced: a multi-clause precondition is parenthesized, because F#'s `==>` binds tighter than `&&` (a single clause stays bare, so the pinned golden output is unchanged). `development/scale/tests.fs` pins the outcome: the compiled initial predicate is falsified over the domain universe, the compiled refined predicate holds over it, the loop's shrunk counterexample flips between the two, and re-emission is byte-identical to each checked-in artifact.

D10 — The verifier is an SMT solver in-process (Z3), not a proof assistant

Decision: the verification backstop is **Z3, in-process via `Microsoft.Z3`**, driven by an encoding generated from the *same parsed document* that produces the F# predicate. Lean is retired as the verification path: the `.lean` obligation and its `sorry` stub are replaced by an SMT obligation. The emitter keeps D9's binding abstraction, so a Lean export could return later as an *optional* binding; it is not the backstop.

Spike (verified in this sandbox, not assumed). `Microsoft.Z3` 4.12.2 restores from NuGet and runs in-process from F# on `net10.0`; `/usr/bin/z3` 4.16.0 is present as an offline fallback. A negated property `(alive && health > 1) ==> canJump` returns `SATISFIABLE` with a structured model (`alive=true, health=2, canJump=false`), and a contradictory claim returns `UNSATISFIABLE`. Models arrive as data, not text.

Why Lean was the debt, concretely.

- It translates *semantics*, not just formulas. The emitted property is a boolean predicate over a small record, but Lean's value (inductive types, tactics) is aimed at definitions and proofs; the route imports the tensor / image / stack model and then owes an argument that the translation is faithful.
- It is a second ecosystem: `elan`/`lake`, no .NET interop, and — the ledger already carried the receipt — a hand-written "Lean-side diagnostic consumer" to parse a prover's text.
- It cannot be cross-checked: the Lean model cannot be run against the compiled F# predicate, so translation drift is invisible until a proof fails.
- Today's artifact is literally `sorry`.

Why SMT fits the actual shape.

- The property is a boolean formula over booleans and linear arithmetic — the decidable core of SMT. `pre ==> post` is checked by asking for a model of `pre ∧ ¬post`: `unsat` proves it over the whole theory, not a sample.
- `Status` and `Model` come back as data. A counterexample is structured, which is exactly what the refiner consumes; the `Model` plus the solver version is the reproducibility handle (D6's spirit, without a seed string).
- The obligation can be emitted as `.smt2`. Z3's own printer renders the AST, so the human-readable artifact and the discharged formula are **one source**, not two.
- **Differential validation is the anti-debt device.** D9 made the F# predicate a compiled, executable artifact, so the encoding can be tested *against* it: exhaustive agreement over the demonstration's enumerable universe, plus agreement on every model Z3 returns. Translation faithfulness becomes a regression test — something the Lean route could not offer cheaply.

Three outcomes, reported honestly. `Proved` (the negation is unsat); `Falsified (model)` (sat, with a structured counterexample); `Unknown` (the solver cannot decide — e.g. nonlinear arithmetic or

quantifiers); `Unavailable` (no solver present). `Unknown`/`Unavailable` are never a silent pass: they are reported like D6's `GaveUp`, and Hedgehog remains the fallback evidence.

Theory alignment (a decision, not an assumption). The encoding's theory must be exact for every value the generators can produce. In the demonstration `Health` is integer-valued by construction (`Gen.int32` then `Gen.map double`), so `Int` is exact; if the domain widens to arbitrary doubles the theory must widen to `Real`, and the NaN/infinity caveat must be named. The differential test is what catches drift.

What this buys, and what it does not.

- Buys: an unbounded, sound check of a property (not a sample); structured counterexamples; a proof-shaped obligation artifact; the end of the `.lean` / `sorry` / diagnostic-consumer tax; and the substrate for **W14**, where the refiner's guard-closure query becomes `unsat` over the theory instead of an exhaustive scan of forty-two players.
- Does not buy: verification of the *evaluator*. The stack machine, image append-onlyness and preemption stay sampled by Hedgehog; if that rung is wanted, the .NET-native route is Boogie + Z3 (an explicit VC language with invariants), not a proof assistant.

Alternatives considered.

Route	Verdict
Z3 in-process, encoding from the parsed document	**Chosen.** Lowest debt; structured diagnostics; .NET-native.
F# quotations ⟶ Z3	Kept in reserve. A property written once and translated from the AST the compiler runs is the purest single source, but function bodies, pattern matches and recursion become their own translation debt. Right rung for "verify the implementation", not for properties.
Boogie + Z3	Right rung *later*: code with loops/heap (the evaluator). .NET-native, but an extra language to emit; unnecessary for formulas.
Dafny	Verifies and compiles, .NET/Z3 under the hood, but translates the code too and the verified artifact is Dafny, not our F# — a faithfulness gap like Lean's without the ecosystem split.
Lean	Only for proof-carrying artifacts or proof-assistant research. Heaviest path to the weakest fidelity for this goal.
SMT-LIB text + `z3` CLI	The portable fallback (the CLI is present at `/usr/bin/z3`) and the reason `.smt2` is a good artifact; its cost is parsing model text — the tax we just deleted.
Fable	**Not a verifier.** A second *compile target* (F# ⟶ JS/TS/Rust/Python) for the emitted property; coherent only for the child-facing/browser product, optionally with z3-wasm. Tracked as Q6, outside the verification path.

On "Fable + Z3". Re-derived from the project's needs, only **Z3** is load-bearing for verification; Fable contributes a runtime, not a proof. If the browser/education product is the goal, Fable + z3-wasm is a sensible *deployment* story and should be its own work item. It does not change the verifier decision.

Status: applied (W13, W14). The obligation slot is no longer `LeanBinding`: `domain-expression.fs` holds the implementation as one body with two backends (an interpreter for Hedgehog and an encoder for Z3), `smt.fs` renders the obligation and reports the four outcomes, and `Emitter.emitSmt`

writes the `.smt2` artifact. The `.lean` stub and its tests are gone. W14 moved the refiner's closure onto the same encoding: a candidate guard is accepted only when `pre ∧ guard ∧ ¬post` is unsatisfiable and `pre ∧ guard` is satisfiable; the universe survives only as the candidate source and coverage ranking. The selected refinement carries the solver's version and the model of the unrefined obligation. The demo's initial obligation is `Falsified (alive=true, health=1)`; the refined one is `Proved`; the suite is 153 checks and the encoding is validated differentially against the compiled predicate.

D11 — Fixed-spec mode: the loop repairs implementations, not specs

Decision: the refinement loop has two explicit modes, and the mode is a setting, not an interpretation:

- **Exploratory** (today's demonstration, D7). The specification is still being written. A witness may mean "the spec is missing a condition" *or* "the implementation is wrong." The refiner appends a candidate refinement, and a human adjudicates between the two readings.
- **Fixed-spec** (the product mode). The developer guarantees the specification. A witness therefore means, by construction, an implementation defect. The refiner **never writes to the spec**; it produces a *repair bundle* — the property, the shrunk witness, the solver's model, the encoded implementation, and the differential result — for a code generator (human or AI) to patch the implementation. A patch is accepted only by the gate.

The gate (what "accepted" means). A candidate patch is accepted iff:

1. the target property is **proved** (`Proved`); `Unknown` is reported and never counted as accepted (sampling can inform a human, but it is not the gate's proof);
2. **no previously passing property regresses** (the witness corpus is the regression suite);
3. **the specification is unchanged** — in fixed-spec mode a patch that edits the spec is rejected, by rule, not by review;
4. the encoding still agrees with the compiled predicate on the differential checks (a patch that changes behaviour the encoding cannot see is not a verified patch).

What this resolves. D7's "two readings" limit is not an unresolved ambiguity; it is a *mode switch*. In fixed-spec mode the adjudication is made once, up front, by the developer — which is sufficient, because the developer is the only party who knows the intent. The loop then runs unattended.

What the loop is and is not. It is an **acceptance oracle**, not a synthesizer. It can say "not yet" or "proved"; it cannot guarantee that a satisfying implementation exists, and it cannot guarantee that a generator will find one. "PBT → formally verified with no extra work" therefore means *no per-property proof authoring*: one spec renders twice (D9/D10), and the gate is automatic. It does not mean the loop writes the program.

Three residual human duties (the rest is automated):

1. **Fidelity.** Z3 proves things about the *encoded* implementation. Differential tests automate the trust; someone must extend the encoding when the implementation's verifiable fragment grows.
2. **`Unknown`.** A property outside the decidable fragment degrades to sampling. That must be visible, never silently promoted to "proved".
3. **Adequacy.** The loop detects vacuity (`pre` unsatisfiable), tautology (`post` true independently of `pre`), and coverage; it cannot detect *missing intent*. A correct-but-too-weak spec is satisfied by a useless implementation, and no solver can tell.

Consequence for the demo. It stays in exploratory mode (it is a demonstration of spec growth); fixed-spec mode is exercised by its own tests — a witness produces a repair bundle, a spec edit is

rejected, a patched implementation passes the gate, and a patch that regresses the corpus or falsifies the target is rejected.

Outcome (W15, landed). `repair.fs` defines the mode, witness, target, bundle, patch, verdict, and corpus records. `accept` checks the spec edit first, then the body's sort, then the hash, then discharges the target; only `Proved` proceeds. It then requires every previously passing corpus entry to still pass and requires `Smt.evaluateBody` to agree with `DomainExpression.interpretBoolean` on every corpus witness. `Unknown` and `Unavailable` are rejected, never promoted. The bundle is versioned JSON (`bundleJson`/`writeBundle`) carrying the property, parsed sentence, spec hash, shrunk witness, solver model, encoded implementation, differential result, and source location; `witnesses/player.json` is the checked-in append-only corpus, and the reader/writer round-trip it. The suite is 197 checks at W15.

Status: applied (W13, W14, W15).

D12 — Set-theoretic typing: structural and nominal as one record

Decision: how sorts are represented, so the language is honestly structural *and* nominal, and W3's surface question (Q3) is settled.

The defect: Scale already is a set theory — models are finite sets, membership is structural (`isMemberId` uses tensor equality), `nor`/`and`/`or` are set algebra and are property-pinned — but a model carries **no declared element type**. That is precisely why `nor` takes an explicit universe operand (D3): there is nothing to default it to. And "structural vs nominal" was framed as a choice, when it is a choice of *projection*.

Decision: a sort is a pair `(N : option Name, X ⊆ U)`.

- **Nominal equality** reads `N`: same name, whatever the members.
- **Structural equality** reads `X`: same extension, whatever the name. (That is extensionality — the axiom, not an analogy.)

Both are facts about the *same* value, so nominal and structural are two projections of one record, not two systems with a translator between them. The set operations are the dial: **narrow** toward nominal (seal a name, restrict `X`), **widen** toward structural (drop the name, enlarge `X`). And the same cell widens *further* to more than the two endpoints — multiset (membership with multiplicity), sequence (a position function), co-product, recursive types — which is to say set theory is not a compromise between the two type systems but the ground both stand on.

Surface (Q3 settled). A sort has two components, so it needs two words. The design adopts **special syntax**, free in the current surface (`?x`, `"…"`, `{…}` are taken):

- `[ … ]` — a **structural** set (extension `[ 1 2 3 ]`, comprehension `[ ?x : ?x > 1 ]`, empty `[ ]`), with `subset`/`union`/`intersect`/`except`/ `member`/`same-shape` and the existing `nor`.
- `:Name` — a **nominal** seal (`:player [ ]`, `:player [ 1 2 3 ]`), with `seal`/`unname`/`same-name`/`sort-of`.

This rejects **name-as-sort** (today's `model <name>`): it collapses the pair to the name alone, discards the structural half, and with it every cross-sort comparison — and therefore all of W10. The pair has two components, and W10 dispatches on the **extension** while W4 unifies under the **name**.

Consequences for the ledger: W3 (D14) built the sort registry (`N` as an image entry, `X` as the member list, `ValueSetMismatch`/`ValueSetNotClosed` named, `complement` defaulting its universe to the declared sort); W4 unifies under `N` (landed, D15); W10 dispatches on `X`; `ValueTag` is the nominal

component made image-resident; and `Guard` is retired as a special case, recognised as a comprehension (a refinement sort).

Why: it makes the two readings local and visible (`same-shape` vs `same-name` are different words over the same value), it keeps the level model (a sort is `(option EntryIdentity, EntryIdentity list)`, both projecting into tensors), and it is checkable by the existing properties unchanged: `nor` is still set complement, `and`/`or` still agree with ∩/∪.

Status: applied via D14 (W3). The full argument, the operation table, and the keystroke-by-keystroke migration are in `docs/set-theoretic-typing.typ`.

D13 — Decision models: a symbolic table, optional, proven rather than generated

Decision: whether a "decision model" belongs in the architecture, in which sense, and with what claim.

The term has two unrelated readings, and only one fits:

- **Decision Model and Notation** — a symbolic table of `condition → action` rows. No probabilities. A *specification*.
- **Machine-learning decision model** — a classifier or scorer over options supplied in advance. A *policy*, whose output is a probability or an index.

Decision: adopt the first reading, as an optional specification refinement; exclude the second from the verified path.

The symbolic table fits because both halves already existed. `ScaleFile.Guard` with `Refiner.evaluateGuard` decides a guard at runtime; `Smt.guardToBooleanExpression` encodes the same guard for Z3. A decision *row* was already expressible. The table adds the ordered collection, the hit policy, and the two checks that make it trustworthy:

- **completeness** — `satisfiable (¬(g₁ ∨ … ∨ gₙ))`; a model is an input no row covers;
- **disjointness** (under `Unique`) — `satisfiable (gᵢ ∧ gⱼ)`; a model is an input two rows both cover.

`Sound` is therefore the result of two `unsat` answers, and each failure is a concrete witness. `Undecided` is reported and never promoted to `Sound`; an unencodable table is an `Error` (unknown field, or a threshold outside the integer theory). Three failure modes, kept distinct, as with `Outcome` in `smt.fs`.

Rejected: a conditional verb body. Adding `Ite` to `DomainExpression.Expression` and encoding `MkITE` would let one body *be* the decision. It was rejected because it enlarges a fragment D10 kept deliberately small, and because it makes the interesting claim (no gaps, no overlaps) an assertion rather than a proof. It remains available if a verb ever needs to branch inside one body.

Rejected: an ML decision model inside the pipeline. Scale's trust boundary admits proofs (`unsat`) and witnesses (shrunken counterexamples). A probability is neither, so it can never be recorded as `Proved`, `Falsified`, `Sound`, or `Unsound`. Triage, prioritisation, gating, and edge-case labelling all produce labels, and no consumer here turns a label into a verdict. Such a model could only advise, outside the verified path, and would also fight the repository's determinism and offline-artifact rules.

Optionality is structural, not a flag. `decision-table.fs` is referenced by `development/scale/tests.fs` and nothing else: not the parser, vocabulary, emitter, evaluator, or demo. A domain opts in by constructing a `Table` and calling `check` and `select`. A domain that never does is bit-for-bit unaffected. At the prepared stage there were no new words, no parser change, and no emitter

binding; D73 later added the host-level emitter. `Transition` is reachable through D55, and a table does not depend on it.

What a table does and does not do. It *scaffolds*: one obligation per row, with the row's guards as the precondition (`guards ∧ ¬body`), plus the table's completeness obligation. It does **not** generate the invariant, the generator, or the body; an outcome is a domain value, and the core does not interpret it. That boundary is why the "gap/no-overlap" claim is worth having: it is the part of a decision model that can be *proved* rather than guessed.

Status: prepared, then promoted as D73; the prepared layer is pinned by tests (15 assertions; suite 271 at W4). The full argument, the operation table, the opt-in recipe, the trust-boundary argument, and the seams where it would attach if promoted are in `docs/decision-models.typ`.

Promoted as D73. The prepared layer above was promoted: `DecisionTableEmitter.emit` renders one property per row through a host `TableBinding`, and `obligationFor` exposes the row as an `Smt.Obligation`. The completeness check remains `DecisionTable.check`. The "no emitter binding" and "prepared, optional" statements describe the prepared state, not the current surface. `Transition` is reachable through D55.

D14 — Sorts are a pair, and the pair is landed (W3)

Decision: implement D12's sort record rather than leaving it as a design note. A sort is `(N : option Name, X ⊆ U)`; the language now has both projections, and the set words move between them.

Encoding. A model entry is a rank-1 tensor `[nameValue; member...]`. `-1` in the name slot means *anonymous*: a structural set with no seal. The models root (unchanged) is the registry of models and anonymous sets; a second root holds **declarations** — the same encoding, whose members are the sort's closed domain. Both roots are pointers into image-resident data, so they are execution state like `SetsRoot` (D4) and are re-threaded across persistence. The host `EvaluatorValueSets.ValueSet` record is a projection of the tensor and holds no state absent from the image (level model).

Surface. Operands first, as the stack language requires:

```
[ 1 2 3 ]                 extension literal (the body is evaluated; {3} for
                          [ 1 2 add ])
[ ]                       the empty set
[ 1 2 ] :small seal       a named sort (a seal over the extension)
S unname                  drop the seal (widen to structural)
A B subset                A ⊆ B
A B union | intersect | except
x S member                structural membership
A B same-shape            extensional equality
A B same-name             nominal equality (None = None)
A B sort-of               does A carry B's seal?
A B merge                 sort-preserving union; ValueSetMismatch if the seals differ
S complement              the declared domain minus S; ValueSetNotClosed if S is
                          anonymous or its name has no declaration
```

The design note wrote `:player [ ]`; the landed surface is `[ ] :player seal`. The prefix form cannot be read by a stack language without a pending-seal side channel, and an implicit pending name is exactly the kind of host state the level model forbids. The doc is corrected to the postfix form. `:Name` still works as a *reference* to an existing sort (`1 :small is`).

What this changes and what it does not. `model`/`admit`/`is`, `and`/`or`, and the ternary `nor U A B` keep their existing behavior and diagnostics; the existing tests and properties pass unchanged. `model "s"` now also declares `s` as a sort with an empty domain, `admit` grows that domain, and the derived `and`/`or`/`nor` models are declared too. `union`/`intersect`/`except` return anonymous sets (combining extensions has no canonical seal); `merge` is the seal-preserving form.

`nor`'s universe. The roadmap phrased the default as "`nor` can default its universe to the sort". It is landed as `complement`, a distinct unary word, because a binary `nor` cannot be distinguished from the ternary one on a stack. `complement` reads the declaration, not the models root, so a subset sealed under an existing name complements against the full declared domain. This is the D3 option-C behavior with an explicit word instead of a hidden arity change. A future `nor` arity change would need a stack marker; that is not taken here.

Diagnostics. `ValueSetMismatch(first, second)` is `merge`'s refusal (next move: `union` to widen, or seal both under one name). `ValueSetNotClosed(set)` is `complement`'s refusal (next move: seal the set, or declare the name with `model`). `UnclosedSet` and `UnexpectedSetClose` are the literal's syntax errors; `NotASet(word)` is a direct address that is not a model.

Determinism. The registry is append-only and latest-write-wins, as before; a declaration's domain only grows — a later seal can add members but cannot shrink the universe, and an already-computed complement is an immutable value. Structural membership is tensor equality (`tensorsEqual`), so equal values are one member even when they are different references; literals and unions deduplicate on that equality.

Pinned by tests. Thirty-two example checks (literal evaluation, empty set, membership, subset, union/intersect/except, both equalities, seal/unname, latest-write, `sort-of`, `merge`, `complement`, the three named refusals, both literal syntax errors, tokenizer splitting, and persistence with both roots re-threaded) plus one Hedgehog property: `union`/`intersect`/`except` agree with ∪, ∩, and \ elementwise over random subsets. Suite at W3: **245 checks, 230 examples and 15 properties**.

Status: applied. The full argument and the operation table remain in `docs/set-theoretic-typing.typ`; comprehension syntax (`[ ?x : … ]`) is not landed, and `nor` keeps its ternary form.

D15 — Unification is first-order, image-resident, and nominal (W4)

Decision: W4 upgrades `unify` from one-way variable assignment to a first-order structural unifier with an occurs-check, bidirectional variable binding, recursive substitution resolution, compound terms, and a nominal `typed` wrapper. It stays a **pure structural predicate**: one call either extends the substitution or reports `false`. It is not a Prolog engine, a goal solver, a search strategy, or a general type-inference engine.

What the concatenative neighbours do

The direct research question was what stack-based and concatenative languages do, and what is worth borrowing. The short answer is that they do **not** put general term unification in the runtime; the closest useful implementations are type-level, not value-level.

Language	What exists	What W4 took
Kitten (`Kitten.Unify`, `Kitten.Zonk`, `Kitten.Occurrences`)	A type unifier that substitutes first, checks occurrence on the substituted term, treats the variable cases commutatively, and uses a separate row unifier	Resolve before matching; occurs-check the resolved term; make variable binding order-independent. Row unification is not borrowed: W4 unifies value terms, and W10 owns extension dispatch.

Language	What exists	What W4 took
	for stack effects and permissions.	
Cat (`type-inference Unifier`)	A union-find-style type unifier with a preference order (constants over lists over variables), a deterministic choice between two variables, and recursive list unification.	A deterministic alias direction. Scale canonicalises two unbound variables to the **lower `EntryIdentity`**, which is a persistent analogue of choosing one representative. The substitution is an image-resident association vector, so `walk` replaces mutable path compression.
Factor	No general term unifier. `unify` in the stack checker is stack-effect/branch unification; runtime polymorphism is generic dispatch (`Generic`/ methods), not unification.	Keep polymorphism out of unification. W10 dispatches on the sort's extension; `unify` must not become a dispatch engine.
Joy	No variables and no unification; quotations and combinators are the abstraction.	A concatenative core does not need a logic engine. Logic variables stay a named-state escape hatch, not the evaluation model.
Prolog / miniKanren (not concatenative, but the reference algorithm)	A persistent substitution, `walk`, an occurs-check, and `unify` returning an extended substitution or failure.	The shape of the algorithm: `walk`, `occurs`, `bind`, recursive argument unification. No trail and no backtracking search: the image is immutable, and `unify` is one predicate call.

The synthesis is deliberately narrow: **Kitten's discipline** (substitute then match, check occurrence, commute variables), **Cat's determinism** (a canonical representative), and **Factor's boundary** (dispatch is a different pillar). That is enough for a fully featured first-order unifier without turning the interpreter into a logic-programming runtime.

Encoding

Terms are ordinary image entries; there is no new root and no host term state.

Term	Image encoding
variable	an interned text entry `?name`
atom	any other image entry
compound	a rank-1 vector `[TermTag; head; argument…]`, `TermTag = -2.0`
typed term	a rank-1 vector `[TypedTag; sortName; term]`, `TypedTag = -3.0`

`ValueSet.decode` accepts exactly `-1` as the anonymous marker and rejects every other negative first slot, so a term can never be mistaken for an anonymous sort. The tags are image data, not a host tag; the view (`Term`) is transient. The substitution remains the existing `BindingsRoot` association vector: the key is the variable's bare name (`?x` uses `"x"`, matching the existing `lookup` surface), and the value is the bound term's `EntryIdentity`.

Algorithm

`unifyTerms` walks both operands, then matches the views:

1. same `EntryIdentity` ⟶ success, no new binding;
2. two variables ⟶ alias the higher identity to the lower identity, so `?x ?y unify` and `?y ?x unify` produce the same substitution;
3. one variable ⟶ bind it to the other term, after an occurs-check;
4. two typed terms ⟶ their sort **names** must be equal, then unify the inner terms; a typed term never unifies with an untyped non-variable;
5. two compounds ⟶ same argument count, then unify heads and arguments pairwise;
6. two atoms ⟶ tensor equality.

`occursIn` walks through substitutions and rejects a binding that would make the variable part of its own value. It handles direct and indirect cycles and is cycle-safe for legacy bindings made by `bind`. `resolveTerm` applies the substitution: it resolves variables everywhere and rebuilds a compound or typed term only when a child changed. `lookup` uses it, and the `resolve` word exposes it.

A failed unification is **transactional**. The unifier may speculate while matching a compound's arguments, but `unify` returns the pre-call `BindingsRoot` and image when the result is `NotUnified`; a failed match never leaves a partial binding.

Surface

```
?x ?y unify              \ alias, canonical direction
?y 5 unify               \ bind the alias
"x" lookup               \ 5: lookup resolves the chain

"f" ?x 1 term            \ f(?x): head, one argument, arity
?x "f" ?x 1 term unify    \ 0: the occurs-check refuses a cycle

"player" set
?x :player typed         \ a nominal term
?x :player typed 5 :player typed unify   \ 1; binds ?x to 5
?x :player typed ?y :enemy typed unify   \ 0; names differ
?x resolve               \ apply the substitution
```

`term` consumes `head arg… arity`; `typed` consumes `term :Name`. The words are lab surface, not child-facing sentence surface. `typed` validates the name against `DeclarationsRoot`, so an undeclared sort is `UnknownModel`, and a term already sealed under a different name is `ValueSetMismatch`.

Deliberately out of scope

No higher-order unification, no rational/cyclic trees, no attributed variables, no goal search, no cut, no `if`-as-a-logic-connective, and no general Hindley– Milner inference. Those would add complexity that the project's goals do not need: the sentence layer is not a theorem prover, and polymorphism belongs to W10's dispatch on the extension. Equality is nominal at the sort layer and structural inside a term.

Pinned by tests

Twenty-four example checks in `development/scale/tests.fs`: bidirectional chains, commutative aliasing, idempotence, conflict, occurs-check direct and indirect, compound success/head mismatch/ arity mismatch/zero arity, recursive substitution, transactional failure, same/different/inherited typed sorts, typed-versus-unsorted refusal, typed substitution, undeclared sort, and invalid term arities. Two Hedgehog properties: a random variable chain resolves to its tail value, and an occurs-check rejects a variable nested at random depth. Suite: **271 checks (254 examples + 17 properties)**.

Status: applied.

D16 — Metaobject encoding: a tagged tensor, address identity, and explicit levels

Decision: a metaobject is a rank-1 tagged tensor `[tag; field…]` in the image. The tag is a negative sentinel from a single append-only registry below `-1`; fields are `EntryIdentity` references or scalars by a documented schema. Identity is the `EntryIdentity` (address), not `tensorsEqual`; structural equality remains value equality. Levels are explicit: base values (0), sorts and words (1), constructors and metaclasses (2), protocol (3). A level-n metaobject may refer only to levels at or below n; the protocol's fixed point is a single level-3 root described by itself by convention.

Why a host record is not enough. `ValueSet` and `VocabularyEntry` are already image projections, but their *behavior* is host code: `executeWord` matches token strings, `EvaluatorValueSets` owns the set operations, and `FsharpName` hands control back to F#. The level model says host types must project into tensors and hold no state absent from the image; a Metaobject Protocol adds the missing half — the *mechanism* must be image-resident too, not only the data.

Options.

Option	Verdict
Host records (status quo)	Rejected: the semantics stay host-resident; `FsharpName` is the seam.
Tagged tensors, address identity, explicit levels	**Chosen.**
Untagged universal cell	Rejected: the layers collapse; a term can be mistaken for a sort. D15 already had to reserve negative sentinels to keep them apart.

Precedent. D15's terms are `[TermTag; …]` with `TermTag = -2.0` and `TypedTag = -3.0`; `ValueSet.decode` accepts exactly `-1` as the anonymous marker and rejects every other negative first slot. That discipline is load-bearing: without it, a term is a sort is a word. D16 extends it with a registry.

Schema. Only the levels are fixed here; concrete tag numbers are assigned when the first constructor lands, append-only.

Level	Cell	Fields (by schema)	Authority
0	value	`Data`, `Shape`	`Tensor`
1	sort	name, constructor, extension	`EvaluatorValueSets.ValueSet` projection
1	word	token, form, category, active, implementation	`VocabularyEntry` projection
2	constructor or metaclass	kind, arity, policies	new
3	generic function, method, or protocol	name, arity, methods, policy	new

Consequences.

- Every decoder checks its tag; a wrong tag is `None` or `Error`, never a loose decode. `ValueSet.decode` and `VocabularyInImage.readEntry` already do this in part.

- `class-of` is an address lookup; `same-name` remains name equality and `same-shape` remains extensional equality (D12). The three are different questions, and the protocol must not collapse them.
- Persistence is free for the tensor; roots remain execution state (D4) and must be re-threaded. A root table is a follow-on, not part of this decision.
- The kernel exposes only `addTensor`, `lookupTensor`, `addText`, `readText`, and `EntryIdentity`; the encoding adds no host state.
- The tag registry is the single allocator; a duplicate or reused tag is a defect, not a convention.

Acceptance. A metaobject round-trips through `Persistence.toBytes` and `fromBytes`; a term, a sort, a word, and a protocol cell are distinguishable by tag; two structurally equal metaobjects at different addresses are not the same class; no metaobject is stored outside the image; the tag registry has no duplicate.

Status: recorded, partly applied. The first implementation landed as D19 (W10), which allocated `GenericTag = -4.0` and `MethodTag = -5.0` for image-resident generic functions and methods. The next implementation, once the word-metaproperty stage lands, is the word metaobject (D17 stage 2), because the vocabulary already has an image-resident encoding and a meta-level `IsActive` flag.

D17 — Metaobject Protocol scope: open implementation, fixed meta-level, kernel boundary

Decision: adopt the Metaobject Protocol as the **open implementation of the language's extension points**, not as a reflective tower and not as a wholesale CLOS clone. The protocol is a set of operations over metaobjects with default methods; adding behavior means adding a method in the image, not a case in F#. The fixed point is an explicit bootstrap kernel (D16's encoding and `docs/kernel-inventory.typ`).

Three readings.

Reading	What is refined	Verdict
Cell-system protocol (CLOS)	classes, methods, slots, method combinations	Deferred: slots, instances, class precedence, and lexical closures are absent.
Open implementation of language mechanisms	words, sorts, generic functions and methods, guard evaluators, protocol failures	**Chosen.**
Reflective tower	the evaluator itself; `eval` as a protocol	Rejected as a goal: a research project with performance and debugging costs the product does not need.

Near-term scope (stages 2–4).

1. **Words.** A `Word` metaobject carries the token, form, category, active flag, and an implementation reference. The dispatcher consults it before the vocabulary. Acceptance: a new word family needs no F#; the built-in shadowing policy is explicit.
2. **Sorts.** A `ValueSet` metaobject carries a constructor (finite, predicate, union, intersection, complement) and the protocol methods `member`, `subset`, `union`, `intersect`, `except`, and `complement`. The finite constructor is the default; a predicate constructor uses the SMT encoder

and returns `Undecided` when it cannot decide. Acceptance: a new constructor needs no F#; no silent false.

3. **Generic functions and methods.** A generic function carries a name, arity, dispatch projection, method list, and ambiguity policy; a method carries specializers and a body. Dispatch is on the top N stack operands, with the top of stack as the last argument. Applicability means all specializers hold; specificity is subset; ambiguity reuses `DecisionTable`'s `Unique`/`First`/`Priority` and `Selected`/`NoMatch`/`Ambiguous`. Acceptance: a new method needs no F#; ambiguity is never a silent pick.
4. **Guard evaluators.** `Refiner.evaluateGuard` becomes a protocol method per guard constructor, with its SMT encoder alongside it. Acceptance: a new guard kind needs no F#, and can return `Undecided`. **The SMT half is bounded:** a guard whose implementation is an arbitrary quotation has no SMT encoding, so a new guard added as data is executable and generative but not provable; the encoder reports `Unavailable` rather than approximating it. The runtime half landed as D35, the lattice bridge as D41, and the six typed relations keep their host encoders.

Deferred (named, not dropped).

- **The full Cell System protocol.** Slots, instances, class precedence, `:before`/`:after`/`:around`, `call-next-method`, and metaclasses. These need slots, an instance layout, and lexical closures; none exist. A slot decision and a method-combination decision come first.
- **Control refinement.** Quotation, `if`, `apply`, and `define` become protocol methods, which requires a first-class code cell (quotation plus environment). Today a quotation is text and `define` splices it, so methods are dynamically scoped. That is stage 5.
- **Conditions and restarts.** Protocol failures become values and witnesses; `Repair` already has the shape (a witness is a condition, a patch is a restart). Stage 6, and a separate decision if the condition protocol is wanted beyond the gate.
- **A grammar protocol.** The tokenizer's sigils and the parser's precedence are host code. That is a separate surface decision, not the Metaobject Protocol.
- **A reflective tower.** `eval` itself as a metaobject, and an infinite or self-describing evaluator tower. Rejected as a goal.

Bootstrap kernel (the fixed point). Tensor construction and structural equality; image append and lookup, text interning, `EntryIdentity`; root threading; stack push, pop, and peek; the literal cases in `executeWord`; the protocol dispatcher; the top-level `run` loop and reductions. Everything else is a candidate. The inventory in `docs/kernel-inventory.typ` is the baseline, not a promise.

Invariants.

1. The kernel is explicit and counted; growth needs a recorded reason.
2. A protocol decision returns a verdict (`Selected`/`NoMatch`/`Ambiguous`/ `Undecided`, or `Result` with a named reason), never a silent false.
3. The image is the single source of truth; a metaobject change is a new entry.
4. The child-facing sentence layer never sees metaobjects; D9's domain binding remains its surface.
5. The proof-or-witness trust boundary applies to protocol decisions.
6. Every stage has a falsifiable test: *a user adds X without editing F#.* If it fails, the stage is not a protocol yet.

Order. D15 (W4 unification) → W10 (dispatch) → a follow-on predicate and composite sort decision → D16 encoding → stages 2–4. The Metaobject Protocol is downstream of generic functions, not a shortcut around them. D12's pair is the class metaobject; D13's table is the method set without bodies; D14's registry is the metaobject table; D17 generalizes all three.

Status: recorded, partly applied. The generic-function and method substrate landed as D19 (W10): generic functions and methods are image-resident and a new method needs no F#. The runtime guard protocol landed as D35, sort constructors as D36, and the six set-operation protocol methods as D39. Stages 5–7 are deferred and named.

Status update (D51). Stage 5 landed as D51: quotation, `if`, `apply`, and `define` are control words over image-resident code cells, and a new control form is installed with `handler name control` without editing F#. The paragraph above is historical for stage 5 only; stages 6–7 remain deferred and named.

D18 — Scale Basic English: a literal controlled grammar (W2.1)

Decision: the specification sentence language is **Scale Basic English**, a controlled subset with one grammar and one canonical logical form. The parser accepts exactly that subset and refuses everything else by name. Domain words still come from the image vocabulary (D8); the sentence structure is closed.

D8 removed the hardcoded sample verb and made recognition depend on vocabulary category and marker order. That was necessary but not sufficient: a list of categories can still read `Some enemy can jump.` as a universal capability, read `No player can jump.` as an affirmative, or skip an article or a pronoun. D18 turns the implicit order into an explicit grammar and makes the executable projection partial rather than forgiving.

Basic English is a starting point, not a formal grammar

Ogden's Basic English is a restricted lexicon (a core list of 850 words, a small operator set, and defined technical terms), not a formal grammar. Simple English is a readability style, not a formal grammar either. Scale Basic English borrows their discipline — a small closed-class vocabulary and literal phrasing — and supplies the missing parts: a grammar, a canonical logical form, and named refusals.

The target is the child-facing specification sentence, not general natural language. Technical nouns, fields, and verbs are open when the image vocabulary defines them. Articles, pronouns, anaphora, disjunction, conditionals, and implicit quantifiers are not open in Stage 1.

Grammar

```
sentence   ::= quantifier subject relative? verb "."
quantifier ::= "every" | "no"
relative   ::= "that" condition ("and" condition)*
condition  ::= predicate
             | "is" field
             | "has" field comparison integer
predicate  ::= a FieldName whose surface form begins with "is "
verb       ::= a FunctionName
comparison ::= "greaterThan" | "atLeast" | "lessThan" | "atMost" | "equalTo"
```

The grammar is order-sensitive: sentence-initial quantifier, TypeName subject, optional `that` relative clause, final FunctionName verb, required full stop. A full stop is structural. There is no default subject, no default verb, and no silent skip of an unknown or inactive word.

`and` joins conditions in one relative clause. There is no `or`, so there is no precedence question to guess at. There is no nested relative clause and no condition that is itself a sentence.

Quantifiers

Surface	Canonical form	Executable pattern
`every`	universal affirmative	`Capability`
`no`	universal negative	`Invariant`
`some`	existential	refused by the projection as `UnsupportedQuantifier "some"`
`all`	not admitted	refused as `UnsupportedQuantifier "all"`; use `every`

`no` is a real polarity, not an ordinary quantifier that happens to be read as an affirmative. The emitter renders it as a negated postcondition under the same preconditions. `some` is grammatically recognized but the current emitter is universal-only; reading it as `every` would change the meaning, so the projection refuses it. `all` is outside the controlled surface because the subject is singular.

Conditions

A boolean condition is a predicate phrase whose surface form begins with `is`, canonicalized by the tokenizer (`is alive` becomes `isAlive`), or an explicit `is` followed by a field. A bare field noun is not a condition.

A numeric condition is `has <field> <comparison> <integer>`. The grammar records it structurally as `Has(field, comparison, value)`. The Stage 1 executable projection refuses it with `UnsupportedCondition` because the emitter and SMT obligation still carry a list of boolean fields, not structured comparisons. The refusal is named and staged; the parser does not guess a truthiness interpretation.

Comparisons are integer comparisons because the SMT theory is integer-valued. A decimal literal is refused with `NotAnInteger` rather than rounded.

Refusals

The parser has a named `ParseError` for every failure: `EmptySentence`, `ExpectedQuantifier`, `UnsupportedQuantifier`, `ExpectedSubject`, `ExpectedCondition`, `ExpectedComparison`, `ExpectedVerb`, `UnexpectedToken`, `MissingFullStop`, `ArticleNotAllowed`, `PronounNotAllowed`, `ConditionalNotAllowed`, `NegationNotAllowed`, `UnknownToken`, `NotAnInteger`, and `UnsupportedCondition`. The pipeline's `Parser.parse` returns `None` for any refusal; callers that need the reason use `Parser.parseDetailed` and `Parser.toParsedSentence`.

Articles (`the`, `a`), pronouns, `when`, `not`/`never`, `or`, and `all` are not admitted. `when` and `Transition` wait for event semantics. `not` and `never` are replaced by sentence-initial `no`. `or` waits for a precedence and disjunctive-obligation decision. These are decisions, not omissions.

Tokenization boundary

The tokenizer is still the surface-to-canonical boundary. Multiword forms are combined before parsing: `can jump` is `canJump`, `is alive` is `isAlive`, `greater than` is `greaterThan`, and `has health` is `has` followed by `health`. The parser therefore sees canonical tokens, and the grammar does not contain a second copy of the surface lexicon.

What this supersedes and what it does not

D18 refines D8. It does not reopen D8's vocabulary rule: the parser still does not hardcode the sample verb or the domain fields. It does not change D5's emitter shape, D9's domain binding, D10's SMT encoding, D14's sort pair, or D15's unification. It closes the sentence layer's ambiguity debt.

It does not add existential or numeric-condition execution. Those are the named next stages in `docs/prose-grammar.typ`: an existential property shape, and structured conditions in the emitter and SMT obligation. Until then, `some` and `has` are grammar-valid and executably refused.

Pinned by tests
Thirty-seven parser checks in `development/scale/tests.fs` (valid corpus, grammar-valid projection refusals, fourteen named refusals, the `no` emitter path, and the inactive-word refusal) plus one Hedgehog property that generates valid sentences and asserts their canonical pattern, subject, conditions, and verb. Suite: **275 checks (257 examples + 18 properties)**.

Status: applied, Stage 1.

D19 — Multiple dispatch: image-resident generic functions, type-first selection, static fast path (W10)

Decision: adopt multiple dispatch as the language's only polymorphism, with generic functions and methods as **image-resident metaobjects** (D16) and selection that is **type-first**: it uses a typed operand's declared sort, and infers the unique most-specific declared sort for an untyped operand. ValueSet extensions (D12/D15's `X`) are the only source of applicability and specificity; `executeWord` remains the lexical layer beneath, and a token that is neither a built-in, a user word, nor a generic function falls through to the vocabulary.

Encoding (D16 registry, append-only). Two level-3 constructors land:

Tag	Cell	Tensor schema
`GenericTag = -4.0`	generic function	`[GenericTag; nameId; arity; policy; methodId…]`
`MethodTag = -5.0`	method	`[MethodTag; arity; bodyId; specializerId…]`

Both are negative sentinels below `-1`, so `ValueSet.decode` rejects them and a generic or method can never be read as a sort. `GenericFunctionsRoot` is an association vector of generic-function entries, latest-write-wins by name; its address is execution state (D4), re-threaded like `DeclarationsRoot`, never serialized. A method carries specializer **names**, not a snapshot of the extensions, so a later declaration that grows a domain is seen by the next call.

Surface.

```
"combine" 2 generic                          \ declare name and arity
"combine" { :number :number } { add } method \ specializer signature and body
1 2 combine                                  \ invoke by token
```

`method` reads a **signature quotation** and a **body quotation**. The signature keeps repeated specializers and their argument order and lets the arity be checked against the generic; `:Name` and text references are both accepted. A method whose specializer list equals an existing method's replaces it.

Selection. With the top of stack the last argument, the dispatcher:

1. **Static fast path.** When every operand has a known sort and the policy is `MostSpecific`, an exact specializer tuple is looked up by sort name and selected directly; sort extensions are not touched. This is the "mostly static" path Julia-style specialization needs, and it is definitive under the default policy.
2. **Type-first inference.** A typed operand uses its seal's declaration. An untyped operand infers the unique most-specific declared sort that contains it; no unique minimum (none, or two incomparable or extension-equal minima) is `UnknownType`, not a guess.

3. **Dynamic fallback.** For a known sort, applicability is name equality or extension inclusion (`S ⊆ T`); for `UnknownType`, applicability is membership (`value ∈ T`). Specificity is extension inclusion; the maximal applicable methods are kept.
4. **Verdict.** `MostSpecific` selects a sole maximal method and refuses a tie with `AmbiguousDispatch`; `First` and `Unique` are encoded for the protocol's next stage but not yet exposed. No applicable method is `NoApplicableMethod`. Neither is ever a silent pick (D17 invariant 2).

A method body receives the **underlying values**: the explicit `typed` seal is dispatch metadata and is consumed at the call boundary, so a numeric method body adds numbers rather than the wrapper's tag.

Errors (named). `InvalidGenericArity`, `UnknownGeneric`, `GenericArityMismatch`, `MethodArityMismatch`, `NoApplicableMethod` (with the argument sorts), and `AmbiguousDispatch` (with the candidate specializer lists).

Consequences.

- The kernel grows by two explicit cases (`generic`, `method`) and one lookup layer (built-ins ⟶ user words ⟶ generic functions ⟶ vocabulary). Growth is recorded, as invariant 1 requires; the census is in `docs/kernel-inventory.typ`.
- A generic function cannot shadow a built-in word, and a user word can shadow a generic function; the order is a fixed but explicit policy, and D17's word stage can later make it a protocol method.
- The extension is the authority (D12): two sorts with equal extensions and different names are not distinguished by the subtype search, so `AmbiguousDispatch` is the honest verdict; an exact type name is the one nominal tie-breaker, applied only on the fast path.
- Adding a **word** still needs an F# case (D17 stage 2 is not done); adding a **method** needs none, which is D17's stage-3 acceptance.

Acceptance (met). `generic`/`method`/invocation through `development/scale/tests.fs`; inferred-sort dispatch, typed exact dispatch, subtype specificity, equal-extension ambiguity, most-specific selection, method replacement, arity errors, persistence with the root re-threaded, and body execution (underlying values); plus one Hedgehog property that dispatch selects exactly the declared sort members. Twenty-two examples plus one property.

Status: applied (D19, W10). Suite 298 = 279 examples plus 19 properties. D16's first two constructors are now allocated; the word, sort, and guard metaproperties remain D17 stages.

Status summary

Decision	Status
E1 Error semantics	recorded, applied
C4 Stack	recorded, applied
D1 Ownership lifetime	recorded, recommended B
D2 Dictionary encoding	recorded, applied (image-resident, latest-write-wins)
D3 Value typing / sorts	recorded; `nor` fixed via explicit universe; the default universe landed as `complement` over a declared sort (W3/D14)
D4 Program classification	recorded; `Program` re-scoped as control state

Decision	Status
D5 Property-test framework	recorded; Hedgehog 1.0.0 wired, `properties.fs` (19 properties)
D6 Reproducibility of the verification suite	recorded, applied (a failing suite fails the process; counterexamples are reproducible from the recheck seed)
D7 Refinement loop demonstration	recorded, demonstrated in `components/demo/demo/` (specification, emitted property, falsification, synthesized refinement); closure now proved by SMT (W14)
D8 Parser generalization	recorded, applied; recognition by vocabulary category and marker order, no hardcoded domain words; refined by D18, which makes the sentence grammar explicit and the executable projection partial
D9 The loop executes the compiler's output	recorded, **applied** (W12) — one emitter with two domain bindings, two compiled artifacts (`PlayerSpecificationPristine`/ `PlayerSpecificationRefined`), mirror removed; re-emission pinned byte-identical
D10 The verifier is an SMT solver in-process (Z3)	**applied (W13, W14)** — Lean retired as the backstop; SMT encoding from the same parsed document; differential validation; Proved/Falsified/ Unknown/Unavailable outcomes; `.lean` stub retired; the refiner's guard closure is the proved `pre ∧ guard ∧ ¬post` query
D11 Fixed-spec mode	**applied (W15)** — the mode switch, the repair bundle, and the four-rule acceptance gate; `Unknown`/`Unavailable` are reported and rejected, spec edits are rejected by rule, and `witnesses/player.json` is the checked-in corpus
D12 Set-theoretic typing	**applied (W3, D14)** — a sort is `(option Name, Extension)`; nominal reads the name and structural reads the members; `[ … ]` sets, `:Name` seals, and the structural/nominal word pairs are live. Q3 is settled toward the separate-word answer. See `docs/set-theoretic-typing.typ`
D13 Decision models	**promoted as D73** — a symbolic decision table (rows of guards plus an outcome) with Z3-proved completeness and disjointness, a pure `select`, and three distinct failure modes. The host-level `TableBinding` and per-row `DecisionTableEmitter.emit` are landed; the parser, evaluator and demo are not wired, and a Basic English table surface is not built. ML decision models are excluded from the verified path by the proof-or-witness trust boundary. See `docs/decision-models.typ`
D14 Sorts are a pair, and the pair is landed (W3)	**applied** — the image-resident sort registry (models plus declared domains), the `[ … ]`/`:Name` surface, the structural and nominal word pairs, `merge`/`complement`, and the named diagnostics (`ValueSetMismatch`, `ValueSetNotClosed`, `UnclosedSet`, `UnexpectedSetClose`, `NotASet`). Thirty-two example checks plus one Hedgehog property; suite 245 at W3
D15 Unification is first-order, image-resident, and nominal (W4)	**applied** — `unify` now walks and substitutes through operands, binds bidirectionally with a canonical variable direction, rejects direct and indirect cycles with an occurs-check, unifies compounds structurally, and unifies `typed` terms by sort name. `resolve` exposes substitution; `term` and `typed` build the terms. The substitution stays in `BindingsRoot`; a failed match is transactional. Twenty-four examples plus two properties; suite 271

Decision	Status
D16 Metaobject encoding	**recorded, partly applied** — a metaobject is a tagged tensor with address identity and explicit levels; the tag registry is append-only. The first constructors are allocated: `GenericTag = -4.0` and `MethodTag = -5.0` under D19 (W10), word metaobjects under D21, the event-stream under D29, and predicate sorts under D31. The guard metaproperty remains proposed. See `docs/kernel-inventory.typ`
D17 Metaobject Protocol scope	**recorded, partly applied** — the open implementation of words, sorts, generic functions and methods, and guard evaluators, with a fixed meta-level and an explicit bootstrap kernel. Stage 3's substrate lands as D19 (W10): generic functions and methods are image-resident and a new method needs no F#. The reflective tower and the full slot and method-combination protocol are deferred. See `docs/kernel-inventory.typ`
D18 Scale Basic English	**applied, Stage 1** — a strict sentence grammar (quantifier, subject, optional `that` relative clause, conditions, verb, full stop), `every` to `Capability`, `no` to `Invariant`, and named refusals for articles, pronouns, `when`, negation, `or`, `all`, unknown words, and malformed sentences. `some` and numeric `has` are grammar-valid and executably refused. Thirty-seven examples plus one property; suite 275. See `docs/prose-grammar.typ`. D33 later makes `some` a `Possibility` (witness predicate plus satisfiability).
D19 Multiple dispatch (W10)	**applied** — image-resident generic functions (`-4`) and methods (`-5`) under `GenericFunctionsRoot`; type-first selection with a static exact-tuple fast path, extension-subset specificity, and `UnknownType` membership fallback; a method body receives the underlying values; ambiguity is `AmbiguousDispatch`, never a silent pick. `generic`/`method` are new kernel cases. Twenty-two examples plus one property; suite 298

D20 — Channels: image-resident communication endpoints (W6)

Decision: channels are first-class image-resident entities, not host objects. A channel is an `EntryIdentity` addressing a tensor-encoded `Channel` record in the image; the scheduler's `Channels` map (in `SchedulerState`) is the registry.

Why image-resident:

- Consistency with the level model (D1/D2): all mutable state is execution state threaded through `SchedulerState`; persistent state lives in the image.
- Persistence is free: channels serialize with the image, no extra sections.
- The registry is append-only; a channel's address is its identity (D16).

Operations and outcomes:

Word	Stack effect	Outcome
`channel`	`name -> channelId`	creates channel, pushes its `EntryIdentity`
`send`	`channelId message ->`	delivers or queues; never blocks the sender in this version
`receive`	`channelId -> message`	dequeues or blocks actor (halted with `ChannelReceiveFailed`)

Word	Stack effect	Outcome
`close`	`channelId ->`	closes channel, unblocks waiters with `ChannelClosed`

Blocking is a distinct outcome, not an error. `receive` on an empty channel halts the current actor with a `ChannelWaiterType.ReceiveWaiter` payload; the scheduler treats it as a yield, not a failure. This follows E1's railway design: a non-event (would-block) is explicitly not an error.

Channel state (tensor-encoded, `ChannelTag = -6.0` reserved):

```
type Channel =
    { Identity: EntryIdentity
      MessageQueue: EntryIdentity list
      IsOpen: bool
      SendWaiters: EntryIdentity list
      ReceiveWaiters: EntryIdentity list }
```

Superseded in part by D27. The record above was the plan; the landed shape keeps the host side to control state only — the waiters — and puts the open flag and the queued messages in the image as appended state entries (`[ChannelStateTag; channelNumber; isOpen; messageId…]`). The tag allocation also moved on: `ChannelTag` is `-7.0` and the word metaobject took `-6.0`. The rest of this record stands.

Acceptance:

- Channel ID is pushed as an `EntryIdentity` tensor (test: `channel creates a channel and pushes its ID`)
- Channels survive persistence (root re-threaded like `SetsRoot`/`DeclarationsRoot`)
- No host `Map` for channels — registry is in `SchedulerState.Channels`

Non-goals (deferred to W7 ChannelEvents): `choose`/`wrap`/`guard`/`with-fallback`, buffered channels, channel ownership transfer.

Status: applied (W6). `components/prelude/evaluator/channel.fs` implements the four words; `types.fs` declares `Channel`, `ChannelOutcome`, `ChannelWaiterType`, and error cases; `SchedulerState` gains a `Channels` map. One test in `development/scale/tests.fs`.

D21 — Word Metaobjects: image-resident word definitions (D16 Stage 2)

Decision: the vocabulary's `FsharpName` seam is replaced by a `Word` metaobject carrying the token, form, category, active flag, and an implementation reference (a generic function address or a built-in tag). The dispatcher consults the word registry before the vocabulary.

Why a metaobject: D17's thesis is that language mechanisms become image-resident with a protocol. The vocabulary already has an image-resident encoding and an `IsActive` meta-level flag; the missing half is the *implementation* reference.

Encoding (D16 registry, append-only):

Tag	Cell	Tensor schema
`WordTag = -6.0`	word metaobject	`[WordTag; tokenId; formId; categoryCode; active; implRef]`

`categoryCode` uses the `WordCategory` discriminant (0=TypeName, 1=FieldName, …). `implRef` is `None` for built-ins (dispatcher matches token directly), or `Some genericFunctionId` for image-resident words.

Dispatcher change: the token lookup order becomes:

1. Built-in cases (literals, quotations, control words)
2. **Word metaobjects** (new — image-resident, user-extensible)
3. Generic functions (`generic`/`method`)
4. User words (`UserWordsRoot`)
5. Vocabulary (`VocabularyRoot`)

This makes the built-in shadowing policy explicit and lets a new word family be added without an F# case — acceptance: a user adds a word by emitting a `WordTag` tensor and appending to the word root.

Acceptance criteria (D17 invariant 2):

- A new word family needs no F# case.
- A protocol decision returns a verdict, never a silent false.
- The word metaobject round-trips through persistence.

Status: applied (D21). `components/prelude/evaluator/word.fs` implements the `word` declaration word; `types.fs` declares `WordTag = -6.0`, `Word` metaobject type, and error cases; dispatcher order updated in `components/prelude/evaluator/evaluator.fs`; `WordsRoot` added to `EvaluationState`. Tests in `development/scale/tests.fs`: declaration, consultability, invalid category/active errors, latest-write-wins, persistence round-trip. Suite: 313 checks.

D22 — Scheduler: per-process cooperative multitasking (W5)

Decision: the scheduler is per-process state threaded through `EvaluationState`, replacing the module-global mutable sketch. Each process carries its own scheduler; actors are supervision-tree nodes with explicit states (`Running`, `Ready`, `Halted`, `Terminated`).

Why per-process:

- Consistency with the level model (D1): scheduler state is execution state, not image data.
- No module-global mutable state; each process isolates its actors and ready queue.
- Enables future multi-process scenarios without global interference.

Actor model:

- `spawn` creates a child actor from a quotation; parent and child share the scheduler.
- `yield` snapshots the current actor and runs the next ready actor (round-robin).
- `halt` pauses the current actor with an optional error, retaining its state for `resume`.
- `resume` re-queues a halted actor.
- `kill` terminates an actor and all descendants.
- `actors` lists all actors with their status.

Budget fairness: `Reductions` counter per actor; exhaustion triggers `yield` (not an error). Preemption is cooperative: the current actor yields at budget exhaustion or explicit `yield`.

Blocking as distinct outcome: following E1, a non-event (would-block) is not an `Error`. ChannelEvents words (`send`/`receive` on channels) halt the actor with a `ChannelWaiterType` payload; the scheduler treats it as a yield.

Acceptance:

- Per-process scheduler state threaded through `EvaluationState` (test: preemption interleaves parent/child).
- `spawn`/`yield`/`halt`/`actors`/`resume`/`kill` words implemented.

- No module-global mutable state.
- Budget exhaustion is a yield; preemption test really interleaves parent and child.
- Suite: 11 scheduler checks (budget yield, literal budget, interleaving, spawn/yield/halt/resume/kill).

Status: applied (W5). `scheduler.fs` rewritten; `components/prelude/evaluator/scheduler.fs` wires the words; `types.fs` declares `ActorStatus`, `SchedulerState`, `Actor`.

D23 — Verify: a narrow exact-text layer for the emitter artifact (W1)

Decision. Adopt `Verify.Expecto` 30.20.1 as an exact-text regression layer for the emitter artifact, and for nothing else. The scope is **JIT only**: the snapshot runs in the granular suite; CI invokes `dotnet run -- --all-tests`; the NativeAOT gate continues to run the example suite with `--tests` and does not execute the granular suite.

Verify does not replace Hedgehog (laws, generated inputs, shrinking) or the example suite (fast, dependency-free, AOT-proven). Until D5's inline golden is migrated, the two coexist: D5 keeps `expectedGeneratedSample` in `development/scale/tests.fs` and compiles `generated/sample-properties.fs`; the Verify test pins the same emitted text through `snapshots/emitter-sample.verified.txt`.

Roles. granular is the host: it owns discovery metadata, filtering, counts, and exit status. `Verify.Expecto` is only the adapter that supplies a test's identity; Verify is the snapshot assertion engine. SageFs is a front end over granular (`list_tests`, `run_tests`), so the Verify test appears as an ordinary granular test with per-test receipts and watch-runs and needs no extra integration.

Gate wiring. `bases/scale-cli/entry.fs` parses `--granular-tests` into `GranularSuite` and runs `GranularTests.runAll`, which includes every granular list and excludes `aggregateSuite` (the example suite already runs under `--tests`; including it would run those checks twice). The granular exit code reaches the process exit code. `.github/workflows/ci.yml` and `just check` run `--all-tests`, which runs the example suite and the artifact tests once.

Verified (net10.0, SDK 10.0.401, FSharp.Core 10.1.401):

- `Verify.Expecto` 30.20.1 + `Expecto` 11.1.1 restores, builds, and runs: 47 granular tests pass; a missing baseline produces a `VerifyException` and a non-zero exit; accepting the received file makes the run green. Acceptance flow: copy `bin/Debug/net10.0/snapshots/<name>.received.txt` over `snapshots/<name>.verified.txt`, then rebuild so the `Content` copy refreshes.
- Baseline lookup is CWD-independent: `VerifySettings.UseDirectory` points at `AppContext.BaseDirectory/snapshots`, and the baseline is a `Content` item copied to the output directory.
- The diff launcher is disabled (`VerifySettings.DisableDiff`); otherwise a mismatch opens an external diff tool and blocks the process indefinitely.
- Object snapshots are excluded. `PublishAot=true` writes `System.Runtime.CompilerServices.RuntimeFeature.IsDynamicCodeSupported=false` into `runtimeconfig.json` even for Debug builds, and Argon needs `Reflection.Emit` (`PlatformNotSupportedException: Dynamic code generation is not supported`). The string path (`VerifyString`) does not use Argon.
- The example suite is unchanged: `dotnet run -- --tests` reports `ALL TESTS PASSED (390)`.

AOT scope decision. The published NativeAOT binary cannot execute the granular suite with this toolchain: it aborts with `TypeInitializationException -> TypeLoadException: Attempted to load a type that was not created during ahead of time compilation` inside

`granular.Logging`'s module initializer, reached from `ANSIOutputWriter.setColourLevel` in `granular.Tests.runTestsWithCancel`. Reproduced with granular 11.1.1 alone, without Verify; rooting the granular assembly does not change it. Therefore `--granular-tests` is JIT-only, and the AOT jobs keep using `--tests`. This is an explicit scope decision (the AOT binary runs the same production code, and the snapshot is a test artifact), not a claim that the AOT path is exercised. `Verify.Expecto` 31 and later require `FSharp.Core` 11.0.100 (.NET 11); an SDK upgrade would be needed before the AOT path could be reconsidered.

Status: applied (JIT only). `bases/scale-cli/scale.fsproj` adds `Verify.Expecto` 30.20.1 and the `snapshots/*` output copy; `development/scale/granular-tests.fs` adds the snapshot test and `runAll`; `bases/scale-cli/entry.fs` adds the `RunMode` dispatch; CI and `just check` run both suites.

D24 — Executable numeric `has` conditions

Decision. Complete the staged numeric `has` production in Scale Basic English: a sentence such as

> Every player that is alive and has health greater than 0 can jump.

now parses, emits, encodes, and repairs end-to-end. The condition carrier is a typed value, not a string list:

```
type ComparisonOperator =
    | ComparisonGreaterThan
    | ComparisonAtLeast
    | ComparisonLessThan
    | ComparisonAtMost
    | ComparisonEqualTo

type Condition =
    | BooleanCondition of field: string
    | NumericCondition of field: string * comparison: ComparisonOperator * value:
RealNumber
```

`ParsedSentence.Conditions`, `Smt.Obligation.Conditions`, `Repair.Target.Conditions`, and `Repair.CorpusEntry.Conditions` all carry `Condition list`. The comparison cases are deliberately prefixed so a module that opens `Types`, `ScaleFile`, and `DomainExpression` never resolves `GreaterThan` by accident; `Parser.SentenceComparison` is an abbreviation for `ComparisonOperator` and `Parser.Condition` for `Types.Image.Condition`.

Canonical text. `Types.Image.conditionText` renders a boolean condition as its field token (`isAlive`) and a numeric condition as `field comparison value` with invariant formatting (`health > 0`). `Types.Image.parseConditionText` parses that form and is the only parser of stored conditions. Repair bundles and the append-only witness corpus keep the existing JSON shape (a string array), so the wire format does not change; the typed value is reconstructed once when the corpus is read.

Semantics. Boolean conditions are field truth. Numeric conditions compare the domain field against an integer threshold using the same five operators as `ScaleFile.Guard`: `>`, `>=`, `<`, `<=`, `=`. The SMT theory remains integer-valued, so a non-integral threshold is `Unavailable` with the same "not an integer" refusal as a guard; it is never silently truncated. The runtime precondition (`Repair.conditionHolds`, `Demo.preconditionOf`) evaluates the same comparison over the host value.

Emitter. A numeric condition widens the generated record field to `float` (`{ Alive: bool; Health: float }`) and renders the comparison with a float literal (`player.Health > 0.0`). This is the self-contained binding's constraint: a host domain that exposes an integer field must project it as a `RealNumber`/`float` at the binding, exactly as `DemoDomain.Player.Health` does. The generated sample `generated/numeric-sample.fs` is compiled and pinned by a example golden and `snapshots/numeric-sample.verified.txt`.

Grammar boundary. `has` remains a single comparison production: `has <field> <comparison form> <integer>`. The comparison token itself is validated by the vocabulary (`greaterThan`, `atLeast`, `lessThan`, `atMost`, `equalTo`). Existential `some` is still grammatical and still refused by the executable projection (`UnsupportedQuantifier "some"`); numeric `has` is no longer refused.

Status: applied. `types.fs` adds `ComparisonOperator`, `Condition`, `conditionText`, `parseConditionText`, `mapConditionField`; `parser.fs` accepts numeric conditions; `emitter.fs`, `smt.fs`, `repair.fs`, `demo.fs`, `bases/scale-cli/entry.fs`, `properties.fs`, `development/scale/tests.fs`, and `development/scale/granular-tests.fs` carry the typed value; `generated/numeric-sample.fs` and `snapshots/numeric-sample.verified.txt` pin the artifact. Suite: 400 example checks and 48 granular tests.

D25 — The verification type lattice: one coordinate system for Z3, Hedgehog, and the refiner

Decision. The verification loop has one host-bound lattice. A field is a name, a `DomainExpression.FieldType`, an optional finite set of coordinates, and the `ScaleFile.Guard` list that narrows it. A lattice is an ordered field list. The lattice is a projection of the domain binding (D9), not a second source of truth: the `.scale` document remains the one editable specification (D7), and the lattice is never persisted.

`type-lattice.fs` is compiled between `domain-expression.fs` and `smt.fs`. It validates every finite coordinate and every narrowing threshold against the same Int32 QF_LIA boundary the solver uses (`DomainExpression.integerValue`), so Hedgehog cannot sample a value Z3 cannot represent. A symbolic field (`Coordinates = None`) can feed Z3 but not a finite generator or the refiner; an empty finite field is refused. The record representations are private; `create`, `ofFieldTypes`, `applyGuard`, and `withGuards` are the public ways to obtain a lattice.

All three readers use it:

- **Z3** reads the field sorts and `constraints`. `Smt.precondition` conjoins the lattice's guards and deduplicates a guard whose canonical `.scale` text the document already carries, so the pinned `.smt2` artifacts do not move.
- **Hedgehog** reads `coordinatesAfterNarrowing` through the typed generators. An empty or symbolic region is a named `Error` before a generator is built, not `GaveUp`.
- **The refiner** reads `candidateGuards` and the finite universe from the lattice, and writes the chosen guard back with `applyGuard`. `Refiner.refine` folds the document's existing guards into the lattice with `withGuards` before enumerating, so coverage and the returned `RefinementResult` describe the current region. `Refiner.evaluateGuard` delegates to `TypeLattice.guardHoldsValue`.

`DemoDomain.lattice` is the single description; `fields`, `smtFields`, `universe`, and the harness generators are derived from it. The initial binding uses the current lattice (the initial lattice plus the document's existing refinements); the refined binding uses the lattice the refiner returned, and the refined Z3 discharge uses an encoding built from that lattice.

The emitter enforces the projection at its boundary: before rendering, it checks that every lattice constraint is carried by the document, by canonical `ScaleFile.guardText`. A lattice narrowing the document lacks is refused, so the F# artifact (which reads the document) and the SMT artifact (which reads both) cannot state different preconditions.

Scope boundary. D25 implements the *given/precondition* target as lattice narrowing. It does not implement loop invariants (no state/transition/induction; `SentencePattern.Invariant` is the negation-of-postcondition pattern), theorem obligations or lemmas (`Smt.Outcome` has no proof log, subgoals, or lemma store), or type refinement (no D12 sort bridge; `ScaleFile.Guard` is single-field and positive). Minimality remains proved-closure plus coverage, not a lattice-theoretic least region. Property synthesis — forced properties, the permitted-but-unforced residual, mutation/oracle discrimination, and non-vacuity certificates — is the dual operation and is not part of D25; see `docs/type-lattice.typ` for Plans B and C.

Status: applied. `domain-expression.fs` owns `integerValue`; `type-lattice.fs` is new; `smt.fs`, `emitter.fs`, `refiner.fs`, `demo-domain.fs`, `demo.fs`, and `development/scale/tests.fs` are wired. Suite: 424 example checks and 48 granular tests.

D26 — ChannelEvents (W7)

Decision. Events are first-class image-resident tensors. The surface adds `send-event`, `receive-event`, `choose`, `wrap`, `guard`, `with-fallback`, and `sync`; each constructor pushes an event address, and only `sync` commits one. An event tensor is `[EventTag; kind; refs...]` with `EventTag = -8.0`:

Kind	Event	Refs
0	`NeverEvent`	—
1	`SendEvent`	channel, message
2	`ReceiveEvent`	channel
3	`ChooseEvent`	branch event identities
4	`WrapEvent`	inner event, quotation
5	`GuardEvent`	inner event, predicate quotation
6	`WithFallbackEvent`	main event, fallback event

`choose` accepts a set/model built by `[ ... ]`; `readEventList` also accepts a single event reference, and an empty set is `EmptyEventChoice`. A numeric tensor whose shape is `[1]` is an event address, so a spawned actor can name an event without sharing the parent's stack.

Selection is a readiness verdict, not a failure. `ReceiveEvent` is ready only when its channel has a queued message; `SendEvent` is ready while the channel is open; `ChooseEvent` picks the first ready branch; `GuardEvent` runs its quotation against a discarded probe state and blocks when it yields zero; `WrapEvent` is ready when its inner event is. A branch that would block is `WouldBlock`, not an `Error`. `sync` on `WouldBlock` registers the actor in `SchedulerState.EventWaiters` and halts it; a later `send` or `close` wakes every waiter, which restores `eventId sync` on that actor's stack and retries. `EventGuardFailed` and `EventGuardBudget` are genuine refusals; a false guard is not.

Commit order. `sync` reads the ready selection, commits its channel actions, then prepends every wrapper quotation's tokens to `Program`, so `wrap` sees the selected value on the stack. `with-fallback` is first-class: when a `ChooseEvent` selects one branch, the unselected `WithNack` branches commit their fallback events if those are ready, and discard them otherwise. A `guard` quotation runs in a

probe state with an empty stack and output, under `DefaultReductions`, and cannot name scheduler words (`spawn`, `yield`, `halt`, `resume`, `kill`, `actors`).

Image ownership across actors. The image is append-only and shared by every actor in a scheduler. `switchTo` now threads the live image into the resumed actor's state instead of restoring that actor's older snapshot. Without this, a child that interns a message after its parent has written entries queues an `EntryIdentity` that aliases a different parent entry; the parent's retried `sync` then reads the wrong value. The blocked-sync wakeup test is the regression: the child sends `"wake"` after the parent has built the choice event.

Errors. `NotAnEvent`, `NotAnEventList`, `EmptyEventChoice` (plus the existing `ChannelNotFound`/`ChannelClosed`) are the named refusals; `sync` on a closed receive channel is `ChannelClosed`, never a silent block.

Status: applied (W7). `components/prelude/evaluator/event.fs` owns encoding, readiness, fallback collection, commit, and the seven words; `types.fs` adds `EventTag`, the new errors, and `EventWaiters`; `scheduler.fs` adds waiter registration/wakeup and shares the image; `components/prelude/evaluator/channel.fs` wakes event waiters on `send`/`close`; `components/prelude/evaluator/evaluator.fs` dispatches the words. Suite: 441 example checks and 48 granular tests.

D27 — Review hardening: the trust boundaries hold under adversarial input

Decision. A review of the whole tree found nine places where the code could contradict the guarantees the ledger claims, mostly at the edges the review calls "the trust boundary". Each is now refused explicitly, and each refusal is pinned by a regression test. Nothing here changes the language's semantics; it removes the ways the implementation could quietly disagree with the document, the solver, or the image.

1. Inconclusive is not a pass. `Refiner.Falsification` gains a third case, `Inconclusive of reason: string`, and Hedgehog's `GaveUp` maps to it. A generator that could not produce enough admissible samples means the property was neither held nor falsified; reporting it as `Held` let a run with no evidence read as a pass. `Refiner.verdictOf` is the mapping, exposed so it is testable without running a generator, and the demo reports an inconclusive run as inconclusive (`LoopReport.Inconclusive`).

2. The `.scale` document escapes its free text. The format is line-oriented, so a newline inside a field was read back as another directive: a refinement's `origin` could carry a `specification:` line and silently replace the specification. Free text is now escaped (`\\`, `\n`, `\r`) on render and unescaped on parse, so a document round-trips exactly and no field can inject a line.

3. A threshold is emitted in full precision, or refused. A refinement's numeric threshold is an integer in the encoded theory (D25: QF_LIA, the same Int32 boundary the lattice and the generators use). The emitter renders the threshold exactly — the old `"F1"` formatting turned `health >= 1.25` into `player.Health >= 1.2`, so the compiled predicate did not implement the document — and a non-integral threshold is refused rather than rounded. The F# artifact and the SMT obligation now state the same precondition or neither does.

4. A channel's state is image-resident and append-only. The identity entry (`[ChannelTag; marker]`) is written once and never touched; the state — open flag and queued messages — is a separate entry (`[ChannelStateTag; channelNumber; isOpen; messageId…]`, `ChannelStateTag = -9.0`) appended on every change, and the newest entry is the current one. Three things follow. The image is append-only: closing a channel no longer rewrites an entry, it appends a state. A queued message survives persistence, because the queue is in the image and not only in a host cache. A

terminated actor leaves every channel's waiter list, so a later `send` cannot hand a message to an actor that will never collect it; the message is queued instead. The host keeps only the waiters: *who* is blocked is control state, like the ready queue, so it stays host-side.

5. A set literal refuses the words that change or block the actor. A literal runs its body on a fresh stack inside the current actor, so a `halt` left the actor halted while the literal carried on, and a `receive` switched actors mid-literal. `SetLiteralSchedulerWord` names the refusal.

6. An inactive word metaobject is not callable. The dispatcher checked `IsActive` for vocabulary words only, so a declared word with `active = 0` still ran. It is now refused with `InactiveWord`, like any other inactive word.

7. The boundaries validate their input. `Tensor.appendAlongAxis` refuses a tensor whose shape and data disagree, a partial row, a mismatched trailing dimension, and an empty leading dimension — it returns a tensor whose shape describes its data, or a named error. `term` bounds its arity and compares in Int64, so an extreme arity is a `StackUnderflow` rather than a multi-billion iteration loop. Persistence checks what it reads (shapes against sizes, duplicate addresses, the allocation frontier, interned texts) and reports a corrupt file as `Persistence.CorruptImageException`; it also writes through a temporary file and replaces the destination only once the bytes are written, so a failed write leaves the previous image readable.

8. The solver's answers are checked, not assumed. A concrete assignment must name every field exactly once and match each field's sort: a boolean value on a numeric field is a named `Error` instead of a Z3 sort exception, and a duplicated field is refused instead of silently contradicting itself. A model value outside the Int32 range the domain represents is `Unavailable`, not a counterexample that does not describe the satisfying assignment. The adequacy queries report an undecided query as an `Error`: "the solver could not decide" is not "the property is vacuous".

9. The release tooling cannot delete outside the artifact tree. A runtime identifier reaches the filesystem through the output path, so it is checked against a closed allowlist before anything is written or deleted, and it must be a plain name. The GitHub release workflow runs the complete gate — format, build, both suites — before it publishes anything, and the native smoke test runs the full binary, not only `--tests`.

10. The properties cover the surface. The random program generator drew from the arithmetic core alone, so the Railway-totality and determinism laws never generated a scheduler, channel, event, sort, dispatch, or word metaobject. The word list now carries the whole surface, and the dispatch property asserts that the probe really is a member of the declared sort: a successful dispatch of a nonmember no longer satisfies it.

Status: applied. `types.fs`, `tensor.fs`, `scale-file.fs`, `emitter.fs`, `refiner.fs`, `demo.fs`, `smt.fs`, `persistence.fs`, `components/prelude/evaluator/evaluator.fs`, `components/prelude/evaluator/environment.fs`, `components/prelude/evaluator/channel.fs`, `components/prelude/evaluator/event.fs`, `components/prelude/evaluator/value-sets.fs`, `scheduler.fs`, `properties.fs`, `development/scale/tests.fs`, `justfile`, and the two workflows. Suite: 471 example checks and 48 granular tests.

D28 — Thin follow-ons: the fixed-spec CLI and the no-condition subject

Decision. Two items the W15 record named as thin follow-ons are landed without changing the language's semantics.

1. The fixed-spec CLI. `scale --fixed-spec` runs the demonstration target in fixed-spec mode. The specification is held fixed: the command never edits `components/demo/demo/player.scale`, never

re-emits the generated artifacts, and never appends a refinement. The compiled pristine predicate is the property under test; a Hedgehog witness is turned into a repair bundle at `artifacts/repair-bundle.json`, built by the same `Repair.makeBundle` the suite exercises, and the run prints the bundle path. Exit codes: `0` when the fixed specification holds, `1` when it is falsified or the run is inconclusive, `2` when the run cannot start. This is the reporting path, not an acceptance gate: accepting a patch against the bundle stays `Repair.accept`, which is exercised in-process. The run mode lives in `cli.fs`, so the argument grammar is testable without running the program.

2. A subject with no conditions. "Every player can jump." is grammatically valid and has an empty condition list. The self-contained emitter used to render a record with no fields (`type Player = {  }`), which is not valid F#. A subject with no fields now emits a one-case union used only as a marker (`type Player = Player`), and the property is `true ==> canJump player`.

The subject type also now carries every field the artifact names — the sentence's conditions and the refinements' guards, deduplicated — so a repeated condition emits one field and a refinement-only field is declared. One field with two sorts is refused: emitting it would produce a record F# cannot compile. The pinned artifact `generated/no-condition-sample.fs` is compiled into the project, so the no-condition output is proven to compile, not only asserted as text.

Status: applied. `cli.fs` (the argument mode), `bases/scale-cli/entry.fs` (the fixed-spec branch and exit codes), `demo.fs` (`runFixedSpec`, `reportFixedSpecification`, `specPath`, `bundlePath`), `emitter.fs` (the subject type), `generated/no-condition-sample.fs` (the pinned artifact), `development/scale/tests.fs`, and the docs. Suite: 480 example checks and 48 granular tests.

D29 — The event-stream as the program (W8)

Decision. Every executed token appends a step to an image-resident, append-only history. A step is `[StepTag; location; baseline; position; tokenId; stackId; programId]` with `StepTag = -10.0`: the program-counter metadata the split-state decision (D4) keeps out of the image for *data*, the token, the stack before the token ran, and the program the step resumes. The stack and the program are image vectors of addresses. The history root is a vector of step addresses, threaded in `EvaluationState.HistoryRoot` like the other roots: the steps are data, the pointer is execution state.

Rollback. `EvaluatorHistory.rollbackTo` restores the stack, the program, and the program-counter metadata from a step, and re-points the history root at the steps before it (a new appended vector). The image is never rewritten: the later steps stay, unreachable, which is the append-only discipline. The `history` word pushes the step addresses recorded before itself; `rollback` pops a step index and restores it. A batch program that rolls back and re-runs the same suffix is a loop by construction — that is the program's semantics, not the rollback's — so the mechanism is `rollbackTo`, which a debugger calls, and the word's refusal path (an out-of-range step) is pinned by a test.

Bookmarks are absorbed. A bookmark is a degenerate one-point event: `bookmark` records a step and pushes `[location; baseline; position; stepId]`; `jump` restores through `rollbackToStep`. There is no second rollback implementation and no separate bookmark state.

Boundary. The history covers the top-level `run` loop. The inner interpreter a set literal runs, and a `guard` probe, are expressions rather than program steps, so they do not record. The reduction budget still yields, so a runaway program behaves exactly as before.

Status: applied. `components/prelude/evaluator/history.fs` (steps, the history word, rollback), `types.fs` (`StepTag`, `HistoryRoot`, `InvalidHistoryStep`, `NotAStep`), `components/prelude/evaluator/evaluator.fs` (recording before each token, the two words), `components/prelude/evaluator/bookmark.fs` (absorption), `development/scale/tests.fs`, and `properties.fs` (the

generator includes `history`; `rollback` is excluded because a rolling-back program loops by construction). This landing also adds the granular integration and Verify snapshot suites: the fixed-spec CLI, the refinement loop end to end, and snapshots of the no-condition artifact, the repair-bundle wire format, and the escaped `.scale` document. Suite: 495 example checks and 53 granular tests.

D30 — Linear ownership (W9)

Decision. Ownership is per-run execution state (D1, Option B). `EvaluationState.Ownership` holds `Owners: Map<EntryIdentity, EntryIdentity>` (value ⟶ owning actor) and `Consumed: Set<EntryIdentity>`; it is never serialized. The surface is four words:

- `own` takes ownership for the current actor. It is idempotent for a value the actor already owns; a consumed value, or a value another actor owns, is a named refusal.
- `borrow` is a checked read: the value must be owned by the current actor and not consumed. The reference stays on the stack.
- `consume` ends the value's life for the run: a second consume, or a borrow afterwards, is `ValueConsumed`. The entry stays in the append-only image.
- `release` gives ownership up without consuming; releasing a consumed value only drops the ownership record.

A spawned actor starts owning nothing: ownership never transfers implicitly (W9). The checks are dynamic and live at the ownership words — the image is never rewritten, and other words can still read an entry. That is the honest limit of ownership without a static type system; the diagnostics (`NotOwned`, `OwnedByAnother`, `ValueConsumed`) state the rule and the next action.

Why execution state. D1 recommended B: ownership is a property of a running process, not of the stored world. A persisted image carries no ownership, and a resumed run starts owning nothing. A test pins that the image round-trips but the ownership does not.

Status: applied. `components/prelude/evaluator/ownership.fs` (the four words and their named refusals), `types.fs` (`OwnershipState`, `emptyOwnership`, the three error cases), `components/prelude/evaluator/evaluator.fs` (the initial state and the dispatch), `scheduler.fs` (a spawned actor owns nothing), `properties.fs` (the generator includes the four words, so totality covers them), `development/scale/tests.fs`. Suite: 505 example checks and 53 granular tests.

D31 — Predicate sorts (D16/D17 stage 3)

Decision. A sort is either a finite extension (D14) or a **predicate sort**: a base extension narrowed by an image-resident quotation. The predicate sort is encoded as `[PredicateSortTag; name; quotation; member…]` with `PredicateSortTag = -11.0`; the name and the members keep the finite form's meaning, and the quotation is the narrowing. The tag continues the single append-only registry below `-1`.

Surface. `base { predicate } :Name predicate-seal` creates it. The predicate runs with the candidate on the stack and must leave a numeric verdict; nonzero is a member, zero is not. Membership is a three-way verdict:

```
type Membership = Member | NotMember | Undecided of reason: string
```

An evaluation error, an exhausted budget, a scheduler word, or a predicate that leaves no verdict is `Undecided`, never a silent `false`. `is`, `member`, and `subset` read the verdict; `subset` decides

memberwise (the left's predicate filters its base, the right's verdict decides membership) and propagates `Undecided`.

Boundary. Enumeration is refused by name: `union`, `intersect`, `except`, `same-shape`, `merge`, `complement`, `and`, `or`, `nor`, and `admit` cannot enumerate a predicate sort's extension, so they return `ValueSetUndecided` rather than a half-set. `seal` and `unname` preserve the predicate; `same-name` and `sort-of` read the name. A predicate sort used as an event choice list is `NotAnEventList`, since a branch list is an enumeration.

Why. This is D16/D17 stage 3's falsifiable test: a user adds a sort constructor without editing F#. The quotation is image data and runs in a probe state — a fresh stack with the candidate, an empty output, a reduction budget, and the same actor-word refusal as the event guard — so a predicate cannot block or switch the actor it is deciding about.

Status: applied. `types.fs` (`PredicateSortTag`, `ValueSetUndecided`), `components/prelude/evaluator/value-sets.fs` (`ValueSet.Predicate`, the predicate encoding and decoding, the membership verdict, `predicate-seal`, verdict-aware `member`/`subset`, the enumeration refusals), `components/prelude/evaluator/vivid.fs` (`is`, `admit`, `and`/`or`/`nor`), `components/prelude/evaluator/event.fs` (the event-list refusal), `components/prelude/evaluator/evaluator.fs` (the dispatch), `properties.fs` (the generator), `development/scale/tests.fs`. Suite: 515 example checks and 53 granular tests.

D32 — Classification: one membership verdict, `class-of`, entry kinds

Decision. Classification is one verdict and one query.

One verdict. D31 introduced `Member`/`NotMember`/`Undecided reason`. Dispatch now uses it too: type inference and specializer matching call `EvaluatorValueSets.memberOf`, so a predicate sort participates in dispatch instead of being silently read by its base extension (the gap D31 left). An undecidable predicate is `DispatchUndecided`, never a silent no-match; method specificity compares verdict-aware subsets.

One query. `class-of` pops a value and pushes `[ClassTag; status; payload…]` (`ClassTag = -12.0`):

status	meaning	payload
0	Unknown	—
1	Class	the sort's name identity
2	Ambiguous	the candidate name identities
3	Undecided	the reason's text identity
4	Metaobject	the entry-kind code

`EvaluatorDispatch.classify` is the shared query: an explicit `typed` seal wins, otherwise the unique most-specific declared sort is inferred, or the verdict is `TypeUnknown`/`TypeAmbiguous`/`TypeUndecided`. `argumentType` maps that verdict for dispatch, so the word and the dispatcher cannot disagree.

Entry kinds are derived from tags, not stored: word, generic function, method, channel, channel state, event, step, and predicate sort. A finite sort carries no tag of its own — its role is conferred by a root — so `class-of` reports the value class for untagged entries. That is the boundary that keeps the query total without adding a field to every tensor.

The verification lattice is not structurally bridged. `EvaluatorValueSets.ValueSet` classifies *values in the image*; `TypeLattice.FieldType` classifies *fields of a verification subject.* They are

different projections of "sort": one is a set of values, the other a coordinate system over a record. Forcing one onto the other would conflate a value set with a field and would need an ambiguous value⟶field-sort guess. What they share is the verdict vocabulary and the class query; the lattice keeps its finiteness and encodability boundaries (D25).

Status: applied. `types.fs` (`ClassTag`, `DispatchUndecided`), `components/prelude/evaluator/dispatch.fs` (the verdict, `classify`, `entryKind`, `class-of`, and verdict-aware selection), `components/prelude/evaluator/evaluator.fs` (the dispatch and the word), `development/scale/tests.fs`. Suite: 532 example checks and 53 granular tests.

D33 — Existential `some` (D18 stage 1)

Decision. `some` projects to `SentencePattern.Possibility`. The artifact and the obligation change shape, because a possibility is not a universal implication.

F# artifact. The emitter renders a **witness predicate** `pre && post`, named `some`, with a comment saying a harness searches for a subject satisfying it. A single failing subject is not a defect, so this artifact must not be read as a property. The universal and invariant paths are byte-unchanged.

SMT obligation. `Smt.encode` is `pre ∧ post` for a Possibility: it is satisfiable iff a witness exists. `discharge` returns `Witnessed model` on a satisfiable formula and `NoWitness` when it is unsatisfiable; `Unknown` and `Unavailable` stay honest. The universal path remains `pre ∧ ¬post` (`Proved`/`Falsified`). `satisfiablePre` and `tautologicalPost` are unchanged: they ask about the precondition and postcondition, not the quantifier.

Boundary. Fixed-spec acceptance is universal-only: `Repair.accept` rejects a possibility target by name, and the witness corpus refuses a Possibility entry. The refiner is unchanged — a possibility has no counterexample to shrink.

Pinned. A compiled witness artifact (`generated/possibility-sample.fs`), the parser projection, the `Witnessed`/`NoWitness` outcomes, and the two refusals.

Status: applied. `types.fs` (`Possibility`), `parser.fs` (the projection), `emitter.fs` (the witness predicate), `smt.fs` (`Outcome` and the pattern-aware encoding/discharge), `repair.fs` (pattern text/reader and the acceptance refusal), `demo.fs` (the fixed-spec outcome match), `generated/possibility-sample.fs`, `development/scale/tests.fs`, and the docs. Suite: 541 example checks and 53 granular tests.

D34 — The filtering word for comprehensions

Decision. `sort { predicate } filter` narrows a sort to the members its predicate accepts. The predicate is the same probe the predicate sorts use (the candidate on the stack, a reduction budget, actor words refused), and the membership is the one verdict: base membership first, then the filter predicate. An `Undecided` predicate is `ValueSetUndecided`, never a silent drop. The result is an anonymous structural sort.

Boundary. This is the filtering word comprehension syntax needs, not the surface form. `[ ?x : ?x > 1 ]` stays deferred because it has no domain: a comprehension must range over a declared sort, and how the domain and variable are written is still an open surface decision. The `[ … ]` literal still evaluates its body.

Status: applied. `components/prelude/evaluator/value-sets.fs` (`filterWord`), `components/prelude/evaluator/evaluator.fs` (the dispatch), `properties.fs` (the generator), `development/scale/tests.fs`. Suite: 546 example checks and 53 granular tests.

D35 — Guard metaobjects (D16/D17 stage 4, guard half)

Decision. A guard is an image-resident metaobject `[GuardTag; nameId; quotationId]` with `GuardTag = -13.0`; the quotation is the implementation. `"name" { predicate } guard-define` declares one; `value guard guard-holds` runs the quotation in the shared probe (the value on a fresh stack, a reduction budget, actor words refused) and pushes `1` or `0`. A probe error is `GuardUndecided reason`, never a silent false; a reference that is not a guard is `NotAGuard`. `entryKind` and `class-of` recognize guard entries (kind code 9).

Why. This is D16/D17 stage 4's guard half and its falsifiable test: a new guard is added as data, not as an F# case. The probe is the one predicate sorts use (D31), so guard and sort membership cannot drift; the probe itself moved to `components/prelude/evaluator/probe.fs` and `EvaluatorValueSets` now calls it.

Boundary. This lands the *runtime* guard protocol. The verification pipeline's `ScaleFile.Guard` remains the typed SMT/emitter projection of the six relations. **D41 later bridged the lattice:** a document refinement may name a guard metaobject (`<field> guard <name>`), and the lattice evaluates it through `EvaluatorRuntimeGuard.resolver`. Because the implementation is a quotation, it has no SMT encoding; a named guard is executable and generative but cannot be proved, and the encoder and emitter refuse it by name. The six typed relations keep their host encoders.

Status: applied. `types.fs` (`GuardTag`, `NotAGuard`, `GuardUndecided`), `components/prelude/evaluator/probe.fs` (the one probe), `components/prelude/evaluator/value-sets.fs` (the predicate uses it), `components/prelude/evaluator/guard.fs` (the two words), `components/prelude/evaluator/dispatch.fs` (entry kind and class code), `components/prelude/evaluator/evaluator.fs` (the dispatch), `properties.fs` (the generator), `development/scale/tests.fs`. The kernel-inventory headline is corrected to the full dispatcher count (78) in the same change. Suite: 552 example checks and 53 granular tests.

D36 — ValueSet constructors: named, image-resident narrowing (D16/D17 stage 3, constructor half)

Decision: a sort constructor is an image-resident metaobject `[ConstructorTag; nameId; quotationId]` (`ConstructorTag = -14.0`). The `constructor` word creates one; `constructor-seal` builds a named sort whose base extension is the operand sort and whose narrowing runs the constructor's quotation in the shared probe. A sort's existing narrowing slot holds either a raw predicate quotation (D31) or a constructor reference (D36); `memberOf` resolves either to a quotation. `entryKind` and `class-of` recognize constructor entries (kind code 10).

Why. This is D16/D17 stage 3's falsifiable test: a new constructor is added as data, not as an F# case. It reuses D31's probe and D35's refusal shape, so a constructor cannot decide silently and an undecidable constructor is `ValueSetUndecided`. The constructor entry lives in the image and is referenced from the sort entry, so there is no new root and persistence is free.

Options considered.

1. **Hard-code more constructors in F#.** Rejected: it fails the acceptance test and keeps the extension point in the kernel.

2. **Keep only anonymous `predicate-seal`.** Rejected: a predicate sort is a good default, but the constructor is not a reusable metaobject and cannot be inspected by `class-of`.
3. **A named constructor metaobject.** Chosen: the constructor is data, a sort names it, and the same constructor can narrow many base extensions.

Boundary. The constructor decides membership; it does not replace the set-operation policy. `union`, `intersect`, `except`, `same-shape`, `merge`, and `complement` still enumerate the base extension through the kernel defaults, so they refuse a constructor sort with `ValueSetUndecided` rather than approximate it. D39 later moved the six public operations to protocol methods; the default methods still use these primitives. The verification pipeline's typed `ScaleFile.Guard` and the guard verification-lattice integration remain separate.

Acceptance. A constructor is defined and used without editing F#; membership answers `1` or `0`; an undecidable constructor is a named `ValueSetUndecided`; `constructor-seal` refuses a non-constructor with `NotAConstructor`; a constructor sort survives persistence with its constructor reference intact; `class-of` reports kind code 10. The suite adds ten checks (562 example, 53 granular).

Status: applied. `types.fs` (`ConstructorTag`, `NotAConstructor`), `components/prelude/evaluator/value-sets.fs` (`constructor`, `constructor-seal`, constructor encoding and resolution), `components/prelude/evaluator/dispatch.fs` (entry kind and class code), `components/prelude/evaluator/evaluator.fs` (the dispatch), `development/scale/tests.fs`. The kernel inventory moves the dispatcher headline from 78 to 80 explicit cases and adds the constructor row. Suite: 562 example checks and 53 granular tests.

D37 — Value sets, field types, and set roots: naming the thing, not the mechanism

Decision: the code names the evaluator concept a **value set**: `EvaluatorValueSets.ValueSet` in `components/prelude/evaluator/value-sets.fs`. A field's domain type is `FieldType` (`BooleanType`/`IntegerType`) in `DomainExpression` and `TypeLattice`; the lattice field is `Type`, not `Sort`. The execution-state pointers are `SetsRoot` (subsets) and `DeclarationsRoot` (declared domains), and the operations are `readSet`, `writeSet`, `pushSet`, `setsCount`, `derivedSet`, and `setLabel`. The old names `Sort`, `FieldSort`, `ModelsRoot`, `SortsRoot`, `readModel`, `writeModel`, and `pushModel` are gone from the code.

Why. `Sort` read as ordering, and it named three different things: an evaluator value set, a field's domain type, and a constructor metaobject. A newcomer could not tell whether the evaluator sorts something. `Model` was worse: it named a subset of a value set, the root of the set store, and an SMT assignment. `FieldSort` embedded the same overload. The code now says what the thing is; the prose may still use *sort* as the type-theory term, defined as a named value set.

Boundary. This is a code and documentation rename, not a change to the surface language. The words `model`, `sort-of`, `class-of`, and `guard` keep their current spellings and behavior. The SMT assignment type and the remaining qualification of `Event` and `Guard` are separate naming decisions.

Acceptance. No bare `Sort`, `FieldSort`, `ModelsRoot`, `SortsRoot`, `readModel`, `writeModel`, or `pushModel` remains in code or documentation. The build is clean and both suites pass. Set membership, persistence, dispatch, and the SMT/lattice consumers are unchanged in behavior.

Suite. 562 example checks and 53 granular tests.

Status: applied. types.fs, components/prelude/evaluator/value-sets.fs, components/prelude/evaluator/dispatch.fs, components/prelude/evaluator/set-algebra.fs, components/prelude/evaluator/channel-event.fs, components/prelude/evaluator/evaluator.fs, type-lattice.fs, domain-expression.fs, development/scale/tests.fs, bases/scale-cli/scale.fsproj, and the Typst documents.

D38 — Identity and ChannelEvents: full words, no borrowed acronym

Decision: EntryIdentity becomes EntryIdentity; Identity, identity, and identities become Identity, identity, and identities. The concurrency pattern called ChannelEvents is named **ChannelEvents**: first-class synchronous events over channels, with send-event, receive-event, choose, wrap, guard, with-fallback, and sync.

Why. Identity is an abbreviation of identity, and identity is the concept: an EntryIdentity is the store entry's identity. ChannelEvents is a borrowed acronym from another language community; ChannelEvents says what the pattern is and matches the existing ChannelEvent type. The naming philosophy is recorded at the top of tools/rename-names.fsx.

Boundary. The wire format keeps its stable keys (specHash, and external API fields such as sessionId). The rename is source and documentation; it does not change a serialized key or a user-facing .scale directive.

Acceptance. No EntryIdentity, bare Identity, bare identity, bare identities, or ChannelEvents remains in the source or the Typst documents except the historical alias table in docs/glossary.typ.

Suite. 562 example checks and 53 granular tests.

Status: applied. tools/rename-names.fsx stores the mapping and the philosophy; the source, tests, and Typst documents use the new names.

D39 — Set-operation protocol methods: the operation is an image method

Decision: member, subset, union, intersect, except, and complement are image-resident generic functions, seeded by initial with a default method specialized on the empty set declaration. A set entry is classified as set by dispatch, so the operation is selected from the image; the default method body calls the kernel primitive of the same operation (set-member, set-subset, set-union, set-intersect, set-except, set-complement). member and complement have arity 1; the other four have arity 2. member leaves the value below the set, so the method sees both.

Why. D17 stage 3 makes the sort metaobject carry its protocol methods. Before D39 the six operations were explicit executeWord cases, so replacing one meant editing F#; now a method can be replaced with generic/method, and the default is only a body. The set declaration is empty on purpose: a set is classified by being a set entry, not by being a member of another extension, so the constructor and the operation table stay independent.

Boundary. The kernel primitives still implement the finite, predicate, and constructor policies in F#. D39 moves operation *selection* into the image; replacing a primitive still needs an F# change. A constructor sort still refuses enumeration through the primitive with ValueSetUndecided. A non-set operand now reaches dispatch and is refused as NoApplicableMethod rather than NotASet, which is the protocol's named refusal.

Acceptance. All six operations are seeded generics; the existing set tests pass through the generic path; replacing `union` in the image changes its result; a non-set complement has no applicable method.

Suite. 564 example checks and 53 granular tests.

Status: applied. `components/prelude/evaluator/dispatch.fs` (seeding and set classification), `components/prelude/evaluator/evaluator.fs` (public cases removed, kernel primitives named), `development/scale/tests.fs`, `docs/glossary.typ`, and the roadmap.

D40 — Speech-first names: Scale is the product, Lia is the mascot

Decision: names are chosen for a speech-to-text engine as well as a compiler. The product is **Scale** **Lia** is the mascot and appears only in the index and historical notes. Code and generated artifacts use Scale.

Old	Current
`withNack` / `nack`	`with-fallback` / fallback
`Reifier` / `reification`	`Refiner` / refinement
`snoc`	`appendElement`
`BAG`	`Vocabulary`
`F# name`	`Implementation name`
`FixedSpec`	`FixedSpecification` in code; `--fixed-spec` stays the CLI flag
`zonk` in our prose	"apply the substitution"

Why. `withNack` is an acronym (negative acknowledgement) and `nack` sounds like "knack"; a fallback event says what the event is. `Reifier` is a rare nominalization; `Refiner` is an ordinary word and matches `Refinement`. `snoc` is `cons` reversed, `BAG` is a stale metaphor, and `F# name` is an old label. `Lia` is ambiguous when spoken; Scale is the product name and Lia is a mascot, not a synonym.

Kept. `SMT` (satisfiability modulo theories), `AOT` (ahead-of-time), `CLI`, `MCP`, `LSP`, `JSON`, and `Z3` are established technology names. The glossary expands SMT on first use. `Kitten.Zonk` stays verbatim: it is an external API name, a citation, not one of our terms. The same applies to external product and project names.

Acceptance. `dotnet fsi tools/rename-names.fsx --check` is clean; `Lia` is confined to mascot and historical prose; no current `withNack`, `nack`, `Reifier`, `snoc`, `BAG`, or `FixedSpec` remains in the source or the docs.

Suite. 564 example checks and 53 granular tests.

Status: applied. `README.typ`, `docs/glossary.typ`, the emitted artifacts, and the source; `tools/rename-names.fsx` records the mapping.

D41 — The guard-lattice bridge: a document refinement can name an image-resident guard

Decision. A `.scale` refinement may name an image-resident guard metaobject (D35) instead of stating one of the six typed relations:

```
type RefinementGuard =
    | GreaterThan of field: string * value: RealNumber
    | AtLeast of field: string * value: RealNumber
    | LessThan of field: string * value: RealNumber
    | AtMost of field: string * value: RealNumber
    | IsTrue of field: string
    | IsFalse of field: string
    | NamedGuard of field: string * name: string
```

The surface is `refinement: <field> guard <name>`. The name is resolved by the evaluator, not by the host pipeline: `TypeLattice.GuardResolver` is `name -> FieldValue -> GuardVerdict`, and `EvaluatorRuntimeGuard.resolver` backs it with the image-resident guard's quotation. The verdict is three-way:

```
type GuardVerdict = GuardHolds | GuardDoesNotHold | GuardUndecided of reason: string
```

The lattice operations that evaluate guards therefore have resolver-aware forms: `coordinatesAfterNarrowingWithResolver`, `assignmentsWithResolver`, `universeWithResolver`, `applyGuardWithResolver`, `withGuardsWithResolver`, and the matching `generator...WithResolver` functions. The existing names remain the typed-only path; they refuse a named guard by name rather than reading it as an unknown value.

Why. D35 landed the runtime guard protocol and left the verification-lattice half open. D17 stage 4 calls for guard evaluation to be a protocol: a new guard kind is data, and a protocol decision returns a verdict. D41 is the seam between the two: the lattice asks the same image-resident guard the runtime asks, so a field value cannot get one answer from `guard-holds` and another from the refiner or generator. The falsifiable test is met at the evaluation seam: adding a guard needs a quotation in the image, not an F# case.

Options considered.

1. **Encode the quotation as SMT.** Rejected: a Scale quotation is a word program, not a term in the QF_LIA theory. Writing a second symbolic interpreter would be a second semantics, exactly the translation debt D10 exists to avoid.
2. **Leave the runtime guard and the lattice separate.** Rejected: the same refinement could then hold for the runtime and fail for the refiner, with no boundary that notices.
3. **A resolver seam with honest refusals.** Chosen: the lattice can use a named guard where an image is attached; every backend that cannot decide it says so.

Boundary. A named guard is executable and can drive the finite generators and the refiner's coverage. It is **not** provable by Z3: `Smt.refinementGuardToBooleanExpression` returns an `Error` naming the absent encoding, so a discharge is `Unavailable`, never a silent omission. The emitter refuses a named guard for the same reason, so the F# artifact and the SMT obligation cannot state different preconditions. The refiner still proposes only typed guards, and the typed-only lattice path refuses a named guard with a named reason. Nothing is persisted: the guard metaobject is an image entry (D35), and the lattice remains a projection of the document (D25).

Acceptance. A named-guard refinement renders and parses. `EvaluatorRuntimeGuard.resolver` runs the quotation and answers `GuardHolds`/`GuardDoesNotHold`; a missing or undecidable guard is `GuardUndecided reason`. The named guard narrows the demonstration lattice to exactly the same members as the equivalent typed guard. The typed-only path, the SMT encoder, and the emitter each refuse it by name. Twelve checks pin the behavior.

Suite. 576 example checks and 53 granular tests.

Status: applied. `scale-file.fs` (the `NamedGuard` case, text, parse), `type-lattice.fs` (`GuardVerdict`, `GuardResolver`, the resolver-aware guards, coordinates, generators, assignments, universe, `applyGuard`, and `withGuards`), `components/prelude/evaluator/runtime-guard.fs` (`findByName`, `valueTensor`, `evaluateByName`, `resolver`), `smt.fs` and `emitter.fs` (named refusals), `refiner.fs` (the verdict), and `development/scale/tests.fs`.

D42 — DSPy-shaped model-backed programs are an optional, prepared track

Decision. The DSPy proposal is an additional, optional feature set, not a change to Scale's semantics. It is recorded here so it can be promoted deliberately, and it is **not applied**: no source, no dependency, no new word. The documentation is `docs/dspy.typ`.

The fitting rule is: **the signature is a projection of the `.scale` contract, not a second source of truth**. The parsed sentence already names the subject, its typed fields, and the verb; the verification lattice (D25) already describes the input space. A signature adds an instruction and a few-shot example list to that projection. It does not add a parallel type system.

The six invariants of the fit.

1. **One model-touching word.** `predict` is an edge word; every model call goes through it. It returns a named `Result` refusal, never an exception.
2. **The model is an effect.** The adapter is a host port passed at the run boundary, not image state. A call is recorded in an optional replay cassette; the existing determinism law is stated over the recording or excludes the word by name.
3. **The fast metric is a search signal.** Type check, value-set membership, and property tests rank candidates. They never accept a program.
4. **The slow metric is the existing gate.** `Smt.discharge` plus `Repair.accept`: the target proof, the witness corpus, the spec hash, and the differential check. `Unknown` and `Unavailable` never pass.
5. **Optimization appends; it does not rewrite.** The optimizer searches instruction text and examples only. It never edits the `.scale` document and never mutates the program; its output is a new append-only artifact with provenance. This is fixed-spec mode (D11) applied to presentation.
6. **Constrained decoding is graded.** Only the structural fragment (field names, booleans, finite value sets) can drive a decoding grammar. Predicate and constructor sorts are post-parse predicates and may answer `Undecided`; symbolic fields cannot be enumerated. No grammar is promised for a quotation.

Synergies. The parsed sentence is the signature; the lattice is the input schema; Hedgehog shrinking plus the repair bundle is a sharper reflection signal than a full trace; `Repair.accept` is the slow metric; the event-stream (D29) supplies record and replay; and the Metaobject Protocol (D16/D17) gives the signature a data representation without a new kernel case. Nothing in the existing pipeline has to move.

Contrasts. The verified theory stays the finite typed fragment; prompt text is outside SMT. Model calls are nondeterministic, so they need replay. Transcripts may contain user data and the image cannot delete, so they live in an optional redacted cassette, never in the image. The optimizer is host code, not a kernel primitive, and the core project takes no model SDK dependency. DSPy's `compile` is called **optimize** here, because Scale's compiler is the emitter.

Promotion criteria. A signature that projects to the document and the lattice; one `predict` word with named refusals and record/replay; a fast metric that is search-only and a slow metric that

reuses the existing gate; an optimizer that appends a new artifact and never edits the specification; differential tests over a recorded transcript; and no change to a domain that does not opt in.

Non-goals. Training or fine-tuning; proving the model, the prompt, or the instruction; a reflective evaluator; network access in the kernel; provider-specific constrained decoding as a guarantee; unredacted transcripts by default; and the fast metric as an acceptance oracle.

Status: prepared, optional, not applied. Only documentation: `docs/dspy.typ`, this record, the roadmap's prepared section, the ledger row, and the README map. No change to `bases/scale-cli/scale.fsproj`, the image, the evaluator, the scheduler, the verifier, or the suite. A project that never opts in is unaffected.

D43 — The full cell system: classes, slots, instances, precedence, combination, closures, metaclasses

Decision. The full cell system is a **prepared, deferred** track, not an applied change. It builds on the metaobject substrate (D16, D17, D19, D21, D31, D35, D36, D39, D41) and adds the cell layer: classes, slots, instances, class precedence, nominal class dispatch, method combination, closures, and metaclasses. The design is `docs/cell-system.typ`; the glossary is the short description of each term.

The term. The cell-system concept is a **class**. The old `ClassTag` and `class-of` classification were renamed; the current value classification is `ValueTag` and `value-of` (D37, D40). A class's extension is a value set, and the code type for a value set remains `ValueSet`; there is no second type system. The cell query is `cell-class`, not the retired `class-of` spelling.

The surface is Basic English. The cell system is declared in Scale Basic English, not in tensor shapes. The tags and tensor entries in the design note are internal image encodings; a user never writes them. The exact grammar for class and slot declarations is a later decision, and until it lands those sentences are not in the admitted corpus. The surface is the contract; the encoding follows it.

The six design rules.

1. **Class.** A class is an image-resident metaobject that owns a name, a superclass list, a slot list, a metaclass, and a method-combination policy.
2. **Slot.** A slot is an image-resident definition with a name, a value set, an optional default, and an initialization rule.
3. **Instance.** An instance is a value with stable address identity and an appended state: the identity entry is written once, and every slot write appends a new state entry, exactly like channel state (D27). Unbounded history is the honest cost; immutable values or an explicit compaction decision are the alternatives, never a silent rewrite.
4. **Precedence and dispatch.** Class precedence is a deterministic linearization of the superclass graph. Dispatch on a class is nominal (class identity plus precedence), not extensional, so it is stable as instances are added. Ambiguity and no-method are named verdicts.
5. **Methods and closures.** Methods gain a qualifier, and a method combination orders them. Closures are image-resident code cells with an image-resident environment chain, so lexical scope does not become host state. Closures (stage 5) come before real method combination (stage 7).
6. **Metaclasses.** Metaclasses make classes instances of a class, with a finite cell-class closure terminating at a root metaclass. It is not a reflective tower.

Prepared stages. The work is staged so each step has a falsifiable test: encoding and roots; closures and lexical scope; slots and instances; class precedence and nominal dispatch; method qualifiers and combination; metaclasses; conditions and restarts. The design note lists the acceptance test for each

and the next free decision-record numbers. A first useful slice is encoding, closures, slots/instances, and precedence/dispatch.

Boundary. The cell system uses runtime value sets, not the verification `TypeLattice.FieldType`; the two coordinate systems stay separate (D32). It adds no SMT obligation and no prompt/model dependency. The image remains append-only, execution state is never serialized, failures are values, and an undecidable slot type or dispatch is `Undecided`, never a silent false. The kernel grows only by the cell primitives; a new class, slot, method, combination, or metaclass is data.

Status: prepared, deferred, not applied. Documentation only for the full system. The encoding stage landed as D44; the behavior stages remain deferred. No dependency or surface word changed. A project that never uses the cell system is unaffected.

D44 — Cell-system encoding: tags, codecs, roots, persistence

Decision. The cell system's encoding layer is applied as D44. It is data only: tags, tensor codecs, the two execution-state roots, and persistence. There is no allocation, slot access, dispatch, method combination, or metaclass behavior yet. The surface is Scale Basic English; the tensor schemas are internal storage and are never user syntax.

Tags. The append-only registry continues below `-14`:

Tag	Entry	Schema
`-15`	class definition	`[name; metaclass; combination(-1); superclassCount; superclass…; slotCount; slot…]`
`-16`	slot definition	`[name; valueSet; default(-1); initializationCode]`
`-17`	instance identity	`[class; instanceNumber]`
`-18`	instance state	`[instance; slotCount; slot; value…]`
`-19`	closure	`[quotation; environment]`
`-20`	environment frame	`[parent(-1); bindingCount; name; value…]`
`-21`	method combination	`[name; policyCode; qualifierCount; qualifierCode…]`
`-22`	qualified method	`[qualifierCode; arity; body; specializerCount; specializer…]`
`-23`	metaclass	`[name; superclassCount; superclass…; slotCount; slot…]`

The tag is the first scalar of the tensor, as everywhere else. The existing `GenericTag` and `MethodTag` keep their shapes; a qualified method is a new tag.

Roots. `EvaluationState` gains `ClassTable` and `InstanceStateTable`. Both are execution-state pointers into image-resident association vectors, like `SetsRoot`: `[name; value; …]`, latest-write-wins. Persistence serializes the store only; a reader re-threads both roots, exactly as it re-threads the other roots. The decoded records are transient projections: the store remains the only authority, so there is no host metaobject store.

Naming. A method-combination policy is `Standard`, `Append`, `List`, or `LastResult`. `LastResult` is the Common Lisp `progn` combination — run the methods in order and return the last value — named for what it does rather than by the Lisp jargon (D40).

Tests. Thirty-six checks pin each codec (class, slot, instance, instance state, closure, environment frame, method combination, qualified method, metaclass), malformed entries, distinct addresses for

identical data, the class and instance-state roots (including latest-write-wins), a fresh evaluator's roots, and a persistence round-trip with both roots re-threaded.

Suite. 612 example checks and 53 granular tests.

Status: applied. `types.fs` (tags, roots), `components/prelude/evaluator/evaluator.fs` (the fresh state), `components/prelude/evaluator/cell-system.fs` (codecs and root helpers), `bases/scale-cli/scale.fsproj` (the compile item), and `development/scale/tests.fs`. No surface word, dependency, or behavior changed. `entryKind` recognition and all cell behavior remain for the later stages.

D45 — Closures and lexical scope

Decision. D45 lands the first behavior layer of the cell system: lexical frames and closures. A frame is an image-resident environment entry (D44); a closure is an image-resident code cell carrying a quotation and the frame it captured. The current frame and the saved caller frames are execution state, never serialized.

Words.

1. **`let`.** `value \"name\" let` creates a child frame with the binding and makes it current. `bind` and `define` stay global; `define` remains a word definition by text splicing.
2. **`lambda`.** `{ body } lambda` captures the current frame in a closure and pushes its address. With no current frame it captures an empty frame, so the body still reads global bindings.
3. **`apply`.** A bare quotation keeps the old dynamic behavior: it runs with the caller's current frame. A closure runs its quotation in the captured frame and restores the caller's frame when the body ends.

Lookup. `lookup` reads the frame chain first, then the global binding table. A frame chain longer than 1024 links is a miss, not an infinite walk, so a hand-built cycle cannot hang the evaluator. Undecidable membership and the existing unification resolution are unchanged.

The call frame. `EvaluationState` gains `CurrentFrame` and `CallFrames`. Starting a closure call saves the caller frame, switches to the captured frame, and places an internal return token after the body; the token restores the saved frame. A return without a call is the named `UnmatchedReturn`, never a silent no-op.

Isolation and limits. A `QuotationRun` probe resets both frame fields, so a guard or predicate cannot see the caller's lexical frame. A spawned actor starts at the global frame; pass a closure to share lexical state. History rollback returns to the global frame: a step records the stack and program, not the call frames. Capturing call frames in the event stream is a later change.

Why. The cell system needs lexical capture for `:around` methods and for method bodies that close over state. Doing closures before slots, instances, and method combination keeps the dependency order honest: the later stages build on frames that are already image data.

Tests. Eleven checks pin `let`, the frame-chain walk, closure capture, a closure applied after its creating program ends, a bare quotation keeping dynamic scope, global fallback from an empty frame, restoration of the caller frame, `UnmatchedReturn`, `NotAText`/`NotAQuotation`, and a persistence round-trip where a closure and its frame survive.

Suite. 623 example checks and 53 granular tests.

Status: applied. `types.fs` (`CurrentFrame`, `CallFrames`, `UnmatchedReturn`), `components/prelude/evaluator/closure.fs` (the words and the internal return), `components/prelude/evaluator/cell-

`system.fs` (frame lookup, capture, and binding helpers), `components/prelude/evaluator/control.fs` (`apply` on a closure), `components/prelude/evaluator/environment.fs` (frame-first `lookup`), `components/prelude/evaluator/quotation-run.fs`, `components/prelude/evaluator/history.fs`, `scheduler.fs`, `components/prelude/evaluator/evaluator.fs`, `bases/scale-cli/scale.fsproj`, and `development/scale/tests.fs`. No new dependency. `define` and `bind` keep their old semantics.

Non-goals. Parameter lists and currying; static scope analysis; capturing call frames in history; `entryKind` recognition for closures and frames; slots, instances, precedence, method combination, and metaclasses. Those remain the later cell-system stages.

D46 — Classes, slots, and instances

Decision. D46 lands the first cell-behavior slice: `class`, `slot`, `create-instance`, `slot-value`, and `set-slot-value`. Classes and slots are image entries; an instance has a stable identity and an appended state. The words are the lab surface, not the child-facing Basic English grammar; that grammar is a later decision.

Declarations.

1. **`slot`.** `valueSetName \"slotName\" slot` declares a slot. The value set is resolved by name when the slot is used, so a later set redefinition is visible (latest-write-wins). Slot definitions are global by name for now.
2. **`class`.** `\"ClassName\" \"superclassNames\" \"slotNames\" class` declares a class. The lists are comma-separated; empty text means none. A class stores its superclass IDs, slot IDs, a default `StandardClass` metaclass, and no method combination yet. The class root is latest-write-wins, so redefinition appends a new class entry.

Instances. `\"ClassName\" create-instance` creates a stable instance identity and an initial state; the instance points at the class ID current at creation. The earlier `make-instance` spelling is retired. `slot-value` reads the latest state; `set-slot-value` validates the value against the slot's value set and appends a new state entry. The old state stays in the image and the instance-state root points at the newest entry.

Effective slots. The effective slots are the class's own slots followed by its superclasses' slots, deduplicated by name with the most specific definition winning. A cycle is skipped, not followed. D47 replaces this with the deterministic class precedence list.

Refusals. `UnknownClass`, `UnknownValueSet`, `UnknownSlot`, `MissingSlot`, `UnboundSlot`, `NotAnInstance`, `NotAClass`, `SlotTypeMismatch`, and `ValueSetUndecided` are named values. A value outside the slot's set is `SlotTypeMismatch`; an undecidable predicate set is `ValueSetUndecided`, never a silent accept.

Boundary. The cell system uses runtime value sets, not the verification `TypeLattice.FieldType`. It adds no SMT obligation. Classes and instances are image data; the class and instance-state roots are execution state and a persistence reader re-threads them. `cell-class` and `class-of` recognition are not part of D46.

Tests. Thirteen checks pin class and slot declaration, instance identity, the class pointer, an unbound slot, `set`/`get`, a value outside the set, a missing slot, an unknown class, an unknown value set, an appended state with the previous state still present, distinct identities from two `create-instance` calls, and a persistence round-trip with the roots re-threaded.

Suite. 636 example checks and 53 granular tests.

Status: applied. `types.fs` (the class/slot/instance refusals), `components/prelude/evaluator/cell-system.fs` (slot and metaclass lookup), new `components/prelude/evaluator/class-system.fs` (the five words), `components/prelude/evaluator/evaluator.fs` (the dispatch), `bases/scale-cli/scale.fsproj`, and `development/scale/tests.fs`. No new dependency; the Basic English grammar, class precedence, method combination, and metaclasses remain deferred.

Non-goals. The Basic English declaration grammar; a stable class identity across redefinition; per-class slot namespaces; class precedence; slot defaults exposed as words (the initializer supports the encoded default but `slot` currently declares `Uninitialized`); inheritance dispatch; method combination; metaclass behavior; `cell-class` and `entryKind` recognition.

D47 — Class precedence and nominal class dispatch

Decision. D47 adds the deterministic class precedence list (C3), the class membership queries, and nominal class specializers in the generic dispatcher. A class graph that cannot be linearized is refused at declaration.

Precedence. `EvaluatorClassSystem.classPrecedence` computes the C3 linearization of a class and its superclasses, most-specific first and including the class itself. A cycle is `ClassCycle name`; a conflicting local order is `InconsistentPrecedence name`. `class` validates the new class before it enters the class root, so a cyclic or inconsistent declaration never becomes the latest definition. Effective slots now follow the precedence list, so the most specific slot definition wins in a diamond as well.

Queries. `\"ClassName\" class-precedence` pushes the class names in precedence order. `\"Child\" \"Parent\" subclass-of` and `instance \"ClassName\" instance-of` push 1 or 0; a class cycle is a named error, never a silent false.

Dispatch. `ArgumentType` gains `ClassType` for an instance's class. `PreparedSpecializer` is either a value set or a class definition; `ClassSpecializer` applies when the instance's class is the specializer or a subclass, using the same precedence list. Specificity for classes is precedence inclusion; value-set specificity remains extension inclusion. A class and a value set are incomparable, so a mixed tie is `AmbiguousDispatch`, never a silent pick. The exact-tuple fast path compares a class specializer with the argument's class identity. `method` resolves a specializer name to a class ID when the class exists, so classes must be declared before the methods that specialize on them.

Tests. Eleven checks pin the three-class precedence list, subclass and instance membership in both directions, the most specific class method, an inherited class method, incomparable class methods as `AmbiguousDispatch`, C3 order for a diamond, a conflicting local order, and a hand-built class cycle.

Suite. 647 example checks and 53 granular tests.

Status: applied. `types.fs` (`ClassCycle`, `InconsistentPrecedence`), `components/prelude/evaluator/class-system.fs` (C3, membership queries, the three words, CPL-based effective slots), `components/prelude/evaluator/dispatch.fs` (`ClassType`, class specializers, specificity, labels, method-name resolution), `components/prelude/evaluator/evaluator.fs` (the dispatch), and `development/scale/tests.fs`. No new dependency.

Non-goals. Method combination and `call-next-method`; metaclasses and `cell-class`; conditions and restarts; a stable class identity across redefinition; per-class slot namespaces; a full CLOS MOP. Those remain later stages.

D48 — Method qualifiers and combination

Decision. Methods carry a qualifier: Primary, Before, After, or Around. A standard combination runs Around most-specific-first, all Before most-specific-first, the primary chain most-specific-first, and all After least-specific-first. `call-next-method` advances the current segment; before and after methods have no next method. `next-method-p` answers a verdict. The chain and its continuations are execution state, never serialized.

Why. Closures landed as D45, so an Around method can capture lexical state. The combination is data-driven: a new method is a store entry and a new generic function still needs no F# case. The exact primary remains definitive before the subtype search, and the less specific primaries stay reachable through `call-next-method`.

Status: applied. `types.fs` adds the qualifier, method, and combination frames; `components/prelude/evaluator/dispatch.fs` adds `before-method`, `after-method`, `around-method`, `call-next-method`, and `next-method-p`, and the combination machine; `components/prelude/evaluator/evaluator.fs` routes the words. The suite pins the effective order, the around result, `next-method-p`, `NoNextMethod`, and the qualifier-sensitive replacement rule.

Suite. 653 example checks and 53 granular tests.

D49 — Metaclasses and the cell-class closure

Decision. A class names a metaclass, with `StandardClass` as the default. `metaclass` declares an image-resident metaclass; `use-metaclass` re-declares a class with a named metaclass; `cell-class` answers the cell query: instance to class, class to metaclass, metaclass to the root metaclass, and the root to itself. The answer is a name. The closure is finite and not a reflective tower.

Why. A minimal metaclass layer makes classes first-class without adding an F# case for each user metaclass. The root is self-describing, and class-level behavior remains data. The full CLOS metaclass protocol is a later refinement.

Status: applied. `components/prelude/evaluator/cell-system.fs` already encoded metaclasses (D44); `components/prelude/evaluator/class-system.fs` adds `metaclass`, `use-metaclass`, and `cell-class`; `components/prelude/evaluator/evaluator.fs` routes the words. The suite pins the instance/class/metaclass/root answers, a user metaclass, and the unknown-name refusal.

Suite. 660 example checks and 53 granular tests.

D50 — Conditions and restarts

Decision. A protocol failure can be a value. A condition is an image-resident value with a kind (`error` or `warning`), a name, and a message. A restart is an image-resident value with a name and a handler quotation. `with-handler` runs a body; if the body fails, the failure becomes an error condition and the handler runs with that condition. `signal` raises a condition explicitly; with no enclosing handler it leaves the condition on the stack as a value, never an exception. `invoke-restart` runs a named restart handler with the condition. Handler frames are execution state, never serialized.

Why. D28 and the kernel inventory already treat protocol failures as values or witnesses. Conditions make that failure shape first-class and let a handler or a restart decide, without turning the evaluator into an exception machine. `Repair.conditionWitness` reads a condition's name and message, so the verification side can consume the same value.

Status: applied. `types.fs` adds condition and restart tags, handler phases, and handler frames; `components/prelude/evaluator/cell-system.fs` adds the condition and restart codecs;

`components/prelude/evaluator/condition.fs` adds the words and the error-to-condition bridge; `components/prelude/evaluator/evaluator.fs` routes the words and runs a handler when a body fails. `repair.fs` adds `conditionWitness`.

Suite. 668 example checks and 53 granular tests.

D51 — Control refinement

Decision. `quotation`, `if`, `apply`, and `define` are control words: image-resident `Word` metaobjects whose implementation is either a default kernel primitive or a code cell. A control word's implementation entry names a primitive token or a `Closure` (quotation plus captured environment). `handler name control` installs a code cell under `name`; the handler's frame is captured at declaration, so the form is lexically scoped. The default forms name `control-quotation`, `control-if`, `control-apply`, and `control-define`, which run in the caller's frame; a user code cell runs in its captured frame. The dispatcher consults the control word before the built-in token, so a later declaration replaces a form without editing F#. No tag or state field was added.

Why. D17 stage 5 needs a first-class code cell; before D51 a quotation was text and `define` spliced it dynamically. D45 supplies the closure. Keeping the default implementations as primitive names preserves dynamic `if`/`apply` behavior while making the form itself replaceable data.

Status: applied. `components/prelude/evaluator/control.fs` adds `controlWord`, `ControlImplementation`, and `seedControlForms`; `components/prelude/evaluator/evaluator.fs` routes control implementations and seeds `WordTable`; the `control-*` primitives remain the default methods. Tests in `development/scale/tests.fs`: seeded forms, a new form, lexical capture, replacing `if`, latest declaration wins, inactive refusal.

Suite. 688 example checks and 53 granular tests.

D52 — Reference linearity and reclaim candidates

Decision. The memory track starts with a static, conservative reference-linearity report over the image, not with static ownership types. `Reclaim.report roots store` classifies every entry as `Root`, `Unreferenced`, `Linear`, or `Shared`, and returns the entries that no live root can reach.

Possible references. An element of a tensor is a possible reference when it is a non-negative integer below `store.NextIndex`. That over-approximates the true reference graph: numeric data can look like an address. The over-approximation is deliberate because the image has no per-element type tags. A reclaim candidate is therefore never a live entry, but the report can miss entries that are in fact dead.

Roots. `Reclaim.rootsOf` collects the store roots, the stack, every execution-state image root, call and method frames, handlers, ownership values, and scheduler/channel roots. Text-cache entries are roots so interning survives. When the scheduler holds more than one actor, every entry is treated as a root, because a per-actor report cannot see another actor's stack.

Surface. `reclaimable` appends and pushes a vector of the current candidate addresses. It never deletes or rewrites an entry: the image stays append-only, and the word reports what a later, explicit compaction step could take.

Why this comes before static types. D30 enforces linear ownership dynamically, and `rollback` deliberately leaves later event-stream steps unreachable. Reclaim needs a reference graph before it can use ownership information; calling the graph static linearity keeps the first step falsifiable and independent of a full type system.

Status: applied. `reclaim.fs` (report and roots), `components/prelude/evaluator/ownership.fs` (the word), `components/prelude/evaluator/evaluator.fs` (dispatch), `properties.fs` (generator coverage), and `development/scale/tests.fs` (orphan, linear, shared, text-root, and word checks).

Suite. 688 example checks and 53 granular tests.

Non-goals. Physical deletion, a retained-entry API, static ownership types, and a precise reference encoding. Those need a decided representation for references; this decision only makes the conservative reclaim set visible.

D53 — Opt-in compaction with stable addresses

Decision. Physical reclamation is an explicit, opt-in step. `compact` replaces the in-memory image with a sparse successor that keeps only the entries reachable from the live roots. It keeps every surviving address unchanged, so no tensor needs rewriting; the previous image is not mutated.

Persistence boundary. `NextIndex` is the allocation frontier, not the entry count. The reader accepts a sparse image whose lexicon count is below the frontier, and still refuses a count above the frontier, duplicate addresses, addresses at or beyond the frontier, and text names that point at missing entries.

Surface. `reclaimable` reports candidates and `compact` acts on the current state. Neither takes an operand. With more than one actor, D52′s root rule treats every entry as a root, so compaction keeps the whole image.

Why stable addresses. Renumbering would require a precise reference encoding and a full rewrite of every referring tensor. Keeping addresses makes compaction a checkpoint over an append-only history: surviving entries keep their identity, and the sparse frontier records the address space that was once allocated.

Status: applied. `reclaim.fs` adds `compact` and `compactForState`; `components/prelude/evaluator/ownership.fs` adds `compactWord`; `components/prelude/evaluator/evaluator.fs` dispatches it; `persistence.fs` accepts sparse images; `properties.fs` covers the word; `development/scale/tests.fs` pins compaction, sparse round-trip, the word, and the append-only behavior of `reclaimable`.

Suite. 700 example checks and 53 granular tests.

Non-goals. Automatic collection, address reassignment, precise reference tags, and static ownership types. Compaction is a deliberate action, never a silent side effect.

D54 — Static ownership contracts

Decision. A quotation can be checked against a static ownership contract before it runs. The checker tracks one abstract input reference through a symbolic stack whose slots are `Unowned`, `Owned`, `Consumed`, or `NonReference`. It returns `Linear`, `Nonlinear reason`, or `Undecided reason`.

Rules. `own` moves an unowned reference to owned. `borrow` requires an owned reference and keeps it owned. `consume` requires owned and marks it consumed. `release` returns an owned reference to unowned and leaves a consumed one consumed. A second consume, a borrow after consume, and a use without ownership are nonlinear. `duplicate` and `over` are nonlinear when they alias a reference; `drop` is nonlinear when it discards an owned reference without release or consume.

Undecided is honest. A word outside the modelled ownership-and-shuffle subset returns `Undecided`, never a silent pass. A variable sigil is also undecided because it may alias a live reference. The checker therefore proves only the quotations it can model.

Surface. `linearity` pops a quotation and pushes `[LinearVerdictTag; status; reasonId]`: status 0 linear, 1 nonlinear, 2 undecided. The reason is interned text and is absent for a linear verdict.

Why conservative. D30 enforces ownership at runtime. This static contract catches the obvious linearity errors before execution without pretending that a stack language can infer every word's ownership effect. A full type system needs effect signatures for the whole vocabulary; this step does not claim one.

Status: applied. `ownership-types.fs` (the checker), `types.fs` (`LinearVerdictTag = -26`), `components/prelude/evaluator/ownership.fs` (`linearityWord`), `components/prelude/evaluator/evaluator.fs` (dispatch), `properties.fs` (generator coverage), and `development/scale/tests.fs` (linear, second-consume, borrow-without-own, alias, dropped-owned, undecided, and word checks).

Suite. 718 example checks and 60 granular tests.

Non-goals. Full type inference, effect signatures for every word, ownership transfer, borrow regions, and serializing ownership. Those are separate decisions; the contract stays conservative and local.

D55 — Transition emission (the D18 conditional, now that event semantics exist)

Decision. Scale Basic English admits an event-triggered sentence shape. D18 refused `when` because "`when` and `Transition` wait for event semantics"; those landed as W7 (D26) and W8 (D29). D55 re-admits the conditional as a grammar production with one canonical logical form, and projects it onto `SentencePattern.Transition`.

Grammar.

```
sentence   ::= statement | transition
statement  ::= quantifier subject relative? verb "."
transition ::= "when" subject ("that")? condition ("and" condition)* ","
               "every" subject verb "."
```

A transition is order-sensitive in the same way a statement is: sentence-initial `when`, a TypeName subject, at least one condition, a structural comma, the universal outcome, and a full stop. The comma is a token of its own; the tokenizer splits it, so it cannot be swallowed into a predicate phrase. The optional `that` mirrors a statement's relative clause.

The trigger names its subject explicitly because the grammar still refuses pronouns and articles: `When player is alive, every player can jump.` The trigger subject and the effect subject must be the same token. A different subject is `TransitionSubjectMismatch`, never an anaphor the parser invents. The outcome is universal: an effect is required, so `no` and `some` are refused with `TransitionOutcomeNotUniversal`. A missing comma is `ExpectedTransitionComma`; an empty trigger is `ExpectedCondition`.

Canonical form and artifacts. The trigger's conditions are the precondition and the verb is the effect. The F# artifact is `precondition ==> effect`, named `when <subject> <verb>` and headed `// Pattern: Transition`; `generated/transition-sample.fs` is the pinned, compiled artifact. The SMT obligation is the same universal obligation as a capability (`pre ∧ ¬post`), with `; Pattern: Transition` in the header. The repair corpus accepts `"Transition"` entries, and `Repair.propertyHolds` already read a transition as a guarded effect.

What is deferred, and why. A transition is currently a *guarded effect* over one subject: the same truth condition as `Capability`, under a distinct pattern and artifact. A *state-change* rendering (a before/after pair, or a transition function) needs a two-state domain model that the specification and the verification lattice do not carry. D55 does not invent one; D8′s "state-change rendering is deferred" stands. Also out of scope: anaphora, articles, nested transitions, non-universal outcomes, and coupling the sentence to the event-stream.

Status: applied. `parser.fs` (`SentenceKind`, `parseTransition`, three named refusals), `tokenizer.fs` (comma), `emitter.fs` (transition comment, name, and property shape), `repair.fs` (`parsePattern` accepts `Transition`), `generated/transition-sample.fs`, and `development/scale/tests.fs` (positive corpus, trigger `that`, seven refusals, golden artifact, compiled property, SMT pattern, and the corpus acceptance check).

Suite. 741 example checks and 60 granular tests.

Non-goals. Before/after state-change emission, temporal logic, effect signatures for the vocabulary, and any new dependency. The transition is a sentence shape, not a new kernel case.

D56 — Witness-guided candidate generation for symbolic domains

Decision. A lattice whose fields have no finite coordinates is *large* or symbolic: Z3 can reason about it, but the exhaustive universe and the coverage ranking cannot. D25 said such a lattice "cannot propose guards", which left the refiner unable to narrow a large domain. D56 lets the refiner propose and rank guards from the solver's own counterexample, with the same `pre ∧ guard ∧ ¬post` proof as the finite path. No new oracle and no sampling.

What lands. `TypeLattice.isFinite` distinguishes the two regimes. `TypeLattice.candidateGuardsFromWitness` reads the unrefined falsification model and returns the weakest guards that exclude it: `IsFalse` for a true witness and `IsTrue` for a false one; `GreaterThan(field, n)` and `LessThan(field, n)` for a numeric witness `n`. A field the counterexample omits, or a value the encoded theory cannot represent, is a named `Error`, never a silently skipped field.

`Refiner.refine` keeps the finite path unchanged (D25) and routes a symbolic lattice to `refineSymbolic`. That path requires `ProvenClosure.Witness`, so a symbolic domain without a counterexample is a named refusal. It widens each seed outward (`n ± 1, ±2, ±4, …`, bounded at `2^30`), stops at the first oracle-proved threshold, and bisects the last failing and first proving threshold to return the *weakest* proved guard, not merely a proving one. The first proved guard in field order wins, with the above-side seed before the below-side seed on the same field; a boolean seed is proved or rejected as it stands.

Coverage. The finite path ranks candidates by how many members of the universe they keep. A symbolic domain has no such universe, so `Candidate.Coverage = 0` records that coverage was not measurable. The guard is still chosen only because the oracle proved it, never because it was plausible.

Trust boundary. `Proves` still means `pre ∧ guard ∧ ¬post` is unsatisfiable; `Witness` is the model of the unrefined falsification. `Unknown` and `Unavailable` stay failures inside the oracle, so a symbolic refiner that cannot prove a guard returns `Ok None` rather than a silent pass. The search is bounded: after `2^30` it gives up with `Ok None`, and every candidate is an integer inside the encoded theory.

Status: applied. `type-lattice.fs` (`isFinite`, `candidateGuardsFromWitness`), `refiner.fs` (`scaleFileValue`, `witnessValues`, `tighten`, `refineSymbolic`, and the `refine` dispatch), and `development/scale/tests.fs` (finite versus symbolic, seeded guards, a missing witness field, the

weakest-threshold search, the bisection beyond the first ladder rung, no witness, and no proved guard).

Suite. 753 example checks and 60 granular tests.

Non-goals. Multi-field or conjunctive guards; non-linear arithmetic; exhaustive coverage for a symbolic domain; relational or temporal guards; and weakening the proof gate to sampling. The finite path and its pinned artifacts do not move.

D57 — Violation reporting, phase P0

Decision. Every grammar refusal becomes one uniform, blamable value instead of a bare `ParseError`. The value names the **layer** that detected the failure, the **rule** it broke, the **offending constituent**, what was **expected**, what was **found**, and the **party at fault**: the input or a gap in the fixed specification. This is phase P0 of the proposal in `docs/prose-foundations.typ`.

What lands. `violations.fs` defines `Layer` (Lexical, Structural, Agreement, Sortal, Binding, Semantic, Discourse), `Blame` (Input, CoverageGap), `Span`, `Witness` (UnsatCore, Model), and `Violation`. Its `ofParseError` maps every `Parser.ParseError` case to a named rule, a layer, and a blame, and `Violations.parse` returns `Result<Sentence, Violation>`. Parsing and accepted sentences are otherwise unchanged.

Why P0. The pipeline could accept or refuse, but it could not say which kind of rule failed, on which constituent, or whether the fault was the user's or the language's. `Blame` is the load-bearing field: `Input` should block execution, while `CoverageGap` is a note that the fixed specification does not cover the construct, optionally with the closest legal approximation. The rule is the `ParseError` case name, so a caller can switch on a stable identifier without parsing prose.

P0 boundary. Only the parser is wired. `Span` carries the offending canonical token, not a character offset, because the tokenizer does not retain source offsets yet. `Witness` declares its cases but P0 produces none: a grammar refusal has no solver model or unsat core. `Repair` is a hint for the refusals where a direct edit exists (use `no`, remove the article, add the full stop), not a search.

Status: applied (P0). `violations.fs`, `bases/scale-cli/scale.fsproj`, and `development/scale/tests.fs` (every mapped case, the offending token, expected/given, blame, a repair hint, and a valid sentence with no violation).

Suite. 791 example checks and 60 granular tests.

Non-goals. Phases P1–P5: sort, unification, guard, SMT, and discourse detectors; unsat cores and witness models; the minimum-edit repair search; and the presentation layer. No schema change, no new dependency, and no change to an accepted sentence or a pinned artifact.

D58 — Disjunction (the D18 `or` stage)

Decision. Scale Basic English admits `or` between conditions. `and` binds tighter than `or`, so a relative clause is a disjunction of conjunctions. The executable projection is a disjunctive normal form: one `Condition.Disjunction` atom whose alternatives are conjunctions of atomic conditions. A single conjunctive clause is flattened to its atoms, so every existing sentence, header, corpus entry, and pinned artifact is unchanged.

Grammar.

```
relative     ::= "that" disjunction
disjunction  ::= conjunction ("or" conjunction)*
```

```
conjunction  ::= condition ("and" condition)*
condition    ::= predicate | "is" field | "has" field comparison integer
```

A transition's trigger uses the same `disjunction`. `or` is a `Connective` in the active vocabulary, so it is a structural token, not an unknown word. `and` still joins conditions; `or` joins conjunctions. There is no precedence question left to guess at.

Semantics. The emitted F# precondition is `||` over && clauses, each clause parenthesized so the enclosing conjunction cannot change its shape; the SMT obligation is `MkOr` over `MkAnd` clauses; and the repair gate evaluates the DNF (`any alternative whose atoms all hold`). The canonical text is `(a and b) or (c)` and round-trips through `parseConditionText`; a disjunction is one condition text, so the corpus schema does not change. A bare field is still not a condition: `Every player that is alive or health can jump.` is `ExpectedCondition "health"`, never a silent skip.

Status: applied. `types.fs` (`Disjunction`, `conditionAtoms`, `conditionFields`, the recursive text and text parser), `parser.fs` (`parseDisjunction`), `vocabulary.fs` (`or`), `emitter.fs` (record fields and `||` rendering), `smt.fs` (`MkOr` over `MkAnd`), `repair.fs` (DNF evaluation), and `development/scale/tests.fs` (parser, precedence, transition trigger, emitter, SMT, and text round-trip).

Suite. 801 example checks and 60 granular tests.

Non-goals. Nested parentheses beyond the canonical DNF text, `not` or implication, distributing a disjunction across a transition's effect, and any change to the corpus schema or a pinned artifact.

D59 — Higher-order unification (Miller patterns)

Decision. The unification pillar (W4/D15) grows higher-order terms without becoming full Huet unification. A *flex* term is a variable applied to arguments. When the spine is a list of distinct atoms, a flex term unifies with a rigid side by binding the variable to the curried abstraction of that side over the spine. Lambda terms alpha-rename a parameter. Non-pattern spines, flex-flex pairs with different spines, and would-be capture are `Undecided` with a named reason, never a silent false.

Encoding. `LambdaTag = -15` with schema `[tag; parameter; body]`; application reuses the existing compound encoding (`-2`). The three-way verdict is `UnificationVerdictTag = -27` with schema `[tag; status; reasonId]`, status 0 unified, 1 not unified, 2 undecided.

Words. `lambda-term` pops a parameter and a body and pushes the binder; `reduce` beta-normalizes; `unify-verdict` pushes the tagged three-way verdict. `unify` keeps its 1/0 result for decidable cases and reports `UnificationUndecided` for a non-pattern case. The first-order words and their results are unchanged.

Algorithm. Both sides are beta-normalized first. A flex-rigid pair with a distinct-atom spine binds the variable to `λa1. … λan. body`; the existing occurs-check rejects a recursive binding. Flex-flex with the same spine aliases the variables; different spines are undecided. Lambda-lambda alpha-renames one parameter and unifies the bodies; lambda-against-non-lambda is undecided. Substitution is capture-avoiding by refusing a binder that would capture the replacement.

Boundary. This is the Miller-pattern fragment plus first-order fallback, not full higher-order unification: there is no imitation or projection search, no non-pattern spine, no dependent type, and no eta-conversion. The named `Undecided` is the honest boundary; it is not a false.

Status: applied. `components/prelude/evaluator/environment.fs` (lambda view and encoding, substitution, beta-normalization, pattern binding, three-way unification), `components/prelude/evaluator/evaluator.fs` (dispatch), `types.fs` (`UnificationVerdictTag`, `UnificationUndecided`),

and `development/scale/tests.fs` (pattern binding, beta-reduction, alpha-renaming, non-pattern undecided, flex-flex, occurs-check).

Suite. 814 example checks and 60 granular tests.

Non-goals. Full Huet unification, dependent types, eta-conversion, delta-reduction, and any change to the first-order surface or the pinned artifacts.

D60 — Tensor algebra beyond scalars (W11)

Decision. The arithmetic and comparison words operate elementwise over two tensors of the same shape. A scalar is the rank-0 case, so the old scalar behavior is unchanged. A shape mismatch is a named `ShapeMismatch`; a scalar is never silently broadcast, and a tensor is never read through its first element. `ScalarRead.readScalar` now refuses a non-scalar.

Semantics. Shape equality is the precondition; the result keeps that shape. `divide` propagates `DivisionByZero` from any element; the comparisons produce an elementwise 0/1 tensor for a non-scalar and the old 0/1 scalar for rank 0. `binaryOperation` sequences the per-element results, so a single failing element fails the whole operation rather than being dropped.

Why. The old `scalarValue` read element 0, so `[1; 2] add [3; 4]` silently computed `1 + 3` and discarded the rest. That is the C5 leaf leak and the W11 item: a shape or a non-scalar read must be named, not guessed.

Status: applied. `components/prelude/evaluator/common.fs` (`binaryOperation`), `scalar-read.fs` (rank check), `types.fs` (`ShapeMismatch` and its rendering), and `development/scale/tests.fs` (elementwise vector/matrix operations, a zero divisor, named shape mismatches, the scalar rank-0 case, and the non-scalar read refusal).

Suite. 823 example checks and 60 granular tests.

Non-goals. Broadcasting, matrix multiplication, tensor contraction, and changing the tensor shape/data invariant. A rank-0 read is still `Tensor.item 0` after an explicit rank check.

D61 — Violation reporting, phase P1 (semantic stages)

Decision. The semantic stages join the diagnostics track. Every `EvaluationError` and every guard verdict becomes the same blamable `Violation` as a grammar refusal, through `Violations.ofEvaluationError` and `Violations.ofGuardVerdict`. The parsing behavior, the error type, and every accepted program are unchanged; this is a reporting layer over values the pipeline already produces.

Layers and blame. Stack, shape, and term errors are `Structural`; unknown or inactive words are `Lexical`; ownership and unknown bindings are `Binding`; sorts, dispatch, slots, classes, constructors, guards, and methods are `Sortal`; an undecidable unification is `Agreement`; channels, events, divisions, conditions, and restarts are `Semantic`. A bad program is `Input`. An `Undecided` membership, dispatch, unification, guard, or budget is `CoverageGap`, so the fixed theory's limit is not reported as the user's fault.

What the value carries. The rule is the `EvaluationError` case name, the span is the offending token or entry when one exists, and expected/given are filled where the constructor carries them (`StackUnderflow` names the needed and available counts; `ShapeMismatch` names both shapes; `AmbiguousDispatch` names the candidates). A holding guard is no violation; a failed guard is `GuardDoesNotHold/Sortal/Input`; an undecided guard is `GuardUndecided/Sortal/CoverageGap` with its reason.

Status: applied. `violations.fs` (`ofEvaluationError`, `ofGuardVerdict`, and the shared `makeDetail`), `bases/scale-cli/scale.fsproj` (the module now compiles after the lattice so it can read `GuardVerdict`), and `development/scale/tests.fs` (nine representative evaluation errors across layers, plus the three guard verdicts).

Suite. 853 example checks and 60 granular tests.

Non-goals. Phases P2–P5: SMT unsat cores and models, discourse rollback, the minimum-edit repair search, and the presentation layer. No schema change, no new dependency, and no change to an accepted program.

D62 — Violation reporting, phase P2 (SMT models and unsat cores)

Decision. The SMT stage joins the diagnostics track. `Smt.dischargeWithCore` returns a `Discharge` — the existing `Outcome` plus the assumptions Z3 used — while `Smt.discharge` remains the plain single solve. Both call the same private `solve`, so a verdict and its evidence cannot diverge, and the production path pays for a core only when it asks for one. `Violations.ofSmtDischarge` maps that value to the same blamable `Violation` as the parser and the semantic stages.

The core is labelled, not asserted whole. The precondition is built from named atoms: `condition <n>`, `refinement <guard text>`, and `lattice guard <guard text>`, with the postcondition (or its negation, for a universal property) as the final atom. Only an unsatisfiable verdict triggers a second query that asserts each atom with `AssertAndTrack` under a solver-private name (`__scale_assumption_<i>`); the tracker names in `UnsatCore` are mapped back to those human-readable descriptions. A proved universal property therefore says which part of the theory refuted the counterexample; a refuted possibility says which assumptions made a witness impossible. If that tracking query does not reproduce the unsat verdict, the core is left empty and the plain verdict stands: evidence is optional, a second verdict would not be. The names never enter the encoded formula, so the byte-pinned `.smt2` obligation, the satisfiable model selection, and every accepted program are unchanged. A guard the document already carries is still conjoined once: the lattice filter compares canonical `.scale` text.

What the value carries. `Proved` and `Witnessed _` are successes and produce no violation. `Falsified model` is `Semantic/Falsified/Input` with `Witness = Model model` and the rendered model in `Given`; `NoWitness` is `Semantic/NoWitness/Input` with `Witness = UnsatCore core`. `Unknown` and `Unavailable` remain `CoverageGap`, because a solver limit is not the user's fault. `ofSmtOutcome` delegates with an empty core for callers that hold only an outcome.

Status: applied. `smt.fs` (`Discharge`, `preconditionAtoms`, `solve`, `namedCore`, `dischargeWithCore`, and the plain `discharge`), `violations.fs` (real `Witness` payloads, `ofSmtDischarge`, and `ofSmtOutcome`), `bases/scale-cli/scale.fsproj` (the module now compiles after `smt.fs`), and `development/scale/tests.fs` (a falsified property and its model, a proved property and its core names, a witnessed possibility, a refuted possibility and its core, and the two coverage-gap outcomes).

Suite. 898 example checks and 60 granular tests.

Non-goals. Phases P3–P5: discourse rollback, the minimum-edit repair search, and the presentation layer. No schema change, no new dependency, and no change to an accepted program.

D63 — Preemptive supervision and reduction deadlines (W5)

Decision. A supervised actor is bounded by a budget of executed tokens, and its supervisor decides what a failed or exhausted actor becomes. The per-token budget already exists (`Reductions`, `components/prelude/evaluator/evaluator.fs:330-333`); the deadline is that same counter applied to one actor's lifetime rather than to one fairness turn, and supervision is the policy that consumes the resulting `TimeoutExit`. No wall clock enters the core.

The unit is a token, not a tensor. A token is one string popped from `Program` — the lexical unit the loop executes (`components/prelude/evaluator/evaluator.fs:325`). One reduction is one token attempted, charged at `components/prelude/evaluator/evaluator.fs:342`. The unit is deliberately not a tensor. One token interns its text, a stack tensor, a tensor over every remaining program token, and its step tensor (`components/prelude/evaluator/history.fs:87-111`), and a host word such as a literal, `channel`, or `class` writes an unbounded further number of entries. A token that calls a user word splices that body into `Program` (`components/prelude/evaluator/evaluator.fs:275-284`), and each spliced token is charged when it runs. A tensor count would therefore be neither fair nor stable, and would make the scheduler's accounting depend on the W8 recorder's cost. Tokens are the only unit independent of both the encoding and the recorder.

What is added. `ExitReason` (`NormalExit`, `ErrorExit of EvaluationError`, `TimeoutExit`, `ShutdownExit`, `KilledExit`); `RestartType` (`Permanent`, `Transient`, `Temporary`); `SupervisionStrategy` (`OneForOne`, `OneForAll`, `RestForOne`); `SupervisionLimits` (`MaxRestarts`, `WithinSteps`, and a deadline in tokens); and `ActorSpec` (quotation plus restart type). `Actor` gains its spec, its exit reason, and its consumed tokens. `EvaluationState` gains `Consumed` and `Deadline` beside `Reductions`, so both are carried by the record the loop already rebuilds and are snapshotted with the suspended actor for free. `ActorStatus` is not widened: the `actors` tensor encodes 0–3 and is a pinned contract.

Deadline semantics. `Consumed` counts executed tokens only and is not reset by a fairness yield. A halted actor consumes nothing: waiting on a channel or an event costs no tokens, so a deadline can never fire on a blocked actor — it is a running-token budget, not time since spawn. An actor whose `Consumed` reaches its `Deadline` exits with `TimeoutExit`, and its supervisor decides what happens next. With the deadline encoded as an `int` and a sentinel for "no deadline", the check is one increment and one compare folded into the existing `Reductions <= 0` branch.

Restart. A restart is a fresh actor: the supervisor terminates the old one (which already removes it from the ready queue, the event waiters, and every channel's receive waiters), allocates a new identity, and spawns it from `ActorSpec.Quotation` against the live store. Reusing the old identity is forbidden, because an identity already sent to another actor or queued in a channel would silently change meaning. An unsupervised `spawn` keeps today's behaviour — an error aborts `run` — so existing programs are untouched; only `supervise` opts into exit propagation. Four words land: `one-for-one`, `one-for-all`, `rest-for-one`, and `supervise`.

Compatibility with Concurrent ML. Supervision and preemption leave the event layer unchanged, and three properties make that safe. An event commit is atomic within one token, and preemption happens only between tokens, so no yield can land mid-rendezvous, mid-fallback, or between queueing a message and waking a waiter. A blocked actor is never charged, so receive timeouts remain events composed with `choose`, not scheduler policy. And a terminated actor is removed from every waiter list before any restart, so a restarted actor cannot swallow a message (D27); because the restart takes a new identity, a stale waiter entry can never name a live actor.

Measured cost. On this machine, in a Debug build under F# Interactive, the deadline check is 1.14 nanoseconds per token against 52.1 nanoseconds for a `Stopwatch.GetTimestamp()` read (about 46

times more, and nondeterministic), and the marginal cost of one more `int` field on an already-copied `EvaluationState` is below the measurement noise. The current loop costs 24–80 microseconds per token because W8 records the stack and the remaining program on every token; the deadline is under 0.01 percent of that.

Surface. A strategy word installs the current actor's policy and takes three non-negative integers: `maxRestarts withinSteps deadline one-for-one` (or `one-for-all`, `rest-for-one`); `withinSteps = 0` means no time window and `deadline = 0` means no deadline. `supervise` then takes `<quotation> "permanent" supervise` (or `"transient"`/`"temporary"`) and pushes the child identity, exactly as `spawn` does. The strategy words are stack-neutral; the restart type is the child's, the strategy and limits are the supervisor's.

Status: applied. `types.fs` defines the types, the exit reason, and four named refusals (`NoSupervisionPolicy`, `InvalidRestartType`, `InvalidSupervisionLimit`, `SupervisionLimitExceeded`); `EvaluationState` carries `Consumed` and `Deadline`, and `Actor` carries its specification, exit, consumed tokens, policy, and restart window. `scheduler.fs` adds `addChild`, `superviseActor`, `exitCurrent`, `timeoutCurrent`, restart admission, restart groups, and subtree termination. `components/prelude/evaluator/evaluator.fs` folds the deadline into the token branch and routes a supervised child's error to its supervisor; `components/prelude/evaluator/scheduler.fs` adds the four words and refuses a fractional or negative limit. The suite pins every restart type, all three strategies, a deadline firing at three tokens, a blocked actor consuming nothing, policy and limit refusals, and an unsupervised spawn still aborting the run: 1069 example checks (1034 examples and 35 properties) and 78 granular tests.

Boundary. A supervisor that exhausts its restart budget terminates its whole subtree and surfaces `SupervisionLimitExceeded` as the run's diagnostic; the failure is not propagated to a grandparent supervisor. That is the one place this landing is narrower than Erlang's supervision tree.

Non-goals. Wall-clock deadlines in the core; priorities; mailbox limits; dynamic child specifications; slicing host words (heavy host work is an event, and a word that must block is not preemptible); true parallelism; distribution; and actor migration. Hot code reload is reserved as D66 and distribution as D67; neither is part of this decision.

D64 — Actor handles are private capabilities

Decision. `spawn` and `supervise` return an image-resident *handle*, not the actor's address. `kill` and `resume` accept only a handle the scheduler registered for a live actor; a plain number, a forged entry, a foreign handle, or a retired handle is `NotAnActorHandle`. `actors` reports one status code per actor and exposes no address. The handle map lives in `SchedulerState`, next to the ready queue, and is execution state: it is never serialized.

Why. An actor address was a small integer, and `kill`/`resume` read it with `EntryIdentity.ofInt`, so any program could target any actor by guessing a number. That is the one place Erlang's model is stronger: a PID is a value a process cannot construct from an integer, and possessing it is the authority. The handle restores that property: no surface word allocates a handle except `spawn`/`supervise`, and the scheduler validates the *entry*, not its contents, so duplicating the tensor or writing a set literal with the same bytes yields an entry the map does not know.

Encoding. A handle is a vector `[ActorHandleTag]` with `ActorHandleTag = -28.0`, allocated by `Store.addTensor`; each allocation has its own address even when the contents are equal. The scheduler maps that address to the child identity. `spawn`/`supervise` push the address, so a program can bind it or pass it to another actor exactly like a PID.

Lifecycle. A handle is retired when its actor terminates, and a restart creates a fresh identity and a fresh handle; an old handle never names the restarted incarnation. A handle is per run: because the map is not serialized, a handle from a previous run is not a handle in the next. There is no `self` handle, so `kill self` is no longer reachable from the surface; the host-side guard remains.

Boundary. The handle grants `kill`/`resume` to its holder, like a PID; it does not add authentication or revocation. Channel addresses remain numeric store entries and are not covered here. Diagnostics (`CannotKillSelf`, `NotOwned`/`OwnedByAnother`) still print numeric actor identifiers; they reveal numbers, not capabilities.

Status: applied. `types.fs` adds `ActorHandleTag`, `SchedulerState.ActorHandles`, and `NotAnActorHandle`; `scheduler.fs` registers and retires handles, and `addChild` returns a handle; `components/prelude/evaluator/scheduler.fs` targets `kill`/`resume` through the mapping and makes `actors` status-only. Tests pin that `spawn` pushes a tagged handle, a number is refused, a retired handle is refused, the handle maps to the child, and `actors` carries one status per actor. Suite: 901 checks (874 examples and 27 properties) and 60 granular tests.

Non-goals. Cryptographic unguessability of handles; cross-run handles; delegated or revocable handles; capability transfer in messages beyond the ordinary stack value; and channel-address privacy.

D65 — Actor mailboxes, self, and links

Decision. Every actor owns a private mailbox and a private handle. `self` pushes the handle, `send-to` appends a message to the target's mailbox (resolved through its handle), and `receive-message` dequeues the oldest message or blocks exactly like `receive`. `link` and `unlink` maintain a mutual link between two actors. An abnormal exit (error, timeout, or kill) delivers a pending exit signal to every live linked actor; `runProgram` consumes that signal before the actor's next token, so a blocked actor cannot ignore it. A normal exit signals nobody. Links are per incarnation and are cleared on termination.

Why. D63/D64 gave the scheduler preemption and private addresses, but actors still had no way to send each other a message or observe an exit. A mailbox is the missing half of the actor model, and links are what make supervision more than a tree of restart policies: a failure reaches the actors that depend on it.

Encoding. A mailbox is an ordinary channel identity (`Store.channelIdentityTensor`) created at spawn and never exposed to a program. Its id is held only in `Actor.Mailbox`; `send-to`/`receive-message` push that id internally and reuse the W6/D27 channel state, so the append-only queue, waiters, and persistence are the same code path. `Actor.Handle` is the D64 capability. `Actor.Links` is a host-side list of actor identities; it is execution state, never serialized.

Link semantics. `link` is idempotent and a self-link is a no-op. `unlink` removes both directions. An abnormal exit sets `PendingExit` on each live linked actor and makes it ready; the run loop exits it with the same reason before its next token, so its own supervisor decides what happens next. Normal exits and `ShutdownExit` do not signal.

Boundary. There is no `trap-exit` at D65: a linked actor cannot turn an exit into a message. There are no monitors, no selective receive, and no named registry at D65; D66 adds trap-exit, D67 adds monitors and the registry, and D68 adds selective receive. The mailbox id can appear in a manual-resume error because it is the existing channel error, but it is never on the stack. Mailboxes are unbounded, as D63 records.

Status: applied. `types.fs` adds `Actor.Handle`, `Actor.Mailbox`, `Actor.Links`, `Actor.PendingExit`, and `NoActorMailbox`; `scheduler.fs` creates the root and child resources, propagates abnormal exits, and maintains links; `components/prelude/evaluator/scheduler.fs` adds `self`, `send-to`, `receive-message`, `link`, and `unlink`; `components/prelude/evaluator/evaluator.fs` consumes `PendingExit` before a token. Tests pin self, mailbox delivery, a blocking empty mailbox, a forged `send-to`, link causation, and unlink. Suite: 909 checks (882 examples and 27 properties) and 60 granular tests.

Non-goals. Monitors (D67); selective receive (D68); a named process registry (D67); bounded mailboxes; distribution; and priorities.

D66 — Trapped exits

Decision. An actor can set `trap-exit` to true. A link's abnormal exit then becomes an `ExitSignal` message in its mailbox instead of a kill; `exit-signal?` and `exit-reason` inspect it. A non-trapping actor keeps D65′s pending-exit kill. Normal exits still signal nobody.

Why. D65 links always kill; an Erlang actor needs a way to observe and decide. Trapping is the minimum: the signal is a message, so the existing mailbox and `receive-message` path carries it without a second delivery mechanism.

Encoding. The signal is `[ExitSignalTag; reasonCode]` (`ExitSignalTag = -29.0`), with Normal=0, Error=1, Timeout=2, Shutdown=3, and Killed=4. `trap-exit` sets `Actor.TrappingExits`; `propagateLinks` delivers via `Scheduler.deliverMessage`, the same delivery and queue path `send` uses, so a blocked receiver wakes and gets the signal as its receive result. If delivery fails (no mailbox), the actor falls back to the pending-exit kill.

Boundary. Only links trap. `trap-exit` does not turn a normal exit into a message, and it does not intercept an explicit `kill` of the actor (which is not a link signal). Monitors and a registry are D67; selective receive is D68.

Status: applied. `types.fs` adds `ExitSignalTag`, `exitReasonCode`, and `Actor.TrappingExits`; `scheduler.fs` adds `deliverMessage` (shared with `send`) and the trapping branch of `propagateLinks`; `components/prelude/evaluator/scheduler.fs` adds `trap-exit`, `exit-signal?`, and `exit-reason`. Tests pin that a trapped link survives and inspects the reason (1 = error) while an untrapped link still aborts. Suite: 915 checks (883 examples and 32 properties) and 72 granular tests.

Non-goals. Selective receive (D68 adds it); bounded mailboxes; distribution; and priorities. (D67 adds monitors and the named registry.)

D67 — Monitors and the named registry

Decision. An actor can monitor another actor one way. `monitor` takes a handle and returns a reference number; `demonitor` removes it. On any exit of the target (normal or abnormal) a `DownTag` message carrying the reference and reason is delivered to the watcher's mailbox once, and the monitor is consumed. `register`/`unregister` map a text name to an actor; `whereis` returns the actor's private handle. Names are removed when the actor terminates, and a live name cannot be re-registered.

Why. Links are symmetric and kill; traps observe only linked exits. Monitors give one-way observation without altering the target's behavior, and a registry gives actors a stable name without exposing an address. Both reuse the mailbox delivery path and the handle capability.

Encoding. `DownTag = -30.0`; a Down message is `[DownTag; monitorRef; reasonCode]`. Monitor references are per-run integers validated against `SchedulerState.Monitors`; a forged or foreign reference is `NotAMonitor`. The registry is `SchedulerState.Registry: Map<string,`

EntryIdentity>; whereis resolves the actor and returns its handle, so a name grants targeting like register/whereis in Erlang. Both maps are execution state, never serialized.

Boundary. A monitor fires on the target's exit, not on a restart as a new identity; an explicit kill of the target fires it, and D68 makes a cascading kill fire the monitors of every terminated descendant. Selective receive was the one actor-model operation still absent at D67; D68 adds it.

Status: applied. types.fs adds DownTag, Monitor, the scheduler maps, and the three refusals; scheduler.fs adds monitor/demonitor/notify and registry operations; components/prelude/evaluator/scheduler.fs adds monitor, demonitor, down?, down-ref, down-reason, register, unregister, and whereis. Tests pin a normal-exit Down, demonitor, name removal on termination, and NameInUse. Suite: 930 checks (898 examples and 32 properties) and 73 granular tests.

Non-goals. Selective receive (D68 adds it); bounded mailboxes; distribution; and priorities. With D68 the actor model is otherwise complete.

D68 — Closing the actor boundaries

Decision. receive-matching takes a predicate quotation and returns the oldest queued message for which the predicate is true. Non-matching messages stay in the mailbox in their original order; when nothing matches the actor blocks, and a later send wakes it to rescan. send and send-event accept either a channel or an actor handle, resolving a handle to that actor's mailbox; receive and close remain channel-only. When a supervisor exhausts its restart budget, its children stop with ShutdownExit and the supervisor exits with the limit error to its own supervisor, so the root surfaces the error only if the chain reaches it. A cascading kill fires the monitors of the target and every descendant it terminates.

Why. Selective receive was the last missing actor operation: without it a receiver could only take the head of its mailbox. Unifying the send target removes a distinction the caller does not need, because a handle is already a capability. Propagating a supervisor's limit failure to its parent is what supervision means; aborting the run in place made a temporary supervisor's breach fatal and left its parent unable to apply a policy.

Encoding. SchedulerState.SelectiveWaiters: Map<EntryIdentity, EntryIdentity list> records selective receivers by mailbox. It is execution state, never serialized, and a terminated actor is removed from every list. A send wakes a selective waiter without dequeuing, so the actor rescans the queue itself. The predicate runs through EvaluatorQuotationRun.run; a predicate failure is the named ValueSetUndecided, never a silent non-match. messageTargetId resolves an actor handle to Actor.Mailbox and leaves a channel reference or numeric channel id alone. killActorAndNotify collects the target's subtree, kills it, and notifies each terminated actor's monitors with KilledExit.

Boundary. A selective receive is one predicate quotation scanned oldest-first; there is no timeout and no multi-clause form. receive-matching is an actor word and is refused inside a quotation that quotationRun or a guard may execute. send-to remains the direct mailbox word; the unified send adds mailbox targets rather than replacing it. Bounded mailboxes, distribution, and priorities remain non-goals; with D68 the actor model is otherwise complete.

Status: applied. types.fs adds SelectiveWaiters; scheduler.fs adds selective registration, selective wake in deliverMessage, and killActorAndNotify; components/prelude/evaluator/scheduler.fs adds receive-matching; components/prelude/evaluator/channel.fs adds messageTargetId and uses it for send; components/prelude/evaluator/channel-event.fs uses it for send-event; components/prelude/evaluator/quotation-run.fs refuses the new actor word.

Tests pin a non-matching message staying queued, a selective receiver waking on a later send, a handle as a `send` and `send-event` target, a supervisor's limit error reaching its own supervisor, and a cascading kill firing a descendant's monitor. Suite: 940 checks (908 examples and 32 properties) and 73 granular tests.

Non-goals. Bounded mailboxes; distribution; priorities; timeouts and multi-clause selective receive. D69 unifies the actor-word guard and closes the nested-quotation path.

D69 — One actor-word guard for quotation runs and set literals

Decision. `EvaluatorQuotationRun.actorWords` is the single list of words that change or block the actor. A quotation run scans the entire quotation before executing it, so a nested quotation or an `apply` cannot smuggle an actor word past the guard. A set literal uses the same list and the same pre-scan; both refuse the named word. The list now covers the mailbox, link, monitor, registry, send, close, and selective-receive words added by D65–D68, not just the scheduler words.

Why. D68 added `receive-matching` to the quotation-run list, but the set literal kept a private copy that did not have it; the two guards had already drifted. The per-token check also saw only top-level tokens, so a nested `{ ... }` or an `apply` could run an actor word inside a quotation. One list and one pre-scan make the rule the same everywhere and close the nested path.

Encoding. `components/prelude/evaluator/quotation-run.fs` owns `actorWords`; `components/prelude/evaluator/value-sets.fs` reads it instead of a private list. `run` splits the quotation and refuses the first actor word before building the probe state; the set literal scans its collected body the same way. Read-only actor words (`self`, `whereis`, `down?`, `exit-signal?`) stay available, and event constructors stay available until `sync` commits.

Boundary. The guard is conservative: naming an actor word inside a quotation that is never applied is still refused. A set literal and a quotation run agree on every word. The list is a policy list, not an effect system; a future word that changes the actor must be added here.

Status: applied. Tests pin the set-literal refusal for `halt`, `receive-matching`, `receive-message`, `send-to`, and `link`; the quotation-run refusal for the four mailbox words; a killed selective waiter leaving no `SelectiveWaiters` entry; and the persisted-state boundary resetting `SelectiveWaiters`. Suite: 953 checks (920 examples and 33 properties) and 74 granular tests.

Non-goals. A general effect system; making read-only actor words illegal; changing the event constructors.

D70 — Discourse rollback (P3)

Decision. One utterance is one attempt. `Discourse.utter` runs it and publishes the resulting violation. An `Input` violation at `Structural` or above rejects the utterance and restores the state before it; a `CoverageGap` does not block. A rejected utterance keeps the pre-utterance image; the rejected utterance's entries are not exposed.

Why. P0–P2 made every failure one blamable `Violation`; P3 is the policy that decides what a bad utterance does to the conversation. The blame-conservative rule is the one the pipeline already describes: an input fault blocks, a coverage gap is recorded. Without it, a failed utterance leaves a partial stack and the next utterance starts from a state the user did not intend.

Encoding. `discourse.fs` defines `Utterance` (`Accepted`, `Committed`, `State`, `Violations`) and `sameObservable`. `Accepted` says the policy did not reject the attempt; `Committed` says the returned state is the attempt's final state. An accepted attempt is committed; a rejected attempt is neither; a

coverage-gap attempt is accepted but not committed, because `Evaluator.run` has no final state to return and the observable state stays at the pre-utterance value. `layerRank` orders `Lexical` (0) through `Discourse` (6); `rejects` is `Blame = Input && rank >= Structural`. The store is append-only and is not part of the equality.

Boundary. The discourse layer is a host policy over one completed attempt; it does not yet publish through an actor mailbox or a `ChannelEvent`. TODO: add that actor/ChannelEvent publication. A coverage-gap failure still surfaces as a violation and leaves the state unchanged. Character offsets, the minimum-edit repair search (D71), and the presentation layer (P5) remain open.

Status: applied. `discourse.fs` is compiled before `properties.fs` and the tests. Tests pin an accepted and committed utterance, an input-fault rollback with state equality, a coverage-gap pass-through that is accepted but not committed, and a later utterance starting from the committed state. The Hedgehog property `discourse rollback restores the state after an input fault` passes. Suite: 962 checks (928 examples and 34 properties) and 75 granular tests.

Non-goals. Actor/ChannelEvent publication; P4 minimum-edit repair (D71); P5 presentation; character offsets.

D71 — Minimum-edit repair search (P4)

Decision. `RepairSearch.search` searches the fixed parser for the nearest accepted sentence. A candidate is one delete, insert, replace, or adjacent swap over the canonical token list, using the active vocabulary plus `.` and `,`; the search stops at the first distance with an accepted candidate, so every result is a minimum edit. `repair` returns the first candidate, and an already accepted sentence is its own zero-edit repair. `searchWith` adds a caller-supplied layer check and distance bound; that is the sortal hook, where several candidates are ranked by distance and then by tokens.

Why. P0–P3 report and roll back failures, but the user still needs the nearest legal sentence. The grammar is fixed, so the repair is a search, not a rewrite of the specification. The existing repair hint was hand-written per refusal; one search gives every layer the same candidate shape and makes idempotence a property rather than a convention.

Encoding. `repair-search.fs` defines `Candidate` (`Tokens`, `Text`, `Distance`); `edits` builds the one-edit neighbourhood; `accepted` is the fixed parser plus the layer check; `searchWith` iterates distances; `search` and `repair` are the default parser-only path. The vocabulary supplies the words, and the literal `./,` tokens cover the punctuation the tokenizer does not store as vocabulary entries. `Text` is the canonical token text, not source formatting.

Boundary. The default search is one edit. A two-edit repair needs an explicit `maxDistance` and costs a larger frontier. Sortal candidates are ranked only through the caller's `validate` hook; the search does not invent a semantic check. The presentation layer (D72) and character offsets remain open at D71. The showcase now uses `RepairSearch.repair`; TODO: add character offsets and wire the P4 search into the CLI.

Status: applied. Tests pin the missing full stop, the missing quantifier, idempotence, multiple ranked candidates, and the layer hook. The Hedgehog property `repair is idempotent on an accepted sentence` passes. Suite: 976 checks (942 examples and 34 properties) and 75 granular tests.

Non-goals. Redesigning the parser or the vocabulary; semantic repair without a layer check; P5 presentation (D72); character offsets.

D72 — Presentation layer (P5)

Decision. `Presentation.render` turns one `Violation` into a layer-aware explanation for a non-logician: a plain-language headline for the layer, the source token, expected/given, the blamed party, the evidence (model/unsat core/shrunk sentence), and the repair. `renderTechnical` adds the stable rule identifier for a developer or log. `withNearest` fills a missing repair from the P4 search. The ordinary rendering never prints the rule id.

Why. P0–P4 produce a precise value, but a non-logician should not have to read `MissingFullStop` or `Sortal` to act. The layer selects the explanation; the expected/given pair says what was wrong; the witness is the evidence; the repair is the next action. Keeping the technical rendering separate preserves the stable identifiers for logs without leaking them into the user output.

Encoding. `presentation.fs` defines `render`/`renderTechnical`; `layerHeadline` maps the seven layers to prose; `blameLine` separates input faults from coverage gaps; `witnessLines` renders `UnsatCore` and `Model` (through `Smt.describeModel`); `optionalLine` renders expected/given/repair; `spanLine` renders the canonical token or "the whole sentence". `withNearest` runs `RepairSearch.repair` only when the violation has no hand-written hint.

Boundary. `spanLine` renders the canonical token; D76 adds optional character offsets when `Violations.parseText` supplies the source. The ordinary rendering is English prose, not a proof translation. A P5 render is presentation only: it changes no verdict and no acceptance gate.

Status: applied. Tests pin the structural, sortal, semantic, coverage-gap, and discourse renderings, the expected/given and span lines, the absence of a rule id in the ordinary output, the technical rendering, and the nearest-repair fill. Suite: 986 checks (952 examples and 34 properties) and 76 granular tests.

Non-goals. Proof-log translation; changing a verdict; a second renderer for each stage.

D73 — Promoting decision tables (D13)

Decision. Decision tables are promoted as a library-level surface, not a new Basic English sentence form. A domain declares a `DecisionTable.Table`, a `DecisionTable.TableBinding` (subject, verb, and one body per outcome), and calls `DecisionTable.check` plus `DecisionTableEmitter.emit`. Each row becomes its own emitted property module with the row's guards as the precondition and `rowN` as the body; the host domain supplies `row0`, `row1`, and so on. `DecisionTable.obligationFor` exposes the same row as an `Smt.Obligation`, so `guards ∧ ¬body` is discharged by `Smt`.

Why. D13 stopped at a prepared, optional table because a surface form and a per-row emitter binding were unresolved. The promotion keeps the grammar closed (D8/D18) and the optional layer's promise: a domain that never declares a table is unchanged. The per-row binding is the `Body: 'outcome ->
Expression` function, which is exactly the missing decision.

Encoding. `decision-table.fs` adds `TableBinding`, `obligationFor`, and `obligations`; `decision-table-emitter.fs` adds `emit`, which suffixes the module name per row, names `rowN` as the body, and reuses `Emitter.emitFSharp` with the row's guards as refinements. `DecisionTable.explain` maps a `NoMatch` selection to a coverage-gap `Violation` and an `Ambiguous` selection to a table-design-fault `Violation`; it is an adapter, not a sink. The two-row demo is pinned in `snapshots/decision-table.verified.txt`; `just test-granular` checks it. Completeness and disjointness remain `DecisionTable.check`.

Boundary. The promotion is host-level: no new word, no parser change, and no vocabulary change. D74 adds the Basic English table surface and the diagnostic sink. The emitter assumes the host supplies `rowN`; the generated artifact is not self-contained. The completeness obligation is the existing SMT check, not a new emitted property. TODO: add a CLI table mode.

Status: applied. Tests pin the row modules, the per-row guards, the row body names, byte-identical re-emit, the row obligation shape, an SMT discharge of a row, and the no-match/ambiguous explanation. The demo snapshot `decision-table` is pinned. Suite: 1018 checks (984 examples and 34 properties) and 76 granular tests.

Non-goals. A self-contained per-row body; a second completeness renderer; ML decision models.

D74 — The Basic English table surface and the diagnostic sink

Decision. A table document is a `table <name>` header, an optional `policy: unique|first|priority` line, Basic English transition rows, and `end`:

```
table jump-or-run
policy: unique
when player has health at least 2, every player can jump.
when player has health at most 1, every player can run.
end
```

A row's trigger conditions become its guards, its verb becomes the outcome, and the first row's subject is the table subject; every row must name the same subject. Input sorts are inferred from the guards (numeric -> integer, boolean -> boolean). `DecisionTableSurface.parse` lives beside the host table and does not change the one-sentence `.scale` document. `Diagnostics.Sink` is the common sink: `add`, `addOption`, and `addSelection`; `render` and `renderTechnical` go through `Presentation`.

Why. D73 promoted the host table but left the surface and sink open. A table of Basic English transitions is the smallest surface that reuses the parser, guards and emitter without adding a new grammar for conditions. The sink is a pure ordered list, so it can be threaded through a run and later published as an actor or `ChannelEvent` without changing the violation shape.

Encoding. `decision-table-surface.fs` defines `TableDocument`, `TableError`, `parse`, `toTable`, `binding`, and `describeError`. Rows are parsed with `Parser.parseDetailed`; a row must be a `Transition`. `ComparisonEqualTo` becomes `AtLeast` + `AtMost`. A `Disjunction` row is refused as `UnsupportedRowCondition "disjunction"` because `DecisionTable.Row.Guards` is a conjunction. `diagnostics.fs` defines `Sink` and the selection adapter.

Boundary. The surface is a document format, not a new sentence form: the parser and vocabulary are unchanged. A disjunctive row, a `NamedGuard` row, and an outcome body are not part of the surface; the host supplies the body. The sink is library-level; `--table <path>` and `--diagnose <sentence>` are the CLI modes. Character offsets remain TODO.

Status: applied. Tests pin the parsed table, inferred inputs, soundness, selection, per-row emission, a gap and an ambiguous sink, and the named surface refusals. The showcase parses and selects a table. Suite: 1035 checks (1001 examples and 34 properties) and 76 granular tests.

Non-goals. A new sentence grammar; row-level disjunction; NamedGuard rows; a CLI table mode; character offsets.

D75 — Reserved actor system messages

Decision. Every actor has one reserved message class handled by the runtime, never by user code. The class is `[SystemMessageTag; operation; payload…]` and covers at least Suspend, Resume, Code-Change, Status, and Shutdown. `deliverMessage` checks the class before it delivers a message to a user stack or appends it to a mailbox queue.

Why. An Elixir-to-Scale compiler needs OTP-style process control: suspend and resume a process, replace its code, query its status, and shut it down. Those operations must work even while user messages are queued and must not be observable as ordinary mailbox messages. The extension is deliberately one pre-stage, not a new MOP surface or a second scheduler.

Encoding. `SystemMessageTag = -31.0` carries an operation code; `SystemStatusTag = -32.0` is the status reply. `Actor.Suspended` is execution state and is never serialized. `suspendActor` marks the actor and removes it from the ready queue; `resumeSuspendedActor` clears the flag, requeues a ready actor, and wakes a blocked actor only when its mailbox has a queued message. `handleSystemMessage` is the runtime pre-stage. Code-Change reads a quotation and replaces the suspended actor's program, resets its location and position, clears the pending error, and leaves the actor ready but suspended. Status replies with `[statusCode; suspended; mailboxDepth]` to an optional reply channel. Shutdown captures the target subtree's links, terminates it with `ShutdownExit`, propagates the exit to live linked actors, and fires the monitors of every terminated actor.

Boundary. System messages are accepted only through an actor mailbox; an ordinary channel refuses them with `InvalidSystemMessage`. A system message never enters the mailbox queue and never reaches `receive-message`. Self-targeted Suspend, Resume, Code-Change, and Shutdown are refused; use normal control flow inside the running actor. The mailbox structure, MOP, persistence, and linear regions are unchanged, and there is no new user word.

Status: applied. Tests pin suspend/queue/resume, the status reply, code-change, shutdown with its link and monitor notifications, refusal outside a mailbox, and the guarantee that a system message is not visible to user code. Suite: 1044 checks (1010 examples and 34 properties) and 76 granular tests.

Non-goals. User-defined system operations; system messages on ordinary channels; self-targeted control; a code-change user callback (the runtime replaces the program).

D76 — Character offsets for source violations

Decision. A `Violation.Span` carries an optional `Start` and `End` character range into the original sentence. `Violations.parseText` tokenizes and parses the sentence, then fills the range from the offending surface form; the token-only `parse` keeps the range absent. `Presentation` renders the range as `characters start-end`, with `End` exclusive.

Why. A token name is enough for a machine, but not for a person reading a source diagnostic. The tokenizer does not retain source positions and the parser errors carry canonical tokens, so a full token-index rewrite is a separate decision. The first offset step is deliberately at the violation boundary: it locates the surface form the canonical token came from without changing the grammar, the parser, or the token contract.

Encoding. `Span` is `{ Token; Start: int option; End: int option }`. `surfaceForm` resolves a canonical token to its active vocabulary form; `withOffsets` finds the first case-insensitive occurrence and sets the exclusive end. An empty token (the whole sentence) has no range, and a form the source does not contain leaves both offsets absent. `Violations.parseText` is the source-

text entry point; the showcase and `--diagnose` use it. `End` is exclusive, so the range is a slice bound rather than a visible last character.

Boundary. A repeated surface form uses its first occurrence; the tokenizer does not yet retain a token index. A canonical multiword form uses its full vocabulary form. Semantic violations have no source text, so their ranges stay absent. The token-only API is unchanged and still produces offset-free spans; offsets are presentation data, never a verdict.

Status: applied. Tests pin an unknown token's start and exclusive end, the rendered character range, and the offset-free token-only path. Suite: 1051 checks (1017 examples and 34 properties) and 76 granular tests.

Non-goals. A token-index rewrite; disambiguating repeated surface forms; offsets for semantic violations without a source sentence.

D77 — A syntactic discourse contradiction detector

Decision. `Discourse.contradicts` detects the two direct contradictory pairs over one claim: Every/No and Some/No. A claim is the same subject, the same ordered condition list, and the same verb. `Discourse.contradiction` finds the first committed statement a candidate contradicts and returns one `Layer.Discourse` violation with rule `Contradiction`. Transitions are excluded.

Why. The Discourse layer existed only as a rollback policy; no production detector produced it. A complete semantic consistency check needs a committed theory and a solver encoding, which is a larger decision. The direct Every/No and Some/No pairs are the smallest useful detector, and naming the limitation keeps the layer honest.

Encoding. `discourse.fs` adds `statementQuantifier`, `contradictory`, `sameClaim`, `contradicts`, and `contradiction`. The violation carries the candidate's subject as its span, `a consistent statement` as expected, the subject and verb as given, `Blame.Input`, and no repair. The showcase parses a committed statement and a candidate and renders the contradiction. The detector uses exact ordered conditions; it does not rewrite or reorder a sentence.

Boundary. This is a syntactic detector, not a complete consistency checker: it does not use SMT, it does not compare paraphrases, and it does not handle transitions or different condition orders. It is a named production detector for the Discourse layer, not a replacement for the SMT gate.

Status: applied. Tests pin Every/No, Some/No, a different verb, the violation shape, and the consistent case. Suite: 1052 checks (1018 examples and 34 properties) and 76 granular tests.

Non-goals. A semantic theorem prover; paraphrase matching; condition normalisation; transition contradictions; a repair for a contradiction.

D78 — The SMT violation in the showcase sink

Decision. The showcase runs one SMT obligation through `Smt.createFromLattice DemoDomain.lattice DemoDomain.canJumpExpression`, discharges an empty-condition `Capability`, converts the result with `Violations.ofSmtDischarge`, and adds it to the same `Diagnostics.Sink` as the parser, table, and discourse examples.

Why. The SMT detector existed as a library adapter but was not visible in the showcase sink. Wiring it there proves that a semantic counterexample and a coverage gap use the same value, sink, and presentation path as the other detectors. It is a wiring decision, not a new verdict.

Encoding. `bases/scale-cli/entry.fs` creates the encoding from the demo lattice, uses `use` to dispose it, discharges the obligation, and calls `Diagnostics.addOption` with the optional violation. `--diagnose` remains a single-sentence path; the showcase is the multi-detector sink demonstration.

Boundary. The showcase pins one falsified property from the demo domain. It does not add a CLI SMT mode, change an acceptance gate, or add a new SMT encoding. A solver `Unknown` or `Unavailable` would still be a coverage-gap violation through the same adapter.

Status: applied. Tests pin the sink accepting a falsified SMT violation; the showcase renders it. Suite: 1051 checks (1017 examples and 34 properties) and 76 granular tests.

Non-goals. A CLI SMT mode; a second SMT renderer; accepting an undecided obligation; changing the proof gate.

D79 — Typed proposals, advice, and the frozen specification

Decision. Option D is applied. The host verification model now has a frozen specification, implementation proposals, specification proposals guarded by an exploratory capability, and diagnostic advice. The mixed `Patch` type with an optional specification edit is gone.

Why. The earlier model represented a specification edit as an optional string inside an implementation patch, duplicated the fixed-spec rule in `accept` and `appendInMode`, and used a string `Violation.Repair` for advice. A proof-backed acceptance, a specification refinement, and sentence advice are three different objects. The type model now says so.

Encoding. `repair.fs` is now `fixed-specification-acceptance.fs`, module `FixedSpecificationAcceptance`; `repair-search.fs` is now `nearest-sentence.fs`, module `NearestSentence`.

- `ImplementationProposal` carries a body and the specification identity it was built against. It has no specification edit.
- `FrozenSpecification` pairs the target with a private `SpecificationIdentity`.
- `accept` takes the frozen specification and an implementation proposal; it checks their identities, then runs the existing sorted-body, proof, corpus, and differential rules.
- `SpecChangeCapability` has a private constructor and is granted only by `specChangeCapability Exploratory`; `SpecificationProposal` is produced from that capability and appended only when the document hash still matches.
- `NearestSentence.nearest`, `candidates`, and `nearestWith` replace the old search names; `NearestSentence.advice` produces `Violations.Advice`.
- `Violation.Suggestion` replaces the string `Violation.Repair`; presentation renders `Suggestion:`. `Presentation.withNearestSentence` fills the advice.
- `AcceptanceBundle` replaces `RepairBundle`. The bundle JSON wire names are unchanged.

Boundary. The capability is a private-constructor value, not a type-theoretic linear type; F# can prevent ordinary construction but cannot prove exactly-once use. Fixed mode has no capability. The acceptance gate never calls sentence search. There is no combined verdict-plus-advice facade; the CLI keeps `--diagnose` and `--fixed-spec` separate. Historical decision records keep their old names.

Status: applied. Tests pin proposal acceptance, numeric targets, corpus regression, falsification, ill-sorted bodies, a proposal against another specification, unencodable targets, differential disagreement, fixed mode without a capability, exploratory append, sentence advice, and the unchanged bundle JSON. Suite: 1057 checks (1022 examples and 35 properties) and 78 granular tests.

Non-goals. A combined edge facade; true linear types in the host; a new SMT rule; a grammar change; a user-facing Scale word; a spec edit in fixed mode.

D80 — Memory vocabulary, handles, views, and actor epochs

Decision. Scale distinguishes the image/store, an actor epoch, a tensor handle, and a tensor view. The image remains the authority for durable and cross-actor values. An actor epoch is a per-incarnation arena; a value crossing its boundary is promoted or copied. Handles and metadata may use `[<Struct>]`, and views may use `[<Struct; IsByRefLike>]`, as host performance choices; they are not semantic guarantees. `Tensor` is not a `ref struct`. The first arena backend is managed storage; unmanaged storage and full per-process GC are later decisions.

Why. The word "heap" had been used for the image, the CLR managed heap, a BEAM process heap, and an arena. Those are different things with different integration rules. The image is an append-only value store, not a heap. The CLR managed heap is a host detail; `double[]` payloads currently live there. `[<Struct>]` removes wrapper allocation, not the arrays inside the struct. `[<Struct; IsByRefLike>]` removes heap escape, but cannot be stored in `Map`, `EvaluationState`, an actor, a mailbox, a closure, or persistence. Neither annotation creates ownership, lifetime, aliasing, or promotion rules.

Scope. The immediate compatibility target is Elixir/BEAM. The Go transpiler is delayed and does not drive the memory model. A global mutable heap is rejected. A per-process tracing GC is not part of this stage.

Staging. Stage 0 is this vocabulary and the handle/view split. Stage 1 introduces the handle, view, and arena contract without changing behavior. Stage 2 adds the actor epoch and promotion boundaries. Stage 3 adds per-actor reclamation. Stage 4 decides copy-on-send and whether the arena is managed or unmanaged. Exact BEAM memory parity remains a separate runtime decision.

Integration rules. The image is append-only; execution state is never serialized; ephemeral identities never enter the image, mailboxes, persistence, or foreign stacks without promotion; a mutable arena lives outside immutable `EvaluationState` snapshots; a per-message arena is invalid because `yield` and `halt` can suspend inside a handler; finalization is separate from memory reclamation. Details are in `docs/memory-model.typ`.

Status: decided, stage 0. No runtime behavior changed. The next implementation step is the `TensorHandle`/`TensorView`/arena contract. Struct annotations are performance choices and an example for readers; they carry no semantic promise.

Non-goals. A classic heap; a global shared object graph; a full per-process GC now; unmanaged arenas now; a `ref struct` `Tensor`; struct annotations as semantics; Go-driven memory requirements.

D81 — Usage modes and trace linear types

Decision. Linearity is a usage mode on the type, not a property of the trace alone. A type carries `ExactlyOnce` (linear), `AtMostOnce` (affine), or `AnyNumberOfTimes` (unrestricted). `OwnershipTypes.checkTrace` checks one trace under one mode; the end-of-trace rule is the mode's obligation. `LinearType` is a named set of traces under one mode: the name is the nominal projection, the language is the extension, and it is the third projection of the sort record (D12). Union, intersect, and `nor` do not apply; linear composition is sequential.

Why. D54 had one `Owned` state, so it could not distinguish a leaked file from a dropped tensor: it rejected both, and it never checked the end of a trace at all. It also treated text and `:Name` literals as non-references even though they are image entries. The mode fixes both directions: a linear trace must not leave an owned reference open, an affine or unrestricted trace may drop it, an unrestricted reference may be duplicated, and a literal is a reference.

Boundary. This is a consumer of the set theory, not a new set algebra. A linear language is a set of traces, and the checker is membership. Composition is deliberately absent: `then` would have to run the second trace from the stack the first one leaves, not concatenate two standalone-valid traces. That is a relational model and its own decision.

Status: applied. `ownership-types.fs` (`UsageMode`, `LinearStep`, `checkTrace`, `LinearType`), `development/scale/tests.fs` (leak, affine drop, unrestricted borrow/duplicate, affine duplicate refusal, literal reference, language membership and projections). The `linearity` word keeps its default linear contract.

Suite. 1068 checks (1033 examples and 35 properties) and 78 granular tests.

Non-goals. Epoch labels in the image, automatic epoch close, unmanaged arenas, trace composition, and session types. Those are stages 2–4.

D82 — Reclamation roots include every live actor

Decision. `Reclaim.rootsOf` collects the roots of the current execution state, the suspended state of every non-terminated actor in `SchedulerState.Actors`, and the scheduler's channels and waiters. A terminated actor is not a root. With more than one live actor, an entry held by no live actor's stack, frames, handlers, tables, or ownership record is reclaimable.

Why. The scheduler already holds each actor's suspended state, so the per-actor stacks are visible. D52 treated every entry as a root when more than one actor existed, which made `compact` keep the whole image in the actor model. The snapshots are the missing roots, not a reason to give up.

Boundary. Reclamation stays conservative and opt-in: `reclaimable` only reports candidates, and `compact` is still an explicit step. The reference scan is unchanged, so numeric payloads can still look like addresses. This decision does not add epoch close, handle indirection, or concurrent compaction; those remain later stages.

Status: applied. `reclaim.fs` (`stateRoots`, `rootsOf`), `development/scale/tests.fs` (a suspended actor's stack is a root; an entry no live actor holds is reclaimable). The `reclaimable` and `compact` words are unchanged.

Suite. 1070 checks (1035 examples and 35 properties) and 78 granular tests.

Non-goals. Epoch labels, automatic close, copy-on-send, concurrent compaction, and exact reference tags.

D83 — Epoch allocation ranges and promotion

Decision. An epoch is a set of half-open allocation ranges in the shared frontier, not a second store. `EpochState` records the current epoch, the ranges each incarnation allocated, a promoted set, and the closed epochs. `recordAllocation` attributes frontier growth to the current epoch; `promote` reclassifies an entry as durable without moving it; `closeEpoch` ends an incarnation; `closedCandidates` names the unpromoted entries of closed epochs. Reachability from live roots remains the reclaimer's decision, so closing an epoch never frees a value a live actor still holds.

Why. The scheduler runs one actor at a time, so the frontier growth between two turns is exactly that turn's allocation. Recording ranges needs no change at the `Store.addTensor`/`addText` call sites. Promotion is a set membership change, so a durable value keeps its address and no copy occurs.

Boundary. `EpochState` is execution state and is never serialized; a persisted image contains durable entries only. The module is pure and does not touch the store: wiring it into the scheduler, automatic close, and the promotion boundaries is the next stage.

Status: applied. `epoch.fs` (`EpochState`, `beginEpoch`, `recordAllocation`, `promote`, `closeEpoch`, `closedCandidates`, `epochOf`) and `development/scale/tests.fs` (range ownership, close candidates, promotion, frontier reset, epoch lookup, durable close, idempotent record).

Suite. 1078 checks (1043 examples and 35 properties) and 78 granular tests.

Non-goals. Scheduler wiring, automatic close, copy-on-send, unmanaged arenas, and persistence of ephemeral values.

D84 — Reflection.Emit is a JIT-only compiler backend

Decision. The language-to-.NET backend may use `System.Reflection.Emit` and `PersistedAssemblyBuilder`, but it is a build-time, JIT-only tool. It is unreachable from the NativeAOT entry point and excluded from the native publish by a separate project or a conditional `Compile` item plus a stub. The native runtime returns a named refusal instead of attempting to emit. The generated assembly is produced before publish and statically referenced by the AOT build.

Why. `PersistedAssemblyBuilder` is the practical path to a native .NET backend, but the emit APIs require runtime code generation, which NativeAOT cannot execute. A runtime `RuntimeFeature.IsDynamicCodeSupported` guard is not enough: the analyzer still sees the call and ILC still compiles the method. The split keeps the AOT binary warning-free and the emitter available where it belongs.

Status: decided. `AGENTS.md` section 9.5 and `docs/nativeaot.typ` carry the rule. No code changes yet.

Suite. 1078 checks (1043 examples and 35 properties) and 78 granular tests.

Non-goals. A runtime interpreter fallback for the emitter; loading a generated assembly with `Assembly.Load`; making the emitter AOT-compatible.

D85 — Epoch allocation recording in the scheduler

Decision. `OwnershipState` carries the actor incarnation's `EpochState`. `Evaluator.initial` opens the root epoch; a spawned child opens its own epoch at the shared frontier; `yieldControl` and `haltCurrent` record the frontier growth of the outgoing turn; `switchTo` resumes the incoming epoch at the current frontier without attributing the interleaved gap to it; `terminateActorWith` closes the epoch. Promotion and physical reclamation remain later stages.

Why. The scheduler runs one actor at a time, so the frontier delta between two switches is exactly that turn's allocation. Resetting the resumed frontier is what keeps interleaved actors from claiming each other's entries.

Boundary. `EpochState` is execution state and is never serialized. A closed epoch names its unpromoted entries; the reclaimer still decides what is reachable, so closing never frees a value a live actor holds.

Status: applied. `types.fs` (epoch data types, `OwnershipState.Epoch`), `epoch.fs` (`resumeAt`, `beginAt`), `components/prelude/evaluator/evaluator.fs` (root epoch), `scheduler.fs` (record, resume, close), `development/scale/tests.fs` (root epoch, yield range).

Suite. 1080 checks (1045 examples and 35 properties) and 78 granular tests.

Non-goals. Promotion at send/persist, automatic physical reclamation, copy-on-send, and unmanaged arenas.

D86 — Promotion at send boundaries

Decision. A value sent through `send` or `send-to` is promoted in the sender's incarnation epoch; `send-event` promotes its message; `sync` promotes the event it consumes. Promotion is a set-membership change on the existing address; no copy and no renumbering. Persistence is host-level and remains a later filtering decision.

Why. A message leaves the sender's epoch and must survive the sender's termination. Without promotion, a closed-epoch sweep could remove a value that a mailbox or channel still names. The send boundary is the single place where that escape is visible.

Boundary. `Promoted` is execution state and is never serialized. The reclaimer still decides reachability; promotion only marks a value as durable for a future closed-epoch sweep.

Status: applied. `epoch.fs` (`promoteAll`), `components/prelude/evaluator/channel.fs` (`sendWord`), `components/prelude/evaluator/channel-event.fs` (`sendEventWord`, `syncWord`), `development/scale/tests.fs` (send and send-event promotion).

Suite. 1082 checks (1047 examples and 35 properties) and 78 granular tests.

Non-goals. Physical reclamation of closed epochs, persistence-time epoch filtering, copy-on-send, and unmanaged arenas.

D87 — Host-level persistence filters by epoch

Decision. `Persistence.toBytesForState` and `writeToFileForState` filter an image to durable entries before serializing it. An entry is durable when it lies outside every actor epoch's allocation ranges or is explicitly promoted. The filter unions the live actor's epoch with every actor snapshot, including terminated actors, and drops unpromoted ephemeral entries from both the lexicon and the text cache. The allocation frontier is unchanged, so the result is a sparse image (D53).

Why. A persisted image must not carry execution state. The epoch data itself is never serialized; only the entries it licenses are. Filtering at the host boundary keeps the core image format unchanged and lets a caller persist a snapshot without a separate compaction pass.

Boundary. The caller must ensure any root that must survive points at a durable or promoted entry. Persisting a running actor's live stack is not the contract; execution state is not serialized, and a resumed run starts owning nothing.

Status: applied. `epoch.fs` (`durableEntries`), `persistence.fs` (`epochsOf`, `durableStore`, `toBytesForState`, `writeToFileForState`), `development/scale/tests.fs` (ephemeral filtered, frontier kept, promoted survives).

Suite. 1086 checks (1051 examples and 35 properties) and 78 granular tests.

Non-goals. Changing the core image format, persisting execution state, physical reclamation of closed epochs, and copy-on-send.

D88 — Physical reclamation of closed epochs

Decision. `Reclaim.compactClosedEpochs` removes the unpromoted entries of every closed actor epoch, across the live actor and all snapshots, but only where no live root reaches them. A promoted entry is never a candidate, and a live stack, channel, mailbox, frame, or table root keeps its entry. The allocation frontier and surviving addresses are unchanged. The scheduler calls it at yield, halt, and kill, so a terminated actor's dead entries leave the image without a separate compaction word.

Why. Epoch close names a precise candidate set; the live-root check keeps the removal safe even if a promotion boundary was missed. Intersecting the two avoids the whole-image reference scan when no closed epoch has entries left.

Boundary. Reclamation is physical but sparse: addresses are never renumbered, and `NextIndex` still records the frontier. It runs on the shared store at scheduler safe points, not during an eval.

Status: applied. `reclaim.fs` (`closedCandidatesForState`, `compactClosedEpochs`), `scheduler.fs` (yield/halt), `components/prelude/evaluator/scheduler.fs` (kill), `development/scale/tests.fs` (closed entry reclaimed, promoted kept, live root kept).

Suite. 1089 checks (1054 examples and 35 properties) and 78 granular tests.

Non-goals. Copy-on-send, unmanaged arenas, a tracing GC, and renumbering surviving addresses.

D89 — TensorHandle, TensorView, and the managed arena contract

Decision. `Arena` introduces a stable `TensorHandle` (arena, offset, length, rank, generation), a stack-only `TensorView` over `ReadOnlySpan`, and a managed backing arena. `allocate` returns a handle; `resolve` returns the tensor when the generation matches; `viewOf` projects a tensor into a view; `close` increments the generation and drops the backing map. A closed arena invalidates every handle from its previous generation. The view is byref-like, so the compiler forbids storing it in a record, option, class, closure, or persistence.

Why. Stage 1 of the memory model needs the handle/view split before any scheduler or store integration. The handle is storable and stable; the view is transient and zero-copy. Struct annotations are host performance choices, not semantic guarantees.

Boundary. The backing arena is managed storage and execution state: never serialized. The image remains the authority for durable values. This step does not route the evaluator through the arena; it establishes the contract and its generation safety.

Status: applied. `arena.fs` (`TensorHandle`, `TensorView`, `ManagedArena`, `allocate`, `resolve`, `isLive`, `viewOf`, `close`), `bases/scale-cli/scale.fsproj`, `development/scale/tests.fs` (handle round-trip, view read/shape, close invalidation, stale generation).

Suite. 1093 checks (1058 examples and 35 properties) and 78 granular tests.

Non-goals. Routing `Tensor`/`Store` through the arena, unmanaged storage, driver integration, and a per-process GC.

D90 — Memory representation freeze

Decision. The memory representation is frozen for the native port: `Tensor` remains `{ Data: double[]; Shape: int[] }`; the image remains the append-only authority; the first backing tier is managed storage. Values are shared by reference because they are immutable, and crossing an actor boundary promotes the address (D86); copy-on-send is rejected. The evaluator is not routed through

the arena: `TensorHandle`/`TensorView` (D89) are a host performance seam for a later tier, not a second value representation. Unmanaged storage and a per-process tracing GC remain later, optional decisions.

Why. A port IR must encode the stack, history, and dispatch of the representation it targets. Freezing the image/Tensor representation now lets P1 proceed without encoding a dying model. Copy-on-send would duplicate large immutable tensors and contradict the promotion boundary. Managed backing keeps the existing persistence and AOT story intact.

Boundary. The freeze is a decision, not a prohibition on a future unmanaged backend: changing it requires a new decision record and a revisit of the port IR. Execution state (ownership, epochs, handles) is never serialized.

Status: applied. The no-copy send test pins the sharing decision; D80–D89 carry the rest. The port's Phase 1 is now unblocked.

Suite. 1095 checks (1060 examples and 35 properties) and 78 granular tests.

Non-goals. Unmanaged storage, copy-on-send, a tracing GC, and routing the evaluator through the arena.

D91 — Real and Natural sorts in the SMT encoding

Decision. The verification encoding carries four field sorts: Boolean, Real, Integer, and Natural. Real is encoded on the SMT `Real` sort and selects `QF_LRA`; Integer and Natural are encoded on `Int` and select `QF_LIA`; a domain with both uses `QF_LIRA`. Natural carries the domain constraint `n >= 0`, so a negative Natural is unsatisfiable by construction rather than filtered at run time. The integer path is unchanged, so every existing integer artifact and snapshot stays byte-identical.

What lands. `DomainExpression.FieldType` gains `RealType` and `NaturalType`; `realValue` requires a finite literal, and `naturalValue` requires an `Int32` value `>= 0`. `Smt.Value` gains `RealValue`; `createDomainFromLattice` emits `Real`/`Int` declarations and the Natural constraints; `modelOf` reads `RatNum`/`IntNum` and skips `AlgebraicNum`; `evaluateBody` compares a Real field through `MkInt2Real`; and `obligationText` chooses `QF_LRA`/`QF_LIA`/`QF_LIRA` and prints rationals such as `(/ 3.0 2.0)`. The emitter's `validateGuardThresholds` takes the domain lattice and validates each numeric guard through the public `TypeLattice.validateThreshold`: Integer and Natural require an `Int32` integer, Real accepts a finite double, and a numeric guard on a Boolean field is a named error. Without a lattice it checks finiteness only, because the emitted artifact is a plain F# `float`. Nothing is rounded. `Refiner.tighten` takes a quantizer, a tolerance, and a clamp; `refineSymbolic` dispatches on `TypeLattice.fieldTypes`: Real uses identity and a `1e-9` tolerance, Natural floors and clamps at zero with a `1.0` tolerance, and Integer/default floors with a `1.0` tolerance. A candidate equal to the last failing threshold stops the search, and bisection returns the first proving threshold (the upper bound) when it cannot make progress.

Why. A sort is a fact about a value. Encoding a fractional threshold as an integer silently changed the specification, and treating a Real field as an Integer made the solver's model and the emitted F# predicate disagree. Natural's non-negativity is a domain axiom, so Z3 can use it in a proof rather than the refiner sampling around it.

Trust boundary. `Proved` still means the obligation is unsatisfiable; `Unknown` and `Unavailable` remain failures, never a pass. A non-integral threshold on an Integer or Natural field is refused, not rounded; a non-finite Real threshold is refused. The Real refiner returns a threshold within its tolerance of the true boundary and never widens a guard past a proof.

Status: applied. `domain-expression.fs`, `smt.fs`, `type-lattice.fs` (`validateThreshold` is public), `refiner.fs`, `emitter.fs`, and `development/scale/tests.fs` (twenty checks under "SMT sorts (D91)"). The checks cover a Real proof with `QF_LRA`, its declaration and rational text, evaluation on both sides of the bound, a Natural proof with `QF_LIA` and `(>= n 0)`, rejection of a negative Natural, a mixed `QF_LIRA` domain, a fractional emitter guard, and a Real symbolic refiner returning a threshold in `[1.5, 2.0)`.

Suite. 1115 checks (1080 examples and 35 properties) and 78 granular tests.

Non-goals. Algebraic reals in models, non-linear Real arithmetic, rational-valued F# artifacts, complex numbers, and changing the integer path or its pinned artifacts.

D92 — Sort-faithful F# emission

Decision. When a domain carries a lattice, the emitted F# artifact is sort-faithful: a `RealType` field is `float`, an `IntegerType` or `NaturalType` field is `int`, and a `BooleanType` field is `bool`. Numeric literals follow the field's sort: an Integer or Natural guard prints an integer literal, and a Real guard prints the fractional literal. Without a lattice the emitter keeps its historical `float` default, so the self-contained no-lattice artifacts and their pinned snapshots stay byte-identical.

What lands. `Emitter.fsharpType` maps a `FieldType` to its F# carrier; `numericLiteral` prints a guard threshold for the field's sort; `fieldSortOf` resolves a binding's lattice into a field-sort lookup and refuses a field the lattice does not name. `conditionExpression` and `guardExpression` take the field sort, and `recordFields` derives each generated record field's type from the same lattice. `validateGuardThresholds` refuses a Boolean guard and a guard that names a field absent from the lattice before emission. `DemoDomain.Player.Health` is now `int`, and the demo, harness, entry sample, and tests construct it through `DomainExpression.integerValue`. The refined demo artifact carries `player.Health > 1`; its SMT companion still declares `health` as `Int`.

Why. An emitted F# predicate is a claim about the same value the SMT obligation describes. A `float` field under an integer sort lets `1.5` type-check where the solver refuses it, and a fractional literal in an integer guard changes the proposition. One sort, carried from the lattice into both backends, removes that disagreement by construction.

Trust boundary. A numeric guard on a Boolean field is a named refusal; a guard that names a field the lattice does not declare is a named refusal; a non-integral threshold on an Integer or Natural field is refused rather than rounded. The no-lattice path still emits `float`, because there is no lattice fact to carry. `Proved` and `Falsified` keep the meaning D91 gave them.

Status: applied. `emitter.fs` (`fsharpType`, `numericLiteral`, `fieldSortOf`), `demo-domain.fs`, `demo.fs`, `bases/scale-cli/entry.fs`, `development/scale/tests.fs` (three checks for the emitted record carriers), and `components/demo/demo/generated/PlayerSpecificationRefined.fs`. The checks cover a Real record field emitted as `float`, an Integer record field emitted as `int`, and a Natural record field emitted as `int`.

Suite. 1118 checks (1083 examples and 35 properties) and 78 granular tests.

Non-goals. A rational or decimal F# carrier, changing the no-lattice artifacts, and changing the integer SMT path or its pinned artifacts.

D93 — The unified language is the programmable IR beneath Basic English

Decision. The unified Julia/Elixir language is the programmable IR and semantic core one level below Basic English. Basic English is a restricted, declarative front-end that compiles into the unified

language. A programmer may write the unified language directly to use its full expressiveness; Basic English may expose only a subset. Scale has no independent implementation or semantics separate from the unified language: its features are implemented in the unified language. The current Scale implementation is the starting implementation of the unified language, reused, extended, or replaced per component. The unified language owns its AST, type hierarchy, `method` and `clause` dispatch, values, modules, macros, and concurrency. Verification is a unified-language feature, not a separate runtime.

What lands. `docs/unified-language.typ` is revised from a side-by-side split to one implementation stack: Basic English compiles into the unified IR, direct programs use the same IR, and the current implementation is the starting implementation. The collision register becomes a reuse/gap register. The test plan starts with current-behavior characterization, Basic English compilation goldens, direct-versus-compiled equivalence, reuse/gap tests, type projection, the dispatch target, the value/runtime target, and the AOT boundary.

Why. A single semantic core gives Basic English and direct programmers the same semantics. It keeps the full language available without making Basic English carry every construct. A second runtime would duplicate dispatch, values, and execution semantics and let the two drift.

Trust boundary. A Basic English specification still proves only its declarative contract. A unified program is not verified merely because it type-checks; the verification feature maps only the declarative subset to `FieldType`, and everything else is `Unavailable`. `Proved`, `Falsified`, `Unknown`, and `Unavailable` keep their meaning.

Status: recorded (documentation). No source or test change.

Non-goals. A separate Scale runtime, verifying arbitrary unified programs, and exposing every unified construct in Basic English.

D94 — Evaluation errors are one taxonomy with typed fault families

Decision. `EvaluationError` is one discriminated union of 41 outer cases whose related faults are grouped into typed families: `UnknownKind`, `ErrorSubject`, `ExpectedValue`, `ArityFault`, `ChannelFault`, `SortFault`, `LimitFault`, `EnumFault`, and `ActorFault`. A family case is always written qualified (`ArityError(ArityFault.InvalidTerm ...)`) because an unqualified union case resolves to the most recently compiled union and would collide with `ParseError` and `Outcome` cases. `describeError` and `Violations.ofEvaluationError` read the families and reproduce every previous message and violation rule exactly.

What lands. `types.fs` carries the family types and the 41-case union, `violations.fs` maps the families to their existing rule strings, and 250 call sites across 19 files and the suite are updated by a SageFs script that asserts each replacement. Two outer names moved to avoid shadowing: `Unknown` became `Unresolved of kind: UnknownKind * name: string`, and `ExpectedCondition` became `ExpectedConditionValue`.

Why. A flat union of dozens of cases made a single typo a compile-everywhere change and hid the shape of each fault. Grouping the faults makes an invalid arity, a channel failure, and a sort failure distinct types, so a handler can be exhaustive over one family at a time while the outer union still forces total handling. Messages and violation rules are unchanged, so the consolidation is behavior-preserving.

Status: implemented and verified (SageFs). The project rebuilds, and the granular suite reports 153 passed and 21 failed both before and after the change; the 21 are the pre-existing snapshot

failures. No regression was introduced. The snapshot failures were then fixed in `33cf99d`, and the suite reports 174 passed.

Non-goals. Splitting `types.fs` (step 2, deferred), changing any reported message or violation rule, or adding a new failure kind.

D95 — The kernel is the abstract machine behind one C ABI

Decision. The three-layer port in `Notes/multi-platform-port.md` defines the target shape. Where it conflicts with `docs/kernel-inventory.typ`, this record wins. The kernel is the abstract machine only: reducer, stack, arithmetic, store, tensor, quotations, control flow, closures, history and rollback, plus the state mechanics of actors and channels. Dispatch, class precedence, unification, set algebra, ownership checking, guards, word metaobjects, condition semantics, supervision policy, ready-queue ordering, monitors, registry, and links are Prelude code written in Scale. The inventory's rule that dispatch stays in the kernel and its exclusion of the scheduler are retired; its remaining metaobject stages stay as acceptance tests for the Prelude port.

Repository mapping. Kernel source material is `types.fs` (kernel half), `epoch.fs`, `tensor.fs`, `index-transformation.fs`, `lexicon.fs`, `store.fs`, `scalar-read.fs`, `stack.fs`, `components/prelude/evaluator/stack.fs`, `components/prelude/evaluator/common.fs`, `components/prelude/evaluator/control.fs`, `components/prelude/evaluator/closure.fs`, `components/prelude/evaluator/history.fs`, `components/prelude/evaluator/quotation-run.fs`, the kernel half of `components/prelude/evaluator/environment.fs`, `components/prelude/evaluator/word.fs`, and `components/prelude/evaluator/evaluator.fs`, plus the state mechanics in `scheduler.fs`. Prelude candidates are `components/prelude/evaluator/bookmark.fs`, `components/prelude/evaluator/runtime-guard.fs`, `components/prelude/evaluator/ownership.fs`, `components/prelude/evaluator/set-algebra.fs`, `components/prelude/evaluator/value-sets.fs`, `components/prelude/evaluator/dispatch.fs`, `components/prelude/evaluator/class-system.fs`, `components/prelude/evaluator/condition.fs`, `components/prelude/evaluator/channel.fs`, `components/prelude/evaluator/channel-event.fs`, `components/prelude/evaluator/cell-system.fs`, `components/prelude/evaluator/scheduler.fs`, `ownership-types.fs`, and the policy half of `scheduler.fs`. Edge files are `vocabulary.fs`, `vocabulary-in-image.fs`, `tokenizer.fs`, `parser.fs`, `violations.fs`, `presentation.fs`, `nearest-sentence.fs`, `discourse.fs`, `diagnostics.fs`, `scale-file.fs`, `decision-table*.fs`, `emitter.fs`, `smt.fs`, `refiner.fs`, `type-lattice.fs`, `domain-expression.fs`, and `fixed-specification-acceptance.fs`. Host files are `bases/scale-cli/entry.fs`, `cli.fs`, `vocabulary-list.fs`, `arena.fs`, and the file-I/O half of `persistence.fs`. `development/scale/tests.fs` and `development/scale/granular-tests.fs` are edge tests; a kernel-only suite is added with the kernel project.

Error boundary. The kernel keeps the typed fault families from D94. The ABI encodes every failure as `(code : int, message : string)`; the Prelude supplies the code for its own failures. Flattening `EvaluationError` into a single `PreludeError(code, message)` inside the kernel is rejected: it would discard the types that make handlers exhaustive and make the F# reference weaker than the port.

Prelude loading. The kernel does not contain the tokenizer or parser. `scale_init` receives a build-time compiled prelude image carrying the ABI version, source hash, and entry quotation as bytes, not Scale source text. The edge compiler produces the image and the kernel checks its ABI version at load.

ABI surface. ABI version 1 has eight exports: `scale_abi_version`, `scale_init` (which installs the callback table, so no handle exists without callbacks), `scale_set_callbacks`, `scale_run`, `scale_transport_send`, `scale_host_call`, `scale_image`, and `scale_free`. `scale_image` imports or

exports the image so a host can marshal tensors without a second ABI. The five callbacks are `alloc`, `free`, `host_dispatch`, `log`, and `abort`, each carrying the opaque host context. `HostState` is not a field of `EvaluationState`; the context is passed through the kernel entry points.

Verification under alpha. The F# restructure is verified through SageFs. MSBuild, NativeAOT publish, and host conformance are release-time gates; the ABI is not frozen until the release gate runs the conformance test. Phase 0 can add the kernel project and header now.

Export warning. The F# compiler still reports that `UnmanagedCallersOnly` is unsupported, but .NET 10 NativeAOT exports the attributed methods; upstream issue 14883 records a working native call from C with the warning present. The warning is not a reason to add a second language to the kernel. The release-gate publish and conformance test confirm the export table.

Status: historical, superseded for execution by D99–D105. `components/kernel/scale_abi.h` and `components/kernel/Scale.Kernel.fsproj` define ABI version 1. `scale_init` installs the callback table, validates the image, and retains it; `scale_set_callbacks` replaces that table for a live handle; `scale_run` executes an exported image unit with the kernel VM; `scale_image` imports or exports the retained image. `scale_host_call` forwards a named package/function call to `host_dispatch`; `scale_transport_send` forwards a framed target/payload to `host_dispatch` as the `transport` function. The kernel compiles `types.fs`, `tensor.fs`, `index-transformation.fs`, `lexicon.fs`, `store.fs`, `scalar-read.fs`, `stack.fs`, `kernel-state.fs`, `lowered-ir.fs`, `lowered-vm.fs`, `prelude-image.fs`, `primitive-wire.fs`, `prelude-execution.fs`, and `abi.fs`. The Prelude image path exists, but the shipped Prelude is still the F# reference implementation, not a production Scale image. Current suite status is in `docs/commitments-ledger.typ`.

D96 — Pattern matching starts with the runtime IR and the ground matcher

Decision. Pattern matching is one shared substrate, not a host feature. The runtime pattern IR is Prelude code (`pattern-ast.fs`); the matcher and the surface are the F# reference (`components/prelude/evaluator/pattern.fs`); the open-subject unifier path is landed, while guard execution and the analyses are staged next. A pattern is constructor-headed: `Wildcard`, `Variable(role)`, `NumberLiteral`, `TextLiteral`, `AtomLiteral`, `Constructor`, and `TypedPattern`. Variable roles are exactly `Fresh`, `Pin`, `Repeated`, and `Shadowing`.

Surface. `subject { pattern -> body | pattern -> body } match` selects the first clause whose pattern matches, binds `?x` names in the binding table, and runs the body tokens. `?x` is fresh on its first occurrence in a clause and repeated after; `^?x` is a pin, `!?x` shadows, _ is a wildcard, `(name child...)` is a constructor, `:name` is an atom, and numbers and quoted text are literals. Clause order is preserved.

Failure. `PatternFault` distinguishes `NoClauseMatched`, `MalformedPattern`, `MissingPin`, `UnboundRepeated`, and `PatternUndecided`. A pin is checked against the outer binding table; a repeated variable is unified with its first binding; an open subject is `PatternUndecided`, never a silent structural false.

Open subjects. When the subject is an unbound variable, the matcher encodes the pattern into unification terms, validates pins and repeated roles against the outer table, unifies, and resolves the pattern variables from the resulting substitution. A shadowing role on an open subject is `PatternUndecided`, not a silent re-binding.

Guards. A clause may write `pattern when predicate -> body`. After the structural match, the matcher binds the pattern variables and runs the named predicate over them under a fresh reduction budget. A true verdict selects the clause; a false verdict falls through to the next clause; an error or a

missing verdict is `GuardFailed`. The guard runs in a probe state, so its scheduler, output, and control frames are discarded.

Analyses. `pattern-analysis.fs` computes unreachable clauses by structural subsumption and top-level exhaustiveness. A catch-all clause, or a closed subject kind whose every constructor is covered by a clause with catch-all children, is `Exhaustive`; a missing constructor is `NotExhaustive`; missing metadata or an open kind is `Undecided`. A guarded clause does not cover later clauses.

Templates. `HoleSignature`, `TemplateNode`, and `PatternTemplate` in `pattern-ast.fs` carry the open form. `PatternPortable.templateToJson` writes `form = template` with declared hole signatures and a separate root; `templateOfJson` refuses an undeclared or duplicate hole. The closed reader refuses a template document. `PatternElaboration` fills every declared hole, checks hole kinds and constructor arities, and returns a closed runtime pattern; a template is never executable by itself.

Portable form. `pattern-portable.fs` writes a closed, versioned JSON document: schema 1, `form = closed`, a canonical kind registry with fixed arities, named-predicate versions, and ordered clauses. `ofJson` refuses an unknown schema, a non-closed form, a missing constructor kind, a kind arity mismatch, and a missing or mismatched predicate version. Templates are refused until hole signatures exist. The writer and reader are explicit `Utf8JsonWriter` and `JsonDocument` code, not reflection serialization.

Why this order. The one-prelude/multiple-host goal needs the portable form before a second host can execute a pattern-bearing package. The runtime IR is the intermediate the host adapters will serialize; landing it first fixes the roles and failure classes before the wire format. The ground matcher is the decision-tree engine's seed; the unifier engine reuses it on open terms.

Verification. `Tests.runAll` reports 1138 passed and 0 failed (1103 examples and 35 properties). The new checks cover fresh binding, first-clause selection, a constructor with a repeated variable, `NoClauseMatched`, the portable document round-trip and schema rejection, two open-subject matches, a passing and a failing named-predicate guard, reachability, redundancy, and closed-kind exhaustiveness, a template round-trip plus closed-reader refusal, a general expression document round-trip, a package round-trip, duplicate-unit refusal, the lowered VM's differential and slot checks, its valid/stale cache-guard checks, bare-quotation and closure `Apply` differentials, both lowered `if` branches, and a `with-fields` slot-binding check.

Non-goals. The host adapters remain open. A host-expression guard is not portable; the surface permits only named predicate references. The analysis is best-effort, not a complete decision procedure.

Status: templates close. `pattern-ast.fs`, `pattern-analysis.fs`, and `pattern-portable.fs` are Prelude code; `components/prelude/evaluator/pattern.fs` is the reference matcher and surface. `PatternElaboration` turns a `PatternTemplate` plus declared-hole arguments into a closed runtime pattern; the matcher and guard execution are still the F# reference, not a Scale port.

D97 — One general IR carries the portable abstractions

Decision. There is one portable IR, not one format per feature. Expressions, patterns, declarations, kinds, predicates, sorts, conditions, dispatch, and actor messages are nodes of the same versioned document. Pattern matching is the first typed sub-language of that IR, not a parallel wire format. `docs/general-ir.typ` is the reference.

Contents. The IR carries values and terms; expressions and quotations; the pattern sub-language with its four roles and ordered clauses; package, module, and code-unit declarations; canonical kind

and constructor registrations; named predicates and guards with pinned versions; sort and type annotations; condition, restart, and handler nodes; generic-function, method, class, and protocol declarations; and actor message and supervision schemas. Ownership stays execution state; the IR carries only its declared linear contract annotations.

Boundary. The IR is code; the image is data. Values cross as tensors and text entries. A host adapter exposes six functions—`tag`, `extract`, `evaluate_guard`, `construct`, `raise_failure`, and `query_kind_metadata`—and nothing else. Bit syntax, open maps, active patterns, type providers, host expression guards, host exception types, host process identifiers, and native closures are host-marked or host-only: legal at a host surface, refused in the portable form.

Versioning. A package document carries the IR schema, package identity and source hash, pinned kind/predicate/sort/condition references, ordered code units, and entry points. A loader checks the schema and every pinned reference before execution. Hot reload replaces a unit only when its dependencies and pins still resolve. An old pattern with a new predicate, or a new pattern with an old predicate, is refused.

Locked contract. The portable core is partial: host-marked extensions are legal at a surface and refused in the portable form. Canonical JSON is the v1 wire form. Ground subjects use a decision tree and open subjects use the unifier; the IR fixes observable results, failure classes, clause order, guard timing, and binding commitment, not the engine. Semantic `TypeRefs` are portable; inferred facts are optional unit-level hints and never change semantics. Versioning is separate for schema, kinds, predicates, packages, and units; kind and predicate pins are exact, and registries are package-local with content hashes plus an optional shared registry. Capabilities are default-deny, and host dispatch or code execution is a grant. Source maps use unit-scoped non-semantic node ids in a side table. The prelude is portable-only, one package with units, bootstrapped on the F# adapter before Elixir.

Phase order. Phase 1 is the pattern IR and its portable document (landed). Phase 2 is the general expression and quotation nodes. Phase 3 is the package and registry document. Phase 4 adds conditions, dispatch declarations, and actor message schemas. Phase 5 adds exhaustiveness and redundancy analyses; the best-effort `PatternAnalysis` is landed. Phase 6 adds the host adapters and the prelude bootstrap.

Non-goals. A second portable format for a single feature; embedding host objects in a node; reflection serialization; and treating a host-marked feature as portable.

Status: designed. `docs/general-ir.typ` records the target model. `pattern-ast.fs` and `pattern-portable.fs` are the first landed nodes; `kernel/expression-ast.fs` is the general expression AST and `kernel/expression-portable.fs` is its canonical JSON v1 document; `kernel/package-ast.fs` and `kernel/package-portable.fs` carry the package/unit document; and `components/prelude/evaluator/pattern.fs` is the reference matcher and surface. The condition/protocol/actor-message nodes remain open.

D98 — The portable IR and high-level slices leave the kernel

Decision. The boundary audit found the kernel carrying portable IR, serialization, history, quotation-run, and high-level evaluator slices. Those move to the Prelude assembly: `pattern-ast.fs`, `pattern-portable.fs`, `pattern-analysis.fs`, `expression-ast.fs`, `expression-portable.fs`, `package-ast.fs`, `package-portable.fs`, `components/prelude/evaluator/common.fs`, `components/prelude/evaluator/stack.fs`, `components/prelude/evaluator/history.fs`, and `components/prelude/evaluator/quotation-run.fs`. The kernel keeps the core types, epoch,

tensor, index transformation, lexicon, store, scalar read, stack primitives, the lowered IR/VM, and the ABI. `LoweredVm` no longer reads `Ownership`; cache-guard validity is a callback supplied by the Prelude.

Why. The audit showed those slices were not machine primitives and made the kernel's assembly reference graph look clean while its type graph was not. Moving them removes the IR/serialization/tooling and the hardcoded actor word list from the kernel without changing language behavior.

Status: state split landed. `runtime-types.fs` now carries the Prelude's `Actor`, `Channel`, `SchedulerState`, `OwnershipState`, `Supervision*`, `Monitor`, `Method*`, `EvaluationState`, and the full `EvaluationError`; `epoch.fs` and channel store encoding also moved to the Prelude. The lowered IR/VM are kernel code (D99). The kernel assembly now contains the core types, tensor/index/lexicon/store/scalar/stack primitives, the lowered IR/VM, the image loader/executor, the primitive wire, and the ABI. A boundary fitness check asserts the forbidden types are absent. Current suite status is in `docs/commitments-ledger.typ`. `scale_init` now validates and retains the prelude bytes and also delivers them to the host adapter through `host_dispatch` with package `scale` and function `prelude` during the transition. `scale_run` resolves an exported unit and executes it with the kernel VM (D101).

D99 — The kernel owns the execution core

Decision. Step 1 of the B architecture is landed. The lowered instruction set and VM return to the kernel, with a generic kernel state that carries the store, the operand stack, the pending program, and one opaque host value. Primitive word calls and cache-guard validity are callbacks. `kernel-state.fs`, `lowered-ir.fs`, and `lowered-vm.fs` are kernel compile items. `EvaluatorKernelBridge` adapts the F# Prelude by carrying its full evaluation state as the host value, so a primitive call may change Prelude-only tables, frames, or ownership and the kernel carries those changes forward without knowing their types.

Why. The kernel boundary was clean but the kernel was not a machine: the execution loop lived in the Prelude and the ABI only forwarded. A generic VM with an opaque host value keeps the boundary clean while making the kernel the interpreter that Step 2 can load an image into. The host value is a transport seam, not a Prelude type in the kernel.

Status: Step 1 landed. A kernel-only test runs the tokens 2, 3, and add with a unit host value and a primitive callback, proving the VM needs no Prelude state. The full F# path runs through `EvaluatorKernelBridge`. The suite reports 1251 checks (1211 examples and 40 properties). The image loader and validation landed in D100; the image-to-VM entry path landed in D101, so `scale_run` executes image-resident units directly.

D100 — The kernel loads and validates the Prelude image

Decision. The kernel owns the Prelude image format and loader. An image is the compiled, ordered set of hot-reload units: each unit carries its lowered instructions, and the document carries the pinned kind and predicate registries, capabilities, imports, exports, and source hashes. `prelude-image.fs` is a kernel compile item. `scale_init` validates the image before the handle exists, retains it, and clears the handle on refusal.

Format. The JSON schema is version 1 with form `image`. The loader checks the schema, form, non-empty name/version/source-hash, unit uniqueness, dependency existence and acyclicity, export uniqueness and unit existence, import and capability uniqueness, kind arity consistency, predicate version consistency, instruction tags, slot non-negativity, guard shape, label uniqueness, and jump

resolution. It caps the image at 16 MiB, the unit count at 4096, and each unit at one million instructions. `JsonDocument` reads and `Utf8JsonWriter` writes, so the path is NativeAOT-safe without reflection serialization. An instruction is one of number, text, push-slot, store-slot, call (with an optional target/epoch guard), apply, jump, jump-if-false, label, or fail.

Why. The image is the unit of portability and hot reload. Validating it in the kernel is what makes the kernel the authority over what can run, and it keeps the host adapter from becoming a second, divergent loader.

Status: Step 2 landed. Valid images load and resolve exports; duplicate units, unresolved labels, unknown instructions, and dependency cycles are refused with named errors; `scale_init` refuses an invalid image and clears the handle. The loader is covered by instruction and image round-trip properties, arbitrary-byte totality, jump and export invariants, named-error examples, ABI lifecycle tests, and kernel VM failure tests. Current suite status is in `docs/commitments-ledger.typ`. Execution wiring is Step 3, recorded in D101.

D101 — The kernel executes an exported image unit

Decision. Step 3 is landed. `scale_run` treats its program bytes as an exported unit name, resolves it with `PreludeImage.resolveExport`, and runs the unit with the kernel VM in `prelude-execution.fs`. A `CallWord` that names another image unit executes that unit recursively with the same machine state; a word with no image unit is dispatched to the host as package `scale` and function `<word>` with no operands. A non-zero primitive status fails the run. The result is the textual top-of-stack scalar, allocated through the host `alloc` callback. `scale_run` no longer forwards the whole program to `host_dispatch`.

Why. Entry resolution, recursive image units, and named primitive effects make the kernel the executor of the Prelude image. The remaining primitive surface is explicit: an unknown word names a host primitive, and a failed primitive is an error rather than a silent continuation.

Status: Step 3 landed. An executable image whose entry calls an image-resident `double` unit returns `42`; an effect primitive `print` is dispatched once and execution continues to return `9`; a failing primitive returns `SCALE_ERROR`; an invalid handle returns `SCALE_INVALID`; the non-primitive run performs no host dispatch; and swapping the image changes the entry result from `42` to `7`. Current suite status is in `docs/commitments-ledger.typ`. Primitive operand marshalling remains open.

D102 — Boundary proof for the kernel machine

Decision. Step 4 is landed as executable proof at the boundary, not as a new runtime feature. The kernel-only VM runs a branch, slots, guards, and a named failure with a unit host value and no Prelude state. The image loader refuses inconsistent kind arity and inconsistent predicate version. `scale_run` writes a named error reason into the result buffer for both a kernel VM failure (`vm: <reason>`) and a primitive failure (`primitive <word> failed with status <n>`), so a host can distinguish the two. The image-swap test changes the entry result from `42` to `7`, and the assembly checks prove the kernel references no Prelude assembly and contains no Prelude types.

Why. The boundary is only real if the kernel can be exercised without the Prelude and if failures keep their origin. These tests make both properties falsifiable: a Prelude leak, a swallowed VM failure, or a host primitive failure misreported as a kernel failure now fails the suite.

Status: Step 4 landed. Current suite status is in `docs/commitments-ledger.typ`. Primitive operand marshalling landed in D103.

D103 — Primitive operands and results cross the ABI as tensors

Decision. Step 5 is landed. An image carries a `primitives` registry: each entry names a primitive, its operand arity, and its result count. The loader rejects duplicate primitive names, a primitive that shares a unit name, and negative arity or result counts. When the VM reaches a word with no image unit, it requires a registry entry, pops its operands, calls the host, checks the result count, and pushes the results.

Wire. The kernel sends `{"word":...,"operands":[{"shape":[...],"data":[...]},...]}` with operands in source order and receives `{"results":[{"shape":[...],"data":[...]},...]}`. Both sides are written with `Utf8JsonWriter` and read with `JsonDocument`, so the path is NativeAOT-safe. Non-finite operands are refused before the call; malformed shapes, non-numeric data, missing results, and a result-count mismatch are named protocol failures. The host owns the response buffer and the kernel frees it through the `free` callback.

Why. Primitive effects alone cannot run the Prelude. Tensor operands and results make arithmetic, I/O adapters, and host services possible without letting the kernel know what a primitive means.

Status: Step 5 landed. An image declaring `add` with arity 2 and one result executes `1 2 add`, receives both operands in source order, and returns `3`. Effect primitives still work with arity and result count zero. The suite reports 1251 checks (1211 examples and 40 properties).

D104 — Pattern templates elaborate to closed patterns

Decision. `PatternElaboration.elaborate` closes the open pattern form. It takes a `PatternTemplate` and a map of declared-hole arguments, rejects a duplicate or missing hole, an undeclared argument, a hole whose argument does not match the declared kind, an unknown constructor kind, and a constructor arity mismatch, then returns a closed `Pattern` that the runtime matcher can use. The result is validated against the template's canonical kinds a second time, so an argument cannot introduce an unregistered constructor.

Why. A template is a portable interface, not an executable clause. Without elaboration, the open form could only round-trip; with it, a host can fill the declared holes and produce the same runtime pattern the closed reader would have accepted.

Status: landed. Template elaboration succeeds for a nested hole and refuses missing, unknown, duplicate, kind-mismatched, unknown-kind, and arity-mismatched cases. Current suite status is in `docs/commitments-ledger.typ`. The matcher and guard execution remain the F# reference; a Scale port is still open.

D105 — The ABI imports and exports the retained image

Decision. Step 6 is landed. `scale_image` direction 0 exports the retained image as canonical JSON through the host `alloc` callback. Direction 1 validates replacement bytes with the same loader and replaces the retained image only on success. Export and import require a live handle and the installed callback table; import leaves the old image unchanged on a load error.

Why. Image transfer is how a host saves, inspects, or hot-swaps a Prelude without rebuilding the kernel. It reuses the one authoritative loader, so an imported image cannot bypass the schema, unit, pin, or primitive checks.

Status: landed. An exported executable image reloads as the same image, an imported image changes the entry result from 42 to 7, and the tests cover both directions. Current suite status is in `docs/commitments-ledger.typ`.

D106 — Callbacks can be replaced on a live handle

Decision. `scale_set_callbacks` replaces the callback table of a live handle. `scale_init` still installs the first table because no handle exists without callbacks; the new export updates that table without changing the handle's opaque value. A null callback table or an unknown handle is `SCALE_INVALID`.

Why. A host may need to move its allocation or dispatch context after initialization. Replacing the table explicitly keeps the bootstrap invariant and makes the lifetime rule visible: the new table and its context must outlive the handle.

Status: landed. Replacing the callbacks on a live handle succeeds; replacing them on a null handle is refused. Current suite status is in `docs/commitments-ledger.typ`.

D107 — DDD is adopted with Typst and current ABI names

Decision. The Documentation-Driven Development principles are adopted with the codebase's current artifacts. Typst is canonical: `docs/README.typ`, `docs/INVARIANTS.typ`, `docs/kernel/abi.typ`, `docs/prelude/README.typ`, and `docs/adapters/README.typ` are the adopted layout. The ABI header is `components/kernel/scale_abi.h`; the kernel exports F# directly with `UnmanagedCallersOnly`, not through a C# shim.

Versioning and conformance. The installed single version is `SCALE_ABI_VERSION` (currently 2; D162), and the image schema is version 1. `SCALE_EXPECTED_ABI_VERSION`, `scale_version()`, and adapter equality are the release-time forms and land with the first host adapter. The C conformance harness does not exist yet; until the first adapter, the SageFs suite is the acceptance gate. The F# Prelude remains the shipped reference; `docs/prelude/` is reserved for feature docs written before a Scale port.

Why. The DDD principles are right for this architecture: the ABI header is the contract, the F# Prelude is the port's oracle, and the invariants are what keep the split honest. The literal Markdown/C#/conformance details were stale relative to the codebase, so they are adapted rather than copied.

Status: adopted. `docs/INVARIANTS.typ` records the nine invariants with their actual enforcement status: four are enforced by boundary/image tests, two are partial, and three are not yet tested (feature independence, upcalls, and version equality). Current suite status is in `docs/commitments-ledger.typ`.

D108 — The ABI forwards named host calls

Decision. `scale_host_call` is implemented. It validates the live handle, reads the package and function names from the caller's borrowed buffers, and forwards the call through `host_dispatch` with the supplied arguments and result out-parameters. A null package/function pointer with a non-zero length, or a dead handle, is `SCALE_INVALID`.

Why. Adapters need a direct, named host-service call that is not a Scale word. Keeping it on the same callback table as primitives means one transport and one lifetime rule.

Status: landed. The call forwards package, function, and arguments unchanged. Current suite status is in `docs/commitments-ledger.typ`.

D109 — The ABI forwards transport payloads

Decision. `scale_transport_send` is implemented. It validates the live handle and borrowed buffers, frames the target and payload as UTF-8 JSON (`{"target":...,"payload":"<base64>"}`), and forwards it through `host_dispatch` as package `scale`, function `transport`. The host owns transport semantics; the kernel owns framing and the handle check.

Why. Transport was the last unimplemented export. A framed primitive keeps the kernel out of host routing while giving every adapter one payload shape.

Status: landed. The tests cover the live handle, the `scale` package, the `transport` function, and the target/payload framing. The suite reports 1251 checks (1211 examples and 40 properties).

D110 — Image units may recurse within a call-depth bound

Decision. `PreludeExecution` no longer rejects a unit because it is already on the call path. A unit call is bounded by `MaxCallDepth = 256`; each nested unit runs with a fresh local-slot map and shares the operand stack, so ordinary and mutual recursion work up to the bound. Exceeding the bound is the named `call depth exceeded: 256` failure.

Why. The previous active-set guard prevented the recursive Prelude functions from ever running in the image. A depth bound keeps runaway recursion from exhausting the host stack while allowing finite recursion.

Status: landed. A recursive image reaches the bound and reports the named failure; ordinary nested image calls still return their value. Closures and dynamic slots remain open. Current suite status is in `docs/commitments-ledger.typ`.

D111 — Core stack and arithmetic opcodes live in the VM

Decision. The kernel instruction set now has `Duplicate`, `Swap`, `Drop`, `Over`, `Rotate`, `Add`, `Subtract`, `Multiply`, `Divide`, `LessThan`, `GreaterThan`, and `Equals`. The VM implements them over tensors, and the image loader/writer round-trips every opcode. Arithmetic is elementwise for equal shapes, scalar for two rank-0 values, and a named failure otherwise; division by zero is a named failure.

Why. A Prelude image cannot call a host primitive for every arithmetic or stack word. These are the kernel's core machine operations and are the first non-control instructions the Scale compiler can target.

Status: landed. The kernel-only VM adds `2 3` to `5` and swaps/drops without a primitive; the image codec accepts the new opcodes. Closures, pattern-matching instructions, store/table primitives, and the Scale-to-image compiler remain open. Current suite status is in `docs/commitments-ledger.typ`.

D112 — Association-vector primitives live in the kernel store

Decision. `Store.findPair` and `Store.addToEnvironment` are kernel store primitives. `EvaluatorCommon.findPair`/`addToEnvironment` delegate to them, so the Prelude and the kernel share one latest-write-wins association implementation. A root is a vector of `[name; value; ...]` identities; an update appends a new pair and returns the new root.

Why. Vocabulary, bindings, user words, and roots all use the same association vector. A Prelude port cannot manage those roots if the primitive exists only in the F# evaluator.

Status: landed. The kernel store finds a pair, appends a pair, and keeps the latest value for a repeated name. Current suite status is in `docs/commitments-ledger.typ`. Image instructions for these primitives are landed in D113.

D113 — Image instructions expose association roots

Decision. `KernelState` carries named association-vector roots. `PushRoot`/`SetRoot` read and write a named root, `AddPair` appends a name/value pair and returns the new root, and `LookupPair` returns the value or the root-none marker `-1`. The image loader and writer round-trip all four instructions.

Why. Vocabulary, bindings, and user words need store-resident association vectors. Exposing the root operations to image units is the minimum needed for a Scale-defined registry; host primitives cannot manipulate the store.

Status: landed. A kernel-only image adds `key -> 42`, looks it up, and returns `42`; a missing key returns `-1`. Current suite status is in `docs/commitments-ledger.typ`. Closures, pattern instructions, dynamic word dispatch, and the compiler remain open.

D114 — Duplicated granular cases leave; portable JSON is shared

Decision. The granular suite keeps only artifact tests: 10 snapshots and 7 integrations (17 cases). The example and property cases remain in `Tests.runAll`; they are no longer re-run by the granular runner. The shared JSON helpers for the pattern, expression, and package documents move to `portable-json.fs`; the three codecs keep their own error constructors.

Why. The granular suite was 78 cases, but 61 of them re-ran checks the aggregate gate already covered. The shared JSON helpers had three copies of the same parser and writer plumbing. Both were redundant maintenance and diluted the signal from the unique artifact tests.

Status: landed. The granular suite is 17 cases; the full suite remains 1251 checks (1211 examples and 40 properties). Current line counts and suite status are in `docs/commitments-ledger.typ`.

D115 — Dynamic word definitions run as lowered images

Decision. `KernelState` carries a dynamic word map. `DefineWord` pops a name and a quotation body, lowers the body tokens to instructions, and records it; `CallWord` gives a dynamic word precedence over an image unit and a primitive. `LoweredInstructions.lowerTokens` maps the core arithmetic and stack words to their opcodes, so a defined body can use them without a host primitive.

Why. A language needs user-defined words. Text quotation plus lowering is the smallest useful definition mechanism, and it reuses the instruction set the image already executes.

Status: landed. A kernel-only image defines `double` as `2 3 add`, calls it, and returns `5`. Closures, pattern instructions, actor/channel primitives, and the Scale-to-image compiler remain open. Current suite status is in `docs/commitments-ledger.typ`.

D116 — Initial port scope and non-porting tracks

Decision. The initial Scale Prelude port covers the executable language core: evaluator and dispatch, pattern/expression/package IR, actor scheduler and channels, ownership, types, store, and the kernel bridge. The following are edge, verification, or product tracks, not part of the initial port:

- decision-table and emitter surface;

- SMT, type-lattice refinement, and repair search;
- parser and Basic English edge;
- persistence/compaction beyond the initial store;
- demos and acceptance harnesses.

They remain F# edge features with their existing tests. `line-counts` reports Prelude core, edge/verification, demo/acceptance, test harness, and tools separately. They are not deleted and not claimed as ported.

Why. A porting-oracle count that mixes runtime semantics with demos, SMT, parser surface, and acceptance harnesses overstates the work and hides the actual Prelude core. The separation is a scope decision, not a code deletion.

Status: adopted. The line-count report separates the categories; live counts are in `docs/commitments-ledger.typ`. Kernel and Prelude tests remain the acceptance gate.

D117 — Closures capture lexical slots

Decision. `MakeClosure` captures named slots into a store tensor tagged `ClosureTag`, `CallClosure` pushes the captured values as a closure frame and executes the named unit, and `PushCapture` reads the current frame. Nested calls keep a stack of frames; the frame is popped after the unit returns. The image loader and writer round-trip all three instructions.

Why. A dynamic word is nullary and captures nothing. The Prelude needs lexical capture for lambdas and local functions, and a capture frame is the smallest form that does not reintroduce host state into the kernel.

Status: landed. A kernel-only image captures `10` in slot 0, closes it into `add-ten`, passes `5`, and returns `15`. Current suite status is in `docs/commitments-ledger.typ`.

D118 — Tensor and text inspection primitives

Decision. The VM gains `TensorRank`, `TensorSize`, `TensorItem`, `TextRead`, and `TextEquals`. They inspect a store identity without exposing the host representation: rank and size push scalars, item pops an index and an identity and pushes the scalar element, text read pushes the interned text, and text equality pushes `1` or `0`. The image loader and writer round-trip all five.

Why. Pattern matching, constructor inspection, and text predicates need to inspect stored terms. Without these primitives a prelude image cannot read a tensor's shape or a text value; it can only push identities.

Status: landed. `tensor-size`, `tensor-item`, and `text-equals` read stored text without a primitive. `tensor-rank` and `text-read` share the same image codec. Current suite status is in `docs/commitments-ledger.typ`.

D119 — Expression lowering is total over the closed node universe

Decision. `EvaluatorLowering.lowerExpression` returns `Result<Instruction list, LoweringRefusal>`. Every constructor of the closed `Expression` union is matched explicitly. A literal, a word, a quotation, a block, an `if`, and a `call` whose target is a word lower; a call whose target is any other node is refused as `CallTargetNotWord`; every other node is refused as `NodeNotLowered node`, where `node` is a case of `LoweredNode`. A refusal names the node and says to use the interpreter. The lowerer no longer emits a `Fail` instruction as its fallback, so an unsupported

construct is a lowering-time decision, not a run-time failure. `EvaluatorLowering.nodeOf` names every node, and a coverage test exercises one representative of each of the fifteen constructors.

Why. D84′s phase-2 tiering keeps the interpreter as the fallback for code the compiler declines. A `Fail` instruction hidden in the lowered stream delays that decision to the wrong layer: the emitter cannot tell an unsupported node from a program that fails, and the failure loses the node's identity. A closed, exhaustively matched lowerer makes "the IR is total" a compile-time property: adding a constructor to `Expression` breaks the build until the lowerer and its node names are updated. The named refusal is a matchable value, not a string, so a caller can fall back by category.

Status: landed. The coverage test classifies all fifteen constructors, and a lowered `call` over a word target agrees with the token interpreter. Current suite status is in `docs/commitments-ledger.typ`.

D120 — Compiled artifacts exclude history, bookmarks, and rollback

Decision. The lowered instruction set carries no step-record instruction and no resumable program counter. `LoweredInstructions.historyScope` is `HistoryExcluded`, and an emitter writes that scope into the artifact metadata. Compiled code does not record the event stream, create bookmarks, or roll back; the interpreter remains the only history-recording engine, and the phase-2 tiering sends any program that uses those words to it. History, bookmarks, and rollback stay Prelude-interpreter features, not IR obligations.

Why. D29 made every executed token append a step recording the token, the stack before it, the program to resume, and the program-counter metadata (`Location`, `Baseline`, `Position`); rollback restores the stack, program, and counter and clears the lexical frames the step did not capture (D45). The lowered IR has no token program to resume and no program counter to resume at. Carrying a resumable counter would put interpreter execution state into the compiled artifact, which is the state the kernel boundary keeps out. Excluding it is the smaller, honest scope and keeps the interpreter load-bearing: a program that asks for history gets the engine that can provide it. The decision is pinned by a test: the closed instruction union contains no `RecordStep`, and adding one fails that test until this record is revisited.

Status: landed. `LoweredInstructions.historyScope` is `HistoryExcluded`; the coverage test reflects over the instruction union and rejects a step record. Current suite status is in `docs/commitments-ledger.typ`.

D121 — Lowered let reuses the interpreter's let word; lambda stays declined

Decision. `Let(name, value, body)` lowers to the interpreter's own `let` word: lower the value, push the name as a text literal, call `let`, then lower the body. The binding lives in the same frame the interpreter uses. A bare `Word` inside the body stays late-bound and does not resolve to the binding; the body reads it through `lookup`, exactly as before. No compile-time name environment, slot allocation, or shadowing is introduced. `Lambda` stays declined as `NodeNotLowered LambdaNode`.

Why. The lowered VM and the token interpreter must agree, including on failures. `let` has exactly one interpreter meaning, so reusing the word keeps the two engines identical and adds no new language rule. Lexical slot resolution would be such a rule: the interpreter binds a name in a frame read only by `lookup`, so a lowerer that resolved a bare word to a slot would make the two engines disagree. `Expression.Lambda` cannot be mirrored the same way: the interpreter's `lambda` is a zero-parameter closure over a text quotation, while the expression node carries named parameters. Lowering the parameters would be a new binding rule, and rendering the body back to text would

lose text literals that contain spaces because the token split is on spaces. Lambda therefore waits for a parameter-binding decision rather than borrowing a meaning it does not have.

Status: landed. A lowered let that reads the binding through `lookup` agrees with the interpreter, and a lowered let whose body uses a bare word fails with the interpreter's diagnostic. The declined-node coverage list no longer includes let. Current suite status is in `docs/commitments-ledger.typ`.

Superseded (D122). The bare-word expectation above is reversed: a bare word now resolves to the nearest lexical binding in both engines. The `lookup` behavior is unchanged, and the lambda rationale still holds.

D122 — Lexical bindings resolve through the frame chain and lowered slots

Decision. A lexical binding is visible to a bare token, not only to `lookup`. `let` still appends a binding to the current frame. The evaluator checks the current frame chain for a bare token before the named-word table, so the nearest frame wins and a lexical binding may shadow a built-in word. Literals and structural tokens stay syntax and are never shadowed: `{`, `}`, `[`, `]`, numbers, text literals, `?name`, `:name`, and the internal return tokens. Lowered `let` writes both representations: it calls the interpreter's `let` so `lookup` keeps working, then stores the same value in a fresh slot, and a lexically bound `Word` lowers to `PushSlot`. The lowerer carries a compile-time name-to-slot environment, so shadowing is lexical and sibling scopes get distinct slots. A `let` may not bind a reserved token, and a `Call` whose target is a lexically bound name is declined because the token interpreter would push that value rather than call it; these are the named refusals `ReservedBindingName` and `CallTargetBound`. Lambda parameters remain declined in this step; the staged target is to pop arguments into parameter slots at closure entry once the lowering output can carry units.

Why. The two engines must agree on every program. Slots give compiled code direct access and map to CLR locals; frames keep `lookup`, persistence, and dynamic names. Writing both on `let` is the hybrid: a bare name is a slot read, and `lookup` is the explicit frame read of the same binding. Resolving bare names only in the lowerer would split the language; updating the evaluator first keeps one rule. The frame check precedes the named-word table so shadowing is uniform, but syntax and literals are excluded because they are not names. Reserving syntax tokens in `let` keeps the two representations from disagreeing on pathological names. Declining a call to a bound name avoids inventing a first-class call the interpreter does not have.

Reversal. D121 recorded that a bare word inside a `let` body stays late-bound and fails. D122 reverses that: the bare word now resolves to the nearest lexical binding in both engines, and D121′s second test is replaced by a resolution test that also covers shadowing. D121′s first test (`lookup` still reads the binding) remains true. D121′s lambda rationale is unchanged.

Status: landed. A lowered let with a bare word agrees with the interpreter, nested lets shadow lexically, an outer binding is still visible, and the two new refusals name their cause. Current suite status is in `docs/commitments-ledger.typ`.

D123 — Lambda parameters bind to slots in a named unit

Decision. `Lambda(parameters, body)` lowers to a named unit plus a `MakeClosure` instruction. The unit prologue copies its frame into local slots (`PushCapture k; StoreSlot k` for each frame entry), the parameters occupy slots `0 .. n-1`, and the conservatively captured enclosing names occupy the slots after them. The body lowers against that slot environment. `MakeClosure` gains an arity, the kernel closure tensor records it, and `CallClosure` pops exactly that many arguments, refuses an underflow by name, reverses them into parameter order, prepends them to the captured values, and

executes the unit. A `Call` whose target is a lambda lowers the arguments, the closure, then `CallClosure`. `lowerExpression` returns a `LoweredProgram` carrying the unit table and the entry instructions, and `LoweredVm.runProgram` resolves unit names inside the VM. D123 captured every enclosing lexical name conservatively; D133 supersedes that with a free-variable analysis that captures only the names the body uses. Duplicate parameter names and reserved parameter names are named refusals.

Why. A unit body runs in a fresh slot map, so it cannot read the caller's slots directly; copying the frame into slots gives every unit one uniform access form and lets a nested lambda capture those slots in turn. Putting the arity in the kernel closure keeps the call contract with the instruction that performs the call, so an underflow is refused before the body runs and the emitter never has to emit a runtime check. Conservative capture is the parse-don't-validate choice for this stage: it cannot miss a free variable, and a later free-variable analysis can shrink the captures without changing the observable semantics. The unit table is the first real code object the lowering produces, which is the shape the phase-2 emitter needs.

Status: landed. A one-parameter lambda returns its argument, a two-parameter lambda subtracts the second from the first (so the check pins argument order), an underflowed call is refused by name, a lambda captures an enclosing `let` binding, and a duplicate parameter name is refused. Current suite status is in `docs/commitments-ledger.typ`.

D124 — A differential corpus guards the lowered VM against the interpreter

Decision. A shared corpus of eighteen token programs lives in `differential.fs`, classified with the test harness. Each case runs through the token interpreter and through `LoweredInstructions.lowerTokens` plus the kernel VM. A success case asserts the same scalar result in both engines; a failure case asserts both fail with the same `FailureKind` (division by zero, underflow, unknown word), not the same prose. The harness also aligns the token-lowering word names with the interpreter: `lessThan` and `greaterThan`, not the hyphen spellings the interpreter never had. The corpus covers literals, arithmetic, comparisons, the stack words, and the three failure families.

Why. Every lowering change can silently diverge from the reference. Scattered ad-hoc comparisons do not scale, and the corpus found two real divergences. `lowerTokens` accepted `less-than` while the interpreter accepts `lessThan`, so one program could succeed in the VM and fail in the interpreter. The VM's `Rotate` also rotated the opposite way from the interpreter: for `1 2 3 rotate` the interpreter puts `1` on top and the VM put `2` on top. The VM now matches the reference. The corpus is the P1 acceptance slice ("the IR VM and the token interpreter agree on the example suite, including failures and diagnostics"). A success value plus a failure keyword is the comparable part of the two engines' outcomes, because their diagnostic prose is intentionally different.

Status: landed. The eighteen-case corpus agrees on every case, and the corpus length is pinned so an empty or truncated list cannot pass. It found and fixed two divergences: the `lessThan`/`greaterThan` spellings and the VM's opposite `rotate` direction. Current suite status is in `docs/commitments-ledger.typ`.

D125 — Pattern matching compiles to inspection, branches, and slots

Decision. `PatternMatch(subject, clauses)` lowers to a subject slot and one test per clause. The subject lowers once into a slot. Each clause copies the subject into a scratch slot, compiles the pattern as a sequence of inspection instructions with a jump to the next clause on mismatch, binds Fresh variables to fresh slots, then runs the body with those names in the lexical environment. A

pattern variable named `?x` binds slot `s`, so a body `Word "?x"` lowers to `PushSlot s`. After the last clause the program runs `Fail "pattern-match: no clause matched"`.

The compiler reads the ground term encoding the interpreter already uses: a compound is `[-2; head; arguments...]`, a typed term is `[-3; sort; term]`, and an atom is any text entry. Two generic kernel primitives are added: `IsText` pops an identity and pushes `1` or `0`, and `TensorEntry` pops an index and an identity and pushes the entry identity stored at that index. They are inspection, not pattern logic.

Supported patterns are `Wildcard`, a `Fresh` variable, `NumberLiteral`, `TextLiteral`, `AtomLiteral`, `Constructor` (the compound form and the zero-arity atom form), and `TypedPattern`. Refused patterns are `Pin`, `Repeated`, and `Shadowing` (one named refusal each), a guard (`PatternGuardNotLowered`), and a duplicate Fresh name inside one clause (`DuplicatePatternVariable`). The lowered path matches ground subjects; an open unification subject remains the interpreter's job, which is stated rather than silently producing a false.

Why. The encoding is already inspectable with `TensorRank`, `TensorSize`, `TensorItem`, `TextRead`, and `TextEquals`; only entry extraction and a total text test were missing. Compiling the test keeps pattern semantics in the Prelude, leaves the kernel with generic inspection, and gives the emitter branches it can turn into methods. Slots are the binding store because the body is lowered code; the interpreter's binding table stays for pins and guards, which is exactly why those are refused until their image-resident form is designed. The scratch-slot protocol means a failed clause never leaves the operand stack dirty, so no backtracking cleanup is needed.

Status: landed. The eleven checks pass: number, text, wildcard, fresh variable, constructor with a child, zero-arity atom constructor, typed pattern, no-clause failure, and the three refusals (guard, pin, duplicate variable). The suite moves by a net ten because the old pattern-match refusal check retires. Current suite status is in `docs/commitments-ledger.typ`.

D126 — The IL writer is a JIT-only backend behind a conditional compile

Decision. `writer/il-writer.fs` is compiled only when `EnableWriter=true`; the project defines `ENABLE_WRITER` and includes the file. The release publish passes `EnableWriter=false`, so neither the writer nor its test reaches the AOT binary. The native entry has no writer path yet; when it gains one it returns a named refusal (D84).

The first slice emits a static `Run(): double` for the arithmetic, slot, stack, and branch subset: `LiteralNumber`, `Add`, `Subtract`, `Multiply`, `Divide`, `LessThan`, `GreaterThan`, `Equals`, `StoreSlot`, `PushSlot`, `Duplicate`, `Drop`, `Swap`, `Over`, `Rotate`, `Jump`, `JumpIfFalse`, `Label`, and `Fail`. Any other opcode returns `WriterRefusal.UnsupportedOpcode` so the caller falls back to the interpreter. The writer matches only `Instruction`; it never matches `Expression`.

The writer saves a `PersistedAssemblyBuilder` assembly to a stream, loads it, and invokes the method. That path is JIT-only and lives only in the granular suite, which the AOT binary does not run. `writer/` is classified with tools in the line-count report: it is a build-time backend, not Prelude core.

Why. D84 requires the writer to be unreachable from the AOT entry; a conditional compile is the mechanism the port document names. The first slice proves the SRM plumbing and the opcode match without pretending the runtime-library calls for store and dispatch exist yet. The named refusal keeps the interpreter load-bearing and makes the compiled subset explicit. Testing through

save, load, and invoke is the only proof that the emitted metadata is the method that actually runs, which is the point of the SRM writer.

Status: landed. The four granular checks pass: arithmetic, a branch, a stored slot, and an unsupported-opcode refusal. The example suite is unchanged at 1288; the granular suite is 21. The write, save, load, and invoke path is the proof that the emitted metadata is the method that runs. Current suite status is in `docs/commitments-ledger.typ`.

D127 — Port review after D126

Decision. Record the review of the nine-commit slice (D119–D126) and its follow-ups. The one code change in this record is the `EnableWriter` default: it is true for a Debug build and false otherwise, so a plain Release `dotnet publish` cannot accidentally include the JIT-only writer. A Release build that needs the writer tests must pass `-p:EnableWriter=true`. The other items are recorded, not changed.

What landed. D119 total lowering; D120 history excluded; D121 `let` through the interpreter; D122 lexical bindings; D123 lambda units; D124 differential corpus; D125 pattern matching; D126 IL writer. Verified at `71d6ed1`: 1288 example checks, 21 granular, docs compile clean. All commits are local and unpushed.

Process deviations. (1) Early `grep`/`sed`/`head` navigation was attempted and denied; the MCP path is mandatory. (2) `hard_reset_fsi_session` with `rebuild=true` invokes MSBuild during alpha. It was used as the SageFs reload path; the alpha rule's intent is no shell MSBuild, and the owner should confirm or replace this. (3) Typst phrase edits used `edit` plus `doc_compile` where the section tools cannot target a sentence; allowed with a report, and not every instance was reported. (4) `AGENTS.md` carries a concurrent agent's edit and was deliberately never committed.

Open items. (1) The writer needs a harness ABI for store, stack, and dispatch calls, and units emitted as methods. (2) Pattern matching still excludes open subjects, pin, repeated, shadowing, and guards. (3) Lambda capture is conservative, not a free-variable analysis. (4) The remaining expression nodes are `DeclareKind`, `DeclarePredicate`, `Condition`, `Restart`, `Generic`, and `Message`. (5) The Scale Prelude port has not started; `docs/prelude/` is reserved. (6) The differential corpus covers the token subset, not every `lowerExpression` path.

Why. A port slice is not done when the tests pass. The deviations must be visible to the next reviewer, and the `EnableWriter` default is a release hazard that is cheap to remove now. Recording the open items separately keeps the ledger honest about what is ported and what is still the interpreter's job.

Status: landed. `EnableWriter` is Debug-only by default; the publish and release recipes continue to pass `EnableWriter=false`. The example and granular counts are unchanged. The open items are in `docs/technical-debt-register.typ`.

D128 — The writer targets a runtime harness over the real machine state

Decision. The emitted method is `static KernelState<EvaluationState> Run(KernelState<EvaluationState>)`. It operates on the real machine state — store, stack, roots, words, capture frames, and host — not on raw doubles. The host type is closed at emit time; the Prelude host is the first target, and a different host is a different emit.

A new AOT-safe `writer-runtime.fs` defines the static `WriterRuntime` class. Value instructions delegate to `LoweredVm.step`, a new public single-instruction entry point, so the interpreter and the emitted code share one semantics. `Push`, `Pop`, `PopNumber`, and `CallWord` operate on the same state

and the same dispatch bridge; `Fail` raises `VmFault`. Slots are IL locals of `EntryIdentity`: `StoreSlot` pops a pair (identity plus new state), `PushSlot` pushes the local, and `JumpIfFalse` uses a pair (scalar plus new state). A pair is used instead of a struct because F#'s struct `val` fields are non-public and `@`-suffixed, so emitted IL cannot read them; the pair's public `Item1`/`Item2` getters are callable.

The writer still matches only `Instruction`; closure, tensor, and pattern instructions refuse in this slice. `LiteralText` is supported through the harness. The D126 raw-double tests are replaced by harness tests.

The runtime is compiled always and is AOT-safe; only the writer is JIT-only.

Why. Phase 2 requires the emitted code to call the runtime library for store and dispatch. A method over the real state is the smallest shape that does that, and it keeps the compiled path on the same value store as the interpreter. Sharing `LoweredVm.step` makes divergence impossible on the supported instructions; a second raw-double semantics would drift, which is the failure D124 was built to catch. IL locals for slots are the compiler's locals, and they are the shape a CLR method body needs; putting slots back into a host map would give up the reason to compile.

Status: landed. The six harness checks pass: arithmetic, a branch, a stored slot, a `CallWord` through the harness, a `Fail`, and an unsupported-opcode refusal. The four D126 checks are replaced by six, so the granular suite moves from 21 to 23; the example suite stays at 1288. The emitted method threads the real state and shares `LoweredVm.step` with the interpreter. Current suite status is in `docs/commitments-ledger.typ`.

D129 — The language specification is canonical

Decision. `docs/specification.typ` is the normative language document. It covers the whole language, core and extensions, and marks every feature as implemented, partial, planned, or refused, with the decision and the test as evidence. The topic documents remain the design rationale and the deep dives. The specification is linked from `docs/README.typ` and from the file-layout section of `AGENTS.md`. Where the specification and the code disagree, one of them is fixed in the same change.

Why. The language grew across 128 decisions and 28 topic documents, and a new reader had no single normative entry point; a feature's status was spread across the ledger, the roadmap, and the decision records. The specification gives one contract and one status matrix, which is the honest meaning of "complete" in alpha: every part of the language is specified, and the implementation status is visible rather than implied.

Status: landed. The specification compiles and is linked from the index. It is a living document; a change that alters language behavior MUST update the affected chapter and the status matrix in the same change.

D130 — Alpha decisions: reload, writer units, interpreter-only nodes, C adapter

Decision. Four open questions are settled:

- **Reload.** SageFs `hard_reset_fsi_session` with `rebuild=true` is the sanctioned alpha reload path. It is the only permitted MSBuild invocation, and only for reloading a persisted file or an `.fsproj` change; shell `dotnet build`/`run`/`test`, `just build`, `just check`, and publishes remain banned.
- **Writer next.** The next writer slice emits one method per `LoweredUnit` and adds closure, tensor, and pattern opcodes to the harness. The entry method calls units. The statically referenced AOT artifact comes after the unit slice.

- **Interpreter-only nodes.** `Pin`, `Repeated`, `Shadowing`, guards, and the declaration/message nodes stay interpreter-only until the image-resident binding, pinned-predicate, and registry/dispatch contracts are designed. They remain named refusals in the lowered path.
- **First adapter.** The first host adapter is C, with the C conformance harness and the version-equality test landing with it. This exercises the ABI with the least glue and makes the conformance test real.

Why. The reload is the only clean way to load a persisted file or an `.fsproj` change without a shell build, so it is an exception in mechanism, not in intent. Units are the next writer step because they unlock closures and pattern matching; the AOT pipeline before units would freeze a shape that is still moving. The pattern roles and declaration nodes are refused because the first-class contracts they need are not designed; lowering them now would bake a wrong model into the instruction set. C is the first adapter because the ABI is C-shaped and a foreign host exposes marshalling defects an in-process host hides.

Status: landed. `AGENTS.md` records the reload exception, the specification status matrix records the interpreter-only nodes and the C target, and the writer unit slice is the next work item. Current suite status is in `docs/commitments-ledger.typ`.

D131 — Surface, subtyping, Basic English, and verification decisions

Decision.

- **Surface.** There is no comment form. Whitespace separates words, and every word stands on its own with its own semantics; a source reads like regular language. The existing quoted form is a single-word text literal; a quoted string with spaces is not one token, and multi-word text is a sequence of words or values, not a quoted string. The tokenizer does not gain a comment or multi-word-string scanner (D131).
- **Subtyping.** `set-subset` is the value-level subset operation. A type-level subtype relation is not a word; add it only when a consumer (guards, the lattice, verification) needs it.
- **Basic English.** No expansion until a product sentence requires it. The controlled grammar is deliberate; speculative growth weakens it.
- **Verification.** Z3 verification is deferred. Get the language stable first. The existing verification features remain, but there is no new scope and no claim on arbitrary unified programs.

Why. The surface is word-oriented: comments and multi-word quoted strings would add a second lexical layer the language does not need, and the one-word literal keeps the tokenizer honest. Value-level subset already exists as `set-subset`; a type-level relation is a different consumer and should not be invented before one exists. Basic English is a controlled language, not a natural-language parser. Verification is a feature of the specified and compiled forms; growing it while the semantics are still moving would verify a shape that is about to change.

Status: landed. The specification's lexical, subtyping, Basic English, and verification chapters and the status matrix are aligned. The beyond-set decision is D132. Current suite status is in `docs/commitments-ledger.typ`.

D132 — Sequences and recursive types are target; multisets and co-products wait

Decision. Sequences and recursive types are target language.

- A **sequence** is a set with an order: a value plus a position function. The `Tensor` is already ordered, so the representation is the first widening past sets. Sequences add an ordered-collection projector, order-preserving operations, and ordered pattern support.
- A **recursive type** is the least fixed point of a type constructor. It lets the type layer describe lists, trees, recursive messages, and the language's own expressions, patterns, classes, and packages, which are already recursive at run time.

Multisets and co-products are not target now: there is no current consumer, and both change the lattice laws and the verification story. `set-subset` remains the value-level subset operation; a type-level subtype waits for a consumer (D131).

Why. Sequences are the representation already in the store and are needed for practical ordered data. Recursive types are needed because the language cannot describe its own structures without them. Multisets and co-products are type-system additions without a consumer; adding them now would complicate the lattice and the verifier before the core is stable.

Status: target adopted; both are planned, not implemented. The next steps are a sequence projector and its words, and a recursive reference in the portable type document.

D133 — Prelude deferral, free-variable capture, distribution, ratchets

Decision.

- **Prelude port.** Deferred until the core port (kernel, lowering, and writer) is complete. `docs/prelude/` stays reserved; the F# reference remains the shipped Prelude.
- **Lambda capture.** The lowerer replaces conservative capture with a free-variable analysis. A lambda captures only the enclosing names its body uses, accounting for its parameters, inner `let` bindings, pattern variables, and nested lambdas. The unit environment and prologue are unchanged; only the captured set shrinks. D123′s conservative-capture note is superseded.
- **Distribution.** Promoted to a target track. It is planned, not implemented; `scale_transport_send` is the ABI seam. Node identity, remote send/receive, and partition/failure semantics are to be designed. It is no longer listed as optional.
- **Ratchets.** After the next writer unit slice, turn the closed `Partial`/`Planned` status-matrix rows into tests.
- **`let`.** D122′s lexical bare-name rule is final; it is not a staging step.

Why. Free-variable capture removes unused captures while keeping the same observable semantics; the analysis is small and directly testable. A distribution target changes the roadmap and must be visible as planned rather than optional. The Prelude port waits for the core to be stable so that a ported feature is measured against a settled machine. Ratchets follow the next slice because it changes several status rows at once.

Status: landed. The free-variable capture passes two new lowering checks: an unused enclosing name is not captured, and a used one is. The suite is 1290 checks. The other items are recorded: the Prelude port is deferred, distribution is a target, and ratchets follow the next writer slice. Current suite status is in `docs/commitments-ledger.typ`.

D134 — The writer emits one method per unit and compiles direct lambda calls

Decision. `IlWriter.emit` takes a `LoweredProgram`. It emits a static method for every `LoweredUnit` and a `Run` entry method. A `MakeClosure(name, captures, arity)` immediately followed by `CallClosure` compiles to a direct call to the unit method: the captures are passed as an

`EntryIdentity[]`, the unit method calls `WriterRuntime.EnterUnit` (which pops the arguments, prepends the captures, and installs the frame), runs the unit body, calls `ExitUnit`, and returns. `PushCapture` is supported. A first-class closure, a `MakeClosure` that is not immediately called, is refused as `UnsupportedOpcode "make-closure"`; a stray `CallClosure` is refused as `"call-closure"`. The unit methods take `(KernelState<EvaluationState>, EntryIdentity[])`; `Run` stays `(KernelState<EvaluationState>)`.

Why. This is the smallest real unit/method slice. It compiles the common direct lambda call without inventing a first-class closure representation or a global dispatch table, and it keeps the interpreter as the fallback for closures that escape. The frame protocol is the kernel's: arguments first, captures after, and the prologue copies the frame into slots. The entry method calls the unit methods, which is the shape the emitter needs.

Status: landed. The three new checks pass: a lambda parameter, a captured enclosing name, and a nested unit call; the first-class closure refusal is the existing unsupported-opcode check. The granular suite is 26. Current suite status is in `docs/commitments-ledger.typ`.

D135 — Closed-decision ratchets

Decision. Add tests that pin the closed status rows from D130–D133:

- the surface has no comments and a text literal is one word: `splitText` leaves `//` as a token and splits `"hello world"` into two tokens;
- `set-subset` is the value-level subset, and `subtype` is not a word;
- the interpreter's `Pin` and `Shadowing` pattern roles work; the lowered refusals were already tested.

Why. A status row is a decision only if a test can catch its reversal. The surface tests pin D131; the subset and subtype pair pins the D131 subtyping row; the `Pin`/`Shadowing` tests close the implemented half of the D130 pattern row. The lowered refusals, the writer first-class refusal, the declaration refusals, and the Basic English closed classes already have tests.

Status: landed. The example suite is 1298 checks (1258 examples and 40 properties); the granular suite stays at 26. Current suite status is in `docs/commitments-ledger.typ`.

D136 — The C adapter and the conformance harness

Decision. The first host adapter is C, in `components/adapter-c/`. It loads the kernel shared library, resolves the eight exports, and calls `scale_abi_version()` before any other call. Its header defines `SCALE_EXPECTED_ABI_VERSION` and a compile-time assertion that it equals the header's `SCALE_ABI_VERSION`. A mismatch is refused at open with `SCALE_ADAPTER_VERSION_MISMATCH`; the adapter never calls `scale_init` on a kernel it does not recognize. The adapter does not own the kernel handle: the host supplies the callback table, calls `scale_init`, and calls `scale_free`.

`components/adapter-c/conformance/conformance.c` is the executable definition of an adapter. It checks the ABI version, both sides of version equality (the adapter refuses a different ABI; the kernel refuses an image with a different schema), the prelude dispatch, a declared primitive dispatch, the result, and handle teardown.

Why. The header is the ABI and the version is the release identity. The adapter must check the version before it trusts any other symbol, because a different version may have a different signature. The image schema is the kernel's half of the same rule: `scale_init` already refuses a schema other than

1. The harness makes the boundary executable instead of prose.

Status: landed; native run release-gated. The adapter and the harness compile with `cc`; the fake-kernel probe exports only `scale_abi_version` returning 2, and the adapter refuses it before resolving the remaining symbols, so the equality rule is exercised in alpha. The full conformance run needs a native kernel shared library, which a NativeAOT publish produces at release. Example suite 1300 (1260 examples and 40 properties); the granular suite stays at 26. Current suite status is in `docs/commitments-ledger.typ`.

The version export gained a managed `versionValue` form, matching every other export; managed hosts must call that, never the `UnmanagedCallersOnly` member. The two ABI checks use it.

D137 — The repository is a Polylith workspace

Decision. Adopt the Polylith architecture for the language-plus-engine monorepo. `workspace.edn` lists the nodes; the `.fsproj` graph is the dependency truth; `just poly-check` (`tools/polylith-check.fsx`) reconciles them and enforces the boundaries: components reference components only, bases reference components, projects compose components and bases, no cycles, and every component has an interface. The Clojure `poly` CLI is not required; `just` is the tool surface, and maintenance recipes are added only when a task needs them.

Stage 1 moved the kernel to `components/kernel` and the C adapter, with its conformance harness, to `components/adapter-c`. The kernel's interface is `components/kernel/scale_abi.h`; the adapter's is `components/adapter-c/scale_adapter.h`. Stage 2a moved the app to `bases/scale-cli` as the base and added `projects/scale` as the publish target; stage 2b splits the Prelude, verification, demo, and development nodes. Nu joins after that as its own node, wrapped by `nu-host`; both Nu and Scale are NativeAOT, so the C ABI is the seam.

Why. The language/engine boundary must be structural, not a convention. The AOT requirement on both sides makes the C ABI the only stable contract; the engine is a host, not a language component. Polylith names the boundary and `poly-check` keeps it from decaying.

Status: stage 2a landed. `poly-check` reports 2 components, 1 base, 1 project, 0 legacy; the C compile and version probe pass. Stage 2b's full split landed: the components are kernel/core/typing/prelude/verification/demo/cli/ adapter-c, `development/scale` holds the suite, and the base is the entry. The example suite is 1300; the granular suite is 26. Current suite status is in `docs/commitments-ledger.typ`.

D138 — The module dependency map and the corrected component cut

Decision. Keep `tools/module-dependencies.fsx` as the JIT-only map: it reads the `.fsproj` compile order, extracts each file's `open` declarations, and reports the compile-order positions where the prefix has no edge into the suffix. The compiler remains the authority; the tool is a hint.

Findings. The app is one sequential chain. The open-only graph reports a clean cut at almost every file, but it misses qualified references: `evaluator/runtime-guard.fs` uses `TypeLattice` and `discourse.fs` uses `Violations` without opening them. The first practical cut after the evaluator is after `evaluator/kernel-bridge.fs`: the prefix then contains `domain-expression`, `type-lattice`, `smt`, `violations`, `nearest-sentence`, `presentation`, `emitter`, and the evaluator. That is the Prelude core; the verification layer is not separable at the earlier cut, and it carries the Hedgehog/Z3 packages with the typing files.

Target. A foundation `components/core` (types, tensors, ASTs, store), then `components/typing` (domain-expression, type-lattice, violations, nearest-sentence, presentation, smt), then `components/`

prelude (evaluator and writer); verification, demo, and development sit above. The immediate safe split is cut 56, not the earlier directory-feel cut.

Status: full split landed. The components are kernel, core, typing, prelude, verification, demo, cli, and adapter-c; development/scale holds the suite. poly-check reports 8 components, 1 base, 1 project, 1 development. The example suite is 1300; the granular suite is 26; 70 discovered tests. The suite counts are in docs/commitments-ledger.typ.

D139 — Verification, demo, CLI, and development split

Decision. Extract components/verification (discourse, refiner, decision tables, diagnostics), components/demo (generated samples, player domain, fixed-specification acceptance, showcase), components/cli (argument parsing), and development/scale (example, granular, property, and differential suite). bases/scale-cli is only entry.fs; it composes the components and the development node.

Why. The CLI parser must be a component because the development suite tests it, and a development node cannot reference a base (the base already references development; that would be a cycle). The demo resolves the repository root by workspace.edn, not scale.fsproj, after the move; the witness corpus stays in the base. The base references development because the NativeAOT binary intentionally ships the example suite and the showcase (D23).

Status: landed. 8 components, 1 base, 1 project, 1 development; example suite 1300 (1260 examples, 40 properties); granular 26; 70 discovered tests.

D140 — The Nu node, the host skeleton, and the jj pilot

Decision. Vendor the Nu engine as the external node components/nu-engine (cloned from the Nu repository with its .git removed). Add the host skeleton components/nu-host, bases/nu-game, and projects/game; the host is a placeholder until the Nu API bridge is written. The jj pilot is deferred because jj is not installed on this host; the repository remains git, with the manifest ready for a later colocated jj migration.

Why. Nu is the AOT game engine and the embedding host. The engine is not one of our components: it is an external node, wrapped by nu-host, and the C ABI/image is the seam. The vendor keeps the source in the monorepo; the engine boundary stays explicit. The jj colocated mode cannot be verified without the tool, so it is not adopted silently.

Status: node and skeleton landed. The external node, host component, base, project, and manifest are in place; the engine is not built or wired. The poly-check counts the external node; the line-count tool excludes the vendored tree. The jj pilot is deferred.

D141 — The demo resolves its content root in a checkout and in a payload

Decision. Resolve the demonstration's files through one resolver instead of the repository root alone. Demo.demoDirectory walks up from the application's base directory for workspace.edn; in a checkout it returns components/demo/demo, and outside a checkout it returns the demo directory beside the binary. specificationPath, runLoop, and the suite's byte-identity baseline reads all use that directory. The base already copies player.scale and demo/generated/* into the output and publish directories, so a payload carries the same files under <base>/demo/.

Why. A NativeAOT publish payload contains neither workspace.edn nor the source tree. The old walk fell back to the base directory and then looked for components/demo/demo, which a payload

does not have, so the showcase failed only after publish. Resolving the repository first keeps development behaviour unchanged: a run still rewrites the checked-in artifacts, and the byte-identity test still guards them. The payload fallback then makes the copied content directory the source of truth there. The suite reads its baselines through the same resolver, so one test covers both layouts.

Status: landed. `Demo.demoDirectory` has both branches, and two checks pin the mapping (a repository root maps to `components/demo/demo`; no repository maps to `<base>/demo`). The run-loop test reads its baselines through the resolver. The example suite is 1302 (1262 examples and 40 properties); the granular suite stays at 26 and discovery stays at 70 tests. The release AOT smoke remains the release-time proof that the copied payload runs the showcase.

D142 — The prose surface: sentence capitalization, no comma, `.prose` files

Decision. The human programming surface is a `.prose` file. A sentence ends with a full stop and starts with a capital, so the clause keywords are `Match`, `When`, and `Otherwise` and the connective is lowercase `then`. Arguments and pattern children are separated by whitespace inside parentheses. There is no `end` keyword and indentation is not semantic.

The first slice is one top-level `Match` with `When`/`Otherwise` clauses, lowering to `ExpressionAst.PatternMatch`. It covers literals, words, calls, and patterns (wildcard, fresh variable, number, text, atom, constructor, typed) and returns `Result<Expression, ProseError>` with named refusals.

Why. Normal language conventions make the surface readable without a second syntax: capitals start a sentence, a full stop ends it, and no `end` closes a block. A separate `.prose` file keeps the specification document (`.scale`) and the program surface distinct.

Deferred. Text literals with spaces, nested matches, the program result, and the separator set are recommendations in `docs/prose-surface.typ`, to be settled by the bootstrapping examples before the full surface.

Superseded in part by D143. The comma is no longer reserved and refused. It now delimits the multiple-choice construct, in the same sentence as the `Match` head; the rest of D142 stands.

Status at D142: first slice landed. `components/core/prose-parser.fs` defines `ProseError`, `describeError`, and `parseProgram`; twelve checks in `development/scale/tests.fs` pinned the first slice at 1314 example checks (1274 examples and 40 properties).

D143 — The comma delimits the multiple-choice construct

Decision. The comma is the multiple-choice delimiter. A match can be written as one prose sentence:

```
Match 5, when 1 then "one", when 2 then "two", otherwise "many".
```

The comma precedes each choice; the sentence's full stop ends the match. The sentence form remains an alternative: `Match 5.` followed by `When ...` and `Otherwise ...` sentences. Inside an inline match the choice words are lowercase `when`/`otherwise`, because they are not the sentence's first word; the clause words are capitalized only when the clause is its own sentence. Names stay case-sensitive at D143; D144 changes that. A comma anywhere else is `UnexpectedComma`; it never separates call arguments or pattern children.

Why. The comma was reserved for a semantic meaning (D142). This is that meaning: it makes the multiple-choice construct a single prose sentence instead of an indentation-shaped block, so the

surface reads as written language. The sentence form stays for long matches where one line is unreadable. Both forms lower to the same `ExpressionAst.PatternMatch`, so the choice is presentation, not semantics.

Status: landed. `components/core/prose-parser.fs` accepts the inline form and the sentence form, lowercase `when`/`otherwise` inside the inline match, and refuses a comma outside the delimiter positions. Eighteen checks pin both forms to the same `PatternMatch`, the canonical portable document, and the refusals; the example suite is 1320 (1280 examples and 40 properties).

D144 — Spaced names, text with spaces, nested matches, and the program result

Decision.

1. **Program result.** A `.prose` file is a sentence sequence, evaluated in order on one stack; the final stack value is the result. The first slice returns the single `PatternMatch`; a multi-sentence program becomes a `Block` when the sequence lands.
2. **Text with spaces.** A `.prose` text literal may contain spaces and keeps its case. `.scale` and the IR keep the one-word rule (D131); the rule is tokenization, not semantics.
3. **Names.** The `.prose` surface writes names as lowercase words separated by spaces, exactly like Basic English: `is alive`, `can jump`, `player`. A surface phrase the vocabulary knows is canonicalized to that vocabulary token (`is alive` -> `isAlive`); a free multiword name uses kebab-case (`player-state`). No camelCase or PascalCase is written in `.prose`. Constructors are names followed by `(`; a type after `:` is a lowercase name. The canonical vocabulary token is the existing IR name, not a surface spelling.
4. **Nested match.** A `Match` in a clause body is allowed. It is closed by the lowercase word `close` (the sentence form writes `close.`). Indentation is not semantic; it stays for the reader. `close` is the chosen delimiter; indentation-based delimitation is not adopted.
5. **Keywords are lowercase.** `match`, `when`, `then`, `otherwise`, and `close` are lowercase everywhere. A line break does not start a new sentence; a clause on the next line is still part of the match, so it is not capitalized. Only quoted text keeps case. This supersedes the sentence-capitalization rule of D142 and D143.

Why. A multi-sentence file with the final stack value keeps the concatenative core and avoids a second evaluation model. Spaced lowercase names make the prose surface read as prose and reuse the Basic English vocabulary canonicalization instead of inventing a second naming scheme. `close` closes a nested match explicitly without making indentation semantic or borrowing `end`.

Staged. The first slice still parses one top-level match and returns it directly; the sentence sequence and `Block` are the next step. The vocabulary canonicalization is the same `VocabularyEntry.Form`/`Token` mapping the Basic English tokenizer uses. Text with spaces, lowercase names, and `close`-delimited nested matches land together; the sequence follows.

Status: landed. `components/core/prose-parser.fs` accepts lowercase keywords, vocabulary-canonicalized spaced names, text with spaces, and `close`-delimited nested matches in both the inline and line forms. Twenty-five checks pin the behavior; the example suite is 1327 (1287 examples and 40 properties). The sentence sequence and the `Block` result remain the next step.

Superseded in part by D145. Kebab-case is not used; a free multiword name is a `<head> of <owner>` phrase such as `state of player`.

D145 — The prose program is a sentence sequence; no kebab-case for now

Decision.

1. **Program sequence.** A `.prose` program is a sequence of sentences. They are evaluated in order on one stack; the final stack value is the result. A one-sentence program keeps its expression AST; a multi-sentence program is an `ExpressionAst.Block` of the sentences in source order. The parser returns the sequence; the evaluator seam follows.
2. **No kebab-case.** A free name is a single lowercase word or a `state of player` phrase. The parser joins the `of` phrase into one lowercase name with spaces. A hyphen is not part of a name, and no camelCase or PascalCase is written. A multiword vocabulary phrase uses the vocabulary's spaced surface form (`is alive` -> `isAlive`).

Why. The stack is the evaluation model; a file that holds several sentences must not invent a second model. `state of player` is ordinary prose, unambiguous because `of` is the connector, and it avoids both kebab-case and a second naming rule.

Status: landed. `parseProgram` returns the single expression for one sentence and an `ExpressionAst.Block` for two or more, in source order. The lexer no longer treats a hyphen as part of a word, and a free multiword name is joined from its `of` phrase (`state of player`). Thirty prose checks; the example suite is 1332 (1292 examples and 40 properties). Evaluation on one stack is the next seam.

D146 — The prose program runs on the kernel stack

Decision. A parsed `.prose` expression is lowered with `EvaluatorLowering.lowerExpression` and run on the kernel `LoweredVm` over the one stack; the final stack top is the result. The first slice returns a `PatternMatch` or a `Block`; the seam is exercised by the suite. A `.prose` CLI mode and a file reader remain later.

Why. The parser must not invent a second evaluator. Lowering already covers literals, words, calls, blocks, and ground pattern matches; the prose front-end reuses it, so the surface and the IR share one semantics. A sequence is a `Block`, and the VM leaves the final value on top.

Status: landed. The suite parses a prose program, lowers it with `EvaluatorLowering.lowerExpression`, and runs it on the kernel `LoweredVm` with `EvaluatorKernelBridge.executeWord`. Three checks cover the selected match body, the final value of a sequence, and two matches in order. Thirty-three prose checks; the example suite is 1335 (1295 examples and 40 properties).

D147 — Prose as a script-only shell

Decision. A `.prose` file can run as a script through a new CLI mode. The shell is a new prose shell that calls external commands; it is not a POSIX clone. Script-only comes first; the REPL follows. Stage 1 is unrestricted: it runs with the user's authority and no capability sandbox, and that is recorded as debt to close before any untrusted script is accepted.

The effect surface is a `shell-host` component (start a process, forward or capture standard streams, wait for the status, later pipes, files, environment, working directory, and signals). The kernel and the Prelude gain no OS words; the host adapter is the only component with OS authority and uses the existing host-call seam.

Why. The prose surface already has the command vocabulary, the stack is a pipeline model, the ABI already owns effects, and the CLI is NativeAOT. A script runner plus a host adapter is a smaller and safer change than moving process or filesystem words into the kernel.

Deferred. The shell implementation is deferred until the test batch and the `.prose` file reader. `docs/prose-as-shell.typ` records the command model, the Stage 1 acceptance tests, the stream and job-control stages, and the security plan.

Status: design accepted; implementation deferred. No shell word, host adapter, or CLI mode exists yet.

D148 — The first test batch: lowering differential, prose properties, lowered-IR snapshot

Decision. Adopt the first batch from `docs/test-strategy.typ`:

1. **P1.** A Hedgehog property generates arithmetic token programs and runs each through the token interpreter and the lowered kernel VM. The interpreter is the oracle; a value or failure-family disagreement falsifies the property.
2. **P2.** A Hedgehog property generates a numeric prose match and asserts that the inline and line forms parse to the same `ExpressionAst` tree.
3. **S1.** A Verify snapshot pins the lowered instruction sequence for the differential corpus, so an instruction-shape change is a reviewed diff.

P3 (the pattern-matcher oracle) and the remaining batch wait for the next slice.

Why. The fixed eighteen-program corpus cannot falsify a lowering bug outside its cases; the property closes that gap. The prose surface is example-only today, so the property extends the D143 equivalence. The snapshot catches a lowering change that preserves behavior but drifts the instruction shape.

Status: landed. P1 and P2 are in the property suite and S1 is the granular snapshot `lowered-corpus.verified.txt`. The example suite is 1337 (1295 examples and 42 properties); the granular suite is 27 cases (11 snapshots, 7 integrations, 9 writer checks) and discovery is 73 tests. The lowering property compares the interpreter and the lowered VM over generated arithmetic; the prose property compares the inline and line forms over generated numeric matches.

D149 — The `.prose` file reader and CLI mode

Decision. `scale --prose PATH` reads a `.prose` file, parses it with the demonstration vocabulary, lowers it with `EvaluatorLowering`, and runs it on the kernel `LoweredVm` over the one stack. The final stack top is printed; a parse, lowering, or run failure names the reason and returns a non-zero exit code. The CLI mode is `Cli.ProseFile`; the runner is `Entry.runProseFile`.

Why. The parser and the stack seam already exist. The file reader is the smallest end-to-end user surface and the first consumer of the script shape. It adds no OS effects; the prose shell (D147) keeps those in a host adapter later.

Status: landed. `Cli.ProseFile` parses `--prose PATH` and `Entry.runProseFile` reads, parses with the demonstration vocabulary, lowers, and runs the program on the kernel `LoweredVm`; it prints the final stack top and returns a non-zero exit code on a named refusal. One CLI parse check joins the suite; the runner was exercised with a match, a sequence, and a parse refusal in the base session. The example suite is 1338 (1296 examples and 42 properties).

D150 — `LoweredVm` becomes `LoweredAbstraction`

Decision. Rename the module and file `LoweredVm` / `lowered-vm.fs` to `LoweredAbstraction` / `lowered-abstraction.fs`. Rename its error type `VmError` to `AbstractionError` and the writer runtime's `VmFault` to `AbstractionFault`. Rename the call-site values `vmState` and `vmValue` to `loweredState` and `loweredValue`, and the prose `kernel VM`, `lowered VM`, and `slot VM` to `kernel abstraction`, `lowered abstraction`, and `slot abstraction`.

Why. `Vm` is an abbreviation for an established idea, but this module is not a virtual machine in the host sense: it is the executable abstraction over the lowered instruction set. The longer name says what the module is and matches the file. The error and fault names follow the module. Historical decision records keep the old name as history; current code and current documentation use the new one.

Status: landed. The module, file, error type, fault type, call-site values, test names, and current documentation use the new names. The suite is unchanged: 1338 example checks (1296 examples, 42 properties), 27 granular cases, and 73 discovered tests.

D151 — Drop the unified Julia/Elixir language; integrate instances by actor IPC

Decision. The unified Julia/Elixir language of D93 is dropped. Scale does not host Julia and Elixir behind one source language, one AST, one type hierarchy, one dispatch model, or one module and macro system, and it does not provide Julia or Elixir front ends. The proposal in `docs/rejected-ideas/unified-language.typ` is rejected.

Scale's own core language remains: the concatenative token surface and the general IR, with Basic English and the prose surface as front ends over it. The term "unified language" is retired; the direct surface is the **core language**.

Different implementations are separate **implementation instances**, each running in its own host language. They integrate through the actor model's IPC: an actor handle may address an actor in another instance, and a message envelope crosses the instance boundary through the host adapter's transport. The local actor model, the image promotion rule, and the `scale_transport_send` ABI seam are the starting point. Instance identity, discovery, wire encoding, delivery, partition, and security are planned, not implemented. `docs/actor-ipc.typ` records the model, the staging, and the open decisions; specification section 11.7 is the normative summary.

Why. Julia and Elixir have incompatible dispatch, type, module, macro, and runtime semantics. A single source language that hosts both either loses one semantics or becomes a third language with its own semantics; either outcome contradicts the rule that a feature is documented before it exists and that a failure is named. The host languages already have mature runtimes. The actor model is the boundary the project already has, the `scale_transport_send` seam already exists, and an actor message is the smallest contract that lets implementations differ internally while interoperating.

What remains. Basic English and the prose surface are front ends of the core language, not of a unified Julia/Elixir language. The kernel/Prelude/image split, the local actor model, the verification subset, and the AOT contract are unchanged. The Scale Prelude port remains deferred (D133).

Trust boundary. A message from another instance is untrusted input. A payload is a portable value, never a host object, closure, frame, or execution state. A remote handle is a capability. A remote failure is a named value; a partition is an outcome, never a silent loss.

Status: recorded (documentation). D93 is revoked; no source or test change. The suite is unchanged: 1338 example checks (1296 examples, 42 properties), 27 granular cases, and 73 discovered tests.

Non-goals. A unified Julia/Elixir source language; Julia or Elixir front ends; a shared type hierarchy or dispatch model across host languages; transparent remote objects; a distributed garbage collector; and a second evaluator or image.

D152 — The actor wire protocol is implemented in-repo, Cap'n Proto-inspired

Decision. Scale implements its own actor wire protocol. It does not depend on the Cap'n Proto runtime, code generator, or library. The design takes the three major Cap'n Proto implementations as references: C++ for the normative wire encoding and the complete RPC state machine, Rust for validation on traversal, explicit `Result` on fallible getters, and lifetime-parameterized reader and builder views, and Go for grow-on-write buffers, pooling, transport-agnostic framing, and connection concurrency. Protobuf is the reference for the code-generator contract and schema evolution; OCapN is the reference for capability references and interop. Neither is a dependency.

The first scope is the actor IPC envelope and portable payloads: segments, a bounded message, struct/list/text/data/scalar fields, far pointers, defaults, a capability table, a version and capability handshake, and a length-prefixed frame over a transport. It is not a schema compiler, an RPC system with promise pipelining, three-party handoff, distributed GC, an arbitrary object graph, or a Cap'n Proto compatibility claim. Those wait for a consumer.

Why. Cap'n Proto's layout fits an actor boundary that may carry zero-copy views, capability handles, and future RPC. Reusing the implementations' design work avoids inventing a pointer format, but depending on the Cap'n Proto runtime would add a foreign runtime and limit the NativeAOT and linear-capability constraints. An in-repo subset keeps the kernel and adapters under one version and one test strategy.

Boundary. This is the Scale actor wire protocol, not Cap'n Proto. We do not claim wire compatibility. If compatibility becomes a goal, it is a separate decision with a conformance suite against the reference implementation; any adjustment then needs a version. `docs/binary-format.typ` is the canonical plan; `Notes/binary-format.md` is the early estimation that seeded it.

Status: recorded (documentation). No source or test change. The suite is unchanged: 1338 example checks (1296 examples, 42 properties), 27 granular cases, and 73 discovered tests.

Non-goals. A Cap'n Proto runtime dependency; a full Cap'n Proto implementation in one step; a schema compiler, RPC promise pipelining, three-party handoff, distributed GC, or a compatibility claim before a consumer.

D153 — Protobuf-style schema tooling over a Cap'n Proto-compatible wire

Decision. The actor message keeps one frame with a logical envelope/payload split: the root struct is the envelope (target, sender, optional reply, message identity, capability-table reference, body pointer), and the body is a union or pointer in the same message. Bulk data may be out-of-line in a segment or stream; there is no second envelope frame.

The wire layer follows Cap'n Proto for the frozen subset. The schema and code-generation layer adopts Protobuf's plugin contract and evolution discipline: a descriptor set, `CodeGeneratorRequest` and `CodeGeneratorResponse`, field numbers and reserved ranges as schema metadata, a

compatibility matrix, and a plugin per language backend. Field numbers and reserved ranges never appear on the wire; the compiler maps them to Cap'n Proto ordinals. `oneof` maps to a union. This is compatible with Cap'n Proto because the Protobuf features are build-time only.

Unknown fields are not preserved by tag; a schema may opt into raw-body retention, and otherwise unknown fields are dropped by explicit policy. `Any` is a typed union plus a schema registry; `AnyPointer` is allowed only behind validation and a whitelist. A codec tag carrying Protobuf wire messages beside Cap'n Proto messages is a second protocol, not a compatibility feature.

This narrows D152: a small schema compiler is in scope (W4) and the Protobuf plugin contract is in scope (W5). A full Cap'n Proto toolchain, RPC promise pipelining, three-party handoff, and distributed GC remain out of scope. D152′s "no compatibility claim" becomes "no claim until the reference conformance test passes"; after it passes, the frozen subset may be called Cap'n Proto-compatible.

Why. Protobuf's plugin and evolution discipline solve multi-language code generation and schema growth better than a hand-written generator. Keeping those features in the descriptor and compiler layer preserves the wire layout and the zero-copy model. Putting them on the wire would make the protocol neither Cap'n Proto nor Protobuf.

Boundary. Compatibility is a tested property, not a label. The reference implementation must accept our bytes and we must accept its bytes for the frozen subset. Adjustments that change the wire are versioned and are not claimed as compatibility. Unknown-field preservation is opt-in and explicit.

Status: recorded (documentation). No source or test change. The suite is unchanged: 1338 example checks (1296 examples, 42 properties), 27 granular cases, and 73 discovered tests.

Non-goals. A Protobuf wire codec in the same protocol; Protobuf extensions and unknown-field tags on the wire; an open `Any`; a full schema compiler, RPC promise pipelining, three-party handoff, or distributed GC before a consumer.

D154 — The first actor IPC slice: a numeric envelope over a Unix socket

Decision. The first working actor IPC slice is implemented. A new `ipc` component contains `Wire` and `ActorHost`. `Wire` implements a single-segment Cap'n Proto subset: a segment table, a root struct, six data words, one pointer word, and numeric or text bodies. `ActorHost` binds a Unix socket, receives one numeric envelope, delivers it to the root actor's mailbox, runs the actor program `receive-message; 1; add`, and returns the numeric reply. The CLI exposes `--wire-listen PATH` and `--wire-connect PATH NUMBER`.

Why. This is the smallest end-to-end proof that the wire, the host adapter, the mailbox, and the actor loop meet. It exercises a real actor rather than a host-side echo: the actor receives the message and performs the addition. It also fixes the component boundary before the schema compiler and capability table arrive.

Boundary. One connection, one numeric message, and the root actor. The codec has a text body and refuses malformed pointers, but the actor host accepts numbers only. There are no capabilities, discovery, far pointers, remote supervision, reference conformance, or schema compiler yet. The local actor words and the kernel ABI are unchanged. `docs/actor-ipc.typ` records the staging; `docs/binary-format.typ` records the wire subset.

Status: landed. The `ipc` component is built and loaded; the granular integration test exchanges 41 and returns 42 over a Unix socket, and the CLI does the same in two processes. The suite is 1338

example checks (1296 examples, 42 properties), 28 granular cases (11 snapshots, 8 integrations, 9 writer checks), and 76 discovered tests.

Non-goals. Capability handles, instance discovery, remote supervision, far pointers, text messages through the actor host, source-level remote send, and a compatibility claim before the reference conformance test.

D155 — S2: worker capabilities and remote supervision

Decision. The actor envelope gains one operation word: send, reply, monitor, monitor acknowledgement, or down. The data section grows from six to seven words. `ActorHost` keeps a capability table; capability 0 is the root and `spawnWorker` creates a child from `receive-message 2 multiply`, assigns it capability 1, and registers a root monitor.

`serveSupervised` and `connectSupervised` implement the first remote supervision: the client monitors capability 1, sends 21, receives 42, and then receives `down normal` when the worker terminates. A missing socket is the named `Unreachable` failure. The CLI uses the supervised path.

The change also fixes a scheduler bug: `continueAfter` now preserves the threaded store when no ready actor remains. Without it, a monitor's Down message was dropped after a child's normal exit, even though the monitor was consumed. The S2 integration test is the regression test for that fix.

Why. A capability index that names a non-root actor is the smallest proof that the boundary is actor-addressed rather than root-addressed. A monitor and a Down message exercise the existing local supervision machinery, so S2 reuses the actor model instead of adding a second one.

Boundary. One connection, two operations, one worker, and the existing monitor machinery. There is no discovery, security handshake, far pointer, remote code change, or reference conformance yet. The wire is still a single-segment Cap'n Proto subset, and the compatibility claim remains gated.

Status: landed. The worker round-trip and the remote exit pass in the granular integration test and in two CLI processes. The suite is 1338 example checks (1296 examples, 42 properties), 29 granular cases (11 snapshots, 9 integrations, 9 writer checks), and 79 discovered tests.

Non-goals. Discovery, authentication, remote code change, far pointers, a second capability form, and a Cap'n Proto compatibility claim.

D156 — The BEAM wire harness: an Elixir peer and listener

Decision. `adapters/elixir` is a wire conformance harness, not a language port. B0 is the Elixir wire codec (`Scale.Wire`) and the supervised peer (`Scale.Peer`): it monitors capability 1, sends 21, reads 42, and reads `down normal` from the F# listener. B1 is the reverse direction (`Scale.Listener`): a BEAM listener creates a BEAM worker process, acknowledges the monitor, doubles the number, and reports the worker's normal exit to the F# client.

The old architecture is unchanged: the kernel, the F# Prelude, and the host adapters remain the shipped path. A full BEAM language port and a Rustler NIF adapter for the C ABI are deferred until the Prelude is in Scale and a native kernel artifact can be built and tested. This harness only proves the wire and the actor semantics across hosts.

Why. BEAM processes, mailboxes, and monitors are a natural host for the actor boundary, and a cross-language peer and listener prove the wire before anyone commits to a port that cannot yet be tested.

Boundary. `adapters/elixir` speaks the Scale actor wire and the send/reply/monitor/monitor-ack/down operations. It has no Scale parser, evaluator, image, tensor store, vocabulary, verification, Rustler NIF, or C ABI adapter. It does not claim Cap'n Proto compatibility. The F# suite is unchanged; the Elixir suite is eight ExUnit tests, and both cross-language directions pass.

Status: landed (wire harness). The Elixir peer works against the F# listener; the F# supervised client works against the BEAM listener. `mix test` passes eight tests. The full BEAM port and the Rustler adapter are not started.

Non-goals. A BEAM evaluator, image/store port, OTP behaviour layer, Rustler C ABI adapter, distributed registry, or Cap'n Proto compatibility claim.

D157 — Language-level remote send records a transport request

Decision. `send-to` recognizes a host-registered remote handle whose tensor is `[RemoteHandleTag; instance; capability]`. A tensor with the tag is not enough: the host must register the entry in `SchedulerState.RemoteHandles`, so a program cannot forge a remote capability. `send-to` pops the message, records a `TransportRequest` in `SchedulerState.PendingTransport`, and returns without requiring a local mailbox. The local `ActorHandleTag` path is unchanged. The host drains the queue with `Scheduler.takePendingTransport`, reads the message entry from the store, and performs the transport.

The wire is unchanged. This is the language seam: the evaluator stays pure and the host owns I/O. A remote send is a recorded effect, not a second delivery path.

Why. S2 proved the transport at the host level, but the language's `send-to` still rejected remote handles. Recording an effect closes that gap without embedding sockets, retries, or host state in the evaluator.

Boundary. The first landing is one-way: the host drains the request and returns the reply to its caller; the reply is not yet delivered back into the sender's mailbox. There is no discovery, authentication, remote spawn, or far pointer. The host test sends 21 to a remote worker, receives 42, and receives `down normal`.

Status: landed. The language test seeds a remote handle, runs `send-to`, observes one `PendingTransport` request with instance 0, capability 1, the message entry, and sender 0, then drains it through the S2 host. The suite is 1338 example checks (1296 examples, 42 properties), 30 granular cases (11 snapshots, 10 integrations, 9 writer checks), and 82 discovered tests.

Non-goals. Reply delivery into the language, remote spawn, discovery, authentication, far pointers, and a Cap'n Proto compatibility claim.

D158 — Remote replies are delivered into the sender's mailbox

Decision. `LanguageTransport.deliverPending` drains `SchedulerState.PendingTransport`, sends each request through the S2 client, adds the numeric reply to the store, delivers it to the sender's mailbox, and runs the sender. A program that does `send-to; receive-message` now receives the remote result. The first slice allows one sender per drain and a numeric reply; the pump is bounded at 64 steps.

Why. D157 was one-way. A remote call is not a call until the reply returns to the sender's mailbox and the actor resumes. Delivering the reply through the existing mailbox path keeps the actor model as the only delivery mechanism.

Boundary. Numeric replies only; one sender per drain; the down reason stays a host result and is not delivered into the language. Discovery, authentication, remote spawn, and far pointers remain out of scope.

Status: landed. The D157 integration test now runs `send-to; receive-message`, drains the request, receives 42 on the stack, and observes the same 42 and `down normal` at the host. The suite is 1338 example checks (1296 examples, 42 properties), 30 granular cases (11 snapshots, 10 integrations, 9 writer checks), and 82 discovered tests.

Non-goals. Text replies, down delivery into the language, multiple senders per drain, remote spawn, discovery, authentication, and a Cap'n Proto compatibility claim.

D159 — The wire golden vectors and reference interop

Decision. Freeze the single-segment wire subset in `components/ipc/vectors/`. `scale_actor.capnp` declares the seven data fields and the text pointer. Each `<name>.txt` is Cap'n Proto text; each `<name>.bin` is generated by `capnp encode`; `truncated.bin` and `bad-root.bin` are negative vectors. The corpus is `number`, `text`, `monitor`, `monitor-ack`, `reply`, and `defaults`.

The interop is two-way. The F# test decodes every `.bin`, then re-encodes the same envelope and requires the bytes to equal the reference bytes. The `check.sh` script re-encodes every `.txt` with `capnp encode` and compares the bytes, then decodes the Elixir encoder's bytes with `capnp decode` and checks the fields. `capnp` 1.5.0 is the reference implementation.

With this test, the frozen subset is Cap'n Proto-compatible for the fields and byte layout it covers. The claim does not extend to multi-segment messages, far pointers, a different field order, or a schema change.

Why. The spec is the contract, but a compatibility claim needs a reference tool and bytes that both sides accept. The vectors make the subset reviewable, reproducible, and independent of either implementation.

Boundary. One segment, seven data words, one pointer, numeric and text bodies, and the five operations. `capnp` is the reference; Rust and Go source implementations are secondary references, not build dependencies. A change that alters the bytes is a new version.

Status: landed. `components/ipc/vectors/check.sh` passes; the F# suite decodes the reference bytes and the F# encoder matches them byte-for-byte. The suite is 1338 example checks (1296 examples, 42 properties), 31 granular cases (11 snapshots, 11 integrations, 9 writer checks), and 85 discovered tests.

Non-goals. A second wire schema, far pointers, multi-segment messages, canonicalization beyond the checked-in subset, and a Rust or Go binary dependency.

D160 — The live program is a typed term list

Decision. The live program becomes `Term list`; a quotation is a `Quote of Term list`; `apply` splices its terms onto the program. The runtime representation is code-as-data, not token text and not a parsed side channel. The core `Term` is the executable term IR: literals, words, quotations, calls, branches, slots, and closures. The existing `ExpressionAst` is the surface IR; it elaborates to the core terms.

The kernel owns the core term encoding, the term-list loop, the structural and stack-effect verifier, the `VerifiedTermProgram` receipt, the `verify` entry, and the ABI/image schema. The Prelude owns the surface elaboration, word signatures, language type rules, effect tiers, dynamic-code policy,

macros and staging, and the lowering to core terms. The kernel must not gain the parser, macros, the domain lattice, SMT, word implementations, classes, dispatch, conditions, actors, or a second evaluator. The Prelude must not own the kernel's structural verifier, receipt, stack safety, or ABI framing.

The term list is verified once and the receipt is cached. Statically created quotations are typed by construction and enter AOT without a runtime check. Runtime-constructed terms are checked once before their first splice. External text is parsed to terms and then verified. Unrestricted `eval` is a separate verified-interpreter path; it is not part of the AOT contract. Macros expand before final verification.

Why. A concatenative evaluator already treats a quotation as a list to splice. The only reason it felt unverifiable was that `Program` held raw token strings. Making the list typed makes the verifier a static property of the artifact, as with JVM bytecode, eBPF, and WebAssembly. It also makes self-modification a well-defined list edit that can be re-verified, not opaque text mutation.

Boundary. This is recorded as a design, not implemented. The current `Program` is still `string list`; `quotation` still stores text; the verifier and receipt do not exist. The domain SMT verification remains separate from term typing. The migration is staged in `docs/term-ir.typ`: P0 this record, P1 the core term/portable document, P2 the kernel verifier and receipt, P3 the runtime representation, P4 stack typing, P5 the verification effect, P6 macros, P7 the AOT writer, P8 removal of the text/token side channels.

Status: recorded (documentation). No source or test change. The suite is 1338 example checks (1296 examples, 42 properties), 31 granular cases (11 snapshots, 11 integrations, 9 writer checks), and 85 discovered tests.

Non-goals. Implementing the term IR in this change; a full row-polymorphic effect system in the first step; unrestricted runtime `eval` in the AOT path; and moving language semantics into the kernel.

D161 — P1: the core term shape and its portable document

Decision. Add the core term type and the canonical term-program document in the kernel:

1. `Term` variants: `LiteralNumber`, `LiteralText`, `LiteralAtom`, `Word`, `Quote of Term list`, `Branch`, `SlotGet`, `SlotSet`, `MakeClosure`, `CallClosure`, `Fail`.
2. `TermProgram`: `schema`, `form` (`term-program`), and `terms`.
3. Canonical JSON with the term tags `number`, `text`, `atom`, `word`, `quote`, `branch`, `slot-get`, `slot-set`, `make-closure`, `call-closure`, `fail`.
4. Golden vectors: `number-add`, `quote`, `branch`, `closure`, `fail`, and the negative `bad-schema` and `unknown-term`.

P1 is shape and serialization only. It does not execute a term, type a stack, or issue a receipt. P4 adds the signature table and stack-effect checker; P2 adds the verifier and receipt.

Why. The runtime representation has to be a concrete, portable data shape before it can replace token strings. Pinning the shape and its bytes now keeps P2/P3 from inventing a second format.

Boundary. The kernel owns the type and the document because the term list is the executable artifact. No Prelude semantics enter the kernel: a `Word` is a name, not a resolution; `Quote` is a term list, not a closure; and `Branch` is a syntactic term, not an evaluator. The domain SMT types and the stack-effect types are not in P1.

Status: landed (P1). `TermIr` and `TermPortable` are in the kernel; the canonical vectors round-trip byte-for-byte. The suite is 1338 example checks (1296 examples, 42 properties), 32 granular cases (11 snapshots, 12 integrations, 9 writer checks), and 88 discovered tests. P2 adds the verifier and receipt.

Non-goals. Execution, stack typing, signature resolution, verification receipts, macros, and AOT emission in P1.

D162 — P2: the kernel structural verifier and receipt

Decision. Add `TermVerification` to the kernel. It re-checks a `TermProgram` after the portable reader and returns an opaque `VerifiedTermProgram` or a named refusal:

1. canonical document round-trips; a number is finite; a slot, capture, or arity is non-negative; a word or closure name is not blank;
2. every quotation body is checked recursively at its path;
3. the receipt is canonical JSON: `{"schema":1,"form":"term-receipt","digest":"<sha-256 of the canonical term-program>"}`;
4. the receipt record is private, so only `verify` can construct one;
5. `scale_verify` is ABI export nine and bumps `SCALE_ABI_VERSION` to 2. It writes the receipt on success and a diagnostic on a malformed document.

P2 is structural only. Stack underflow, quote input and output types, closure arity, and the `apply` effect need the P4 signature table and stack-effect types. There is no splice term before P3; because P2 verifies every quotation body, P3′s splice cannot introduce an unverified body. Execution still uses `Program: string list`, so the receipt is issued but not yet required by the runtime.

Why. A receipt is only meaningful if the kernel re-checks the exact bytes it will later execute. Canonicalizing first and hashing that document binds the spelling, the term shape, and the digest in one step. Keeping the receipt constructor private makes "verified" a capability rather than a claim a caller can fabricate.

Boundary. The kernel owns the structural verifier, the receipt, and the ABI framing. It does not resolve a `Word`, know a signature, type a stack, or run a term. The Prelude may add a stricter language checker later, but cannot authorize a program the kernel refuses.

Status: landed (P2). `TermVerification`, `scale_verify`, and ABI version 2 are in the kernel; the receipt is canonical and the malformed cases are refused. The suite is 1350 example checks (1308 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Stack-effect typing, stack-underflow rejection, `apply`, execution enforcement, macros, and AOT emission in P2.

D163 — P3a: the token-to-term boundary

Decision. Add `EvaluatorProgram` in the Prelude. `ofTokens` maps an edge token list to core terms: a finite numeric token becomes `LiteralNumber`; every other token becomes `Word`. `ofText` splits external text and reuses the same boundary. A non-finite numeric token is `NonFiniteNumber`, never a silent NaN. `toTokens` is the temporary projection for the kernel bridge and the token-vector history; it accepts only `LiteralNumber` and `Word` and refuses every other term as `UnsupportedTerm`.

P3a does not change `EvaluationState.Program`, which is still `string list`. P3b changes the active state and the caller-program frames to `Term list`; P3c replaces text quotation values with tagged

term documents; P3d removes `toTokens`, the token-vector history encoding, and `splitText` from execution.

Why. The token-to-term boundary is the prerequisite for every later P3 slice. Proving it in isolation keeps the eventual state migration from mixing three changes: literal parsing, quotation code values, and history storage. The refusal of a non-finite number also matches the P2 verifier rather than letting a NaN into the term path.

Boundary. This is Prelude code; the kernel's `Term` type is the input. It does not group `{ ... }` into `Quote` yet, does not change the runtime, and does not add a second evaluator. The projection is a migration seam, not the target representation; P3d deletes it.

Status: landed (P3a). `EvaluatorProgram` is in the Prelude and the boundary is exercised in the example suite. The suite is 1355 example checks (1313 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Changing `Program` to `Term list`, quotation grouping or code values, removing text quotation storage, stack typing, or an AOT path in P3a.

D164 — P3b: the typed control state

Decision. P3b changes `EvaluationState.Program` and the `CallerProgram` fields on `HandlerFrame` and `MethodFrame` to `Term list`. The loop executes `LiteralNumber` by pushing a scalar tensor and `Word` through the existing dispatcher. Every other term is outside the P3b invariant; only `EvaluatorProgram.ofTokens` and the control words write the active program.

Control words still use quotation text in P3b. `quotation`, `if`, `apply`, `define`, and `control` project the term list to tokens with `EvaluatorProgram.toKernelTokens`, run the existing text logic, and convert the result back with `ofTokens`. The same projection feeds the kernel bridge and the token-vector history. `toKernelTokens` is total for `LiteralNumber` and `Word`; any other term is a programmer invariant failure. History remains the seven-field token-vector step until P3c.

Why. The active program is the first control-state field that must stop being token text. Changing its type before quotation values lets the loop stop reparsing numeric tokens and makes the remaining text side channels explicit and removable. A single state migration is safer than mixing it with code values or structural history.

Boundary. P3b is Prelude-only. It does not add quote grouping, execute `Quote` or `Branch`, change the quotation value representation, remove quotation text, touch the kernel term loop, or type the stack. `toKernelTokens` and the token-vector history are temporary and P3d deletes them.

Status: contract recorded. The P3b test batch and the state migration are the next work items. No source or test change yet; counts are unchanged at 1355 example checks (1313 examples, 42 properties), 33 granular cases, and 91 discovered tests.

Non-goals. Code values, structural history, removing text quotations, stack typing, AOT execution, and a second evaluator in P3b.

D166 — P3c: quotation code values and structural history

Decision. A quotation becomes the tagged code value `[QuoteTag; documentId]`, where `QuoteTag` is `-34.0` and `documentId` holds the canonical `term-program` document for the body. `EvaluatorCode.encode` creates the value; `EvaluatorCode.decode` returns its terms and refuses anything that is not a well-formed code value. Executing a `Quote` pushes its code value.

`{ ... }` collects a term tree, including nested `Quote` terms, and pushes a code value. `if`, `apply`, `define`, `control`, `lambda`, closures, actor bodies, condition handlers, method bodies, predicates, guards, and set quotations decode code values. User-word expansion decodes the stored code value. No quotation body is stored as token text.

History becomes structural: the step's program field points at a code value for the resuming term list, and its current-term field points at a one-term code value. Rollback decodes the code value. A legacy token-vector step is readable during P3c and removed in P3d. Pattern and set clause grammars continue to use their surface token parser after decoding the code value to terms.

Why. Text quotation values were the last side channel that made a quotation a string rather than code. A tagged term document makes the value inspectable as code, executable as a splice, and portable across an actor or instance boundary. Structural history stops rollback from reconstructing code by splitting text.

Boundary. P3c is Prelude code plus the `QuoteTag` image tag. It does not remove the kernel bridge projection, the surface clause parser, or the temporary token projection; those are P3d. It does not type the stack or add the verification effect.

Status: landed (P3c). Quotation values are tagged code values, control words decode them, and history stores code values. The suite is 1363 example checks (1321 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Removing all token projections, deleting the surface clause parser, stack typing, the verification effect, macros, and AOT in P3c.

D165 — P3b implemented: the typed control state

Decision. P3b is implemented. `EvaluationState.Program` and the `CallerProgram` fields on `HandlerFrame`, `MethodFrame`, and `CombinationFrame` are `Term list`; the loop executes `LiteralNumber` and `Word`, and any other active term is an invariant failure. `EvaluatorProgram.toKernelTokens` is the temporary total projection for those two shapes, and `EvaluatorProgram.ofTokensOrFail` is the total construction form for a program whose tokens are known to be executable. The control words, the token-vector history, and the kernel bridge project at their boundary and convert back with `ofTokens`, so their text semantics are unchanged.

Why. The active program is the control-state field that must stop being token text first. Changing its type before quotation values stops the loop from reparsing numeric tokens and makes the remaining text side channels explicit and removable. Keeping the migration to one state change means the next steps, tagged quotation values and structural history, can be judged on their own.

Boundary. P3b is Prelude-only. Quotation values, quotation text, and the token-vector history remain text. `KernelState.Program`, `LoweredAbstraction.Program`, and the differential corpus's `Case.Program` remain `string list`; the differential case converts only at its `runInterpreter` boundary. No quote grouping, no `Quote` or `Branch` execution, no stack typing, and no AOT path are added.

Status: implemented. The P3b test batch covers a number/word program through `Evaluator.run`, bookmark rollback preserving the typed program, the kernel bridge projecting typed terms, and `toKernelTokens` refusing a quotation. The suite is 1360 example checks (1318 examples, 42 properties), 33 granular cases, and 91 discovered tests. P3c replaces text quotation values, P3d removes the temporary projection and the token-vector history, and P4 adds stack typing.

Non-goals. Code values, structural history, removing text quotations, stack typing, the AOT path, and a second evaluator in P3b.

D167 — P3d: remove the text side channels

Decision. Remove the P3c migration seams. `EvaluatorCode.decode` accepts only `[QuoteTag; documentId]`; a plain text tensor is not code. `EvaluatorProgram.toTokens` is deleted. `EvaluatorHistory` deletes the token writer and the legacy token-vector reader: its current-term and program fields are code values, and a non-code step is a named history refusal. Test helpers that need a quotation encode a `Term list` with `EvaluatorCode.encode`.

The surface clause parsers for pattern matching and set literals may still project terms to tokens while they keep their grammar. That is a parser boundary, not a stored quotation. The kernel bridge keeps its token projection until the kernel term loop replaces it.

Why. A text fallback keeps a second quotation representation alive and makes "is this code?" depend on which tensor happens to hold text. Structural history already stores terms; keeping the token reader lets rollback reconstruct a program from the old side channel. Removing both is what makes code-as-data the only runtime representation.

Boundary. P3d is Prelude cleanup plus test-helper migration. It does not remove the surface clause grammar, replace the kernel term loop, type the stack, add the verification effect, or implement macros/AOT.

Status: landed (P3d). The text fallback, `toTokens`, and the legacy token history are gone. The suite is 1362 example checks (1320 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Removing the surface clause parser, the kernel term loop, stack typing, the verification effect, macros, and AOT in P3d.

D168 — The F-sharp-equivalent pipeline

Decision. Add a `Pipeline` expression node with a `PipelineStyle` of `LastArgument` or `FirstArgument`. The prose surface parses `value |> stage` left-associatively, with the operator looser than a call. The default and the only style the prose parser currently emits is `LastArgument`. Lowering desugars:

1. `x |> f(a)` to `Call(f, [a; x])`
2. `x |> f` to `Call(f, [x])`
3. `x |> f(a) |> g(b)` to `g(b, f(a, x))`.

`FirstArgument` prepends the value instead: `Call(f, [x; a])`. The styles are language-independent; F# is the compatibility case for `LastArgument` and Elixir for `FirstArgument`.

The `Pipeline` node is preserved in the portable expression document with its style. Lowering desugars it to `Call`; the node keeps the source offset and the style for diagnostics.

Why. The stack is a pipeline model, but it is not F# `|>`. A 1:1 port needs the F# argument position and left associativity explicitly, or imported code silently changes meaning. Elixir's macro inserts the first argument, so copying it would be wrong for F# code. A dedicated node also lets the portable document and diagnostics keep the surface form after elaboration.

Boundary. Partial application and currying are not yet expressible. The accepted form is a fully applied stage, such as `x |> f(a)` an argument-free word stage, such as `x |> f`, also works. A stage that is not callable is the existing lowering refusal. The node does not change the concatenative core or the kernel term runtime.

Status: landed (D168). The parser, portable document, lowering, and test batch are implemented. The suite is 1368 example checks (1326 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Currying, partial application, `||>`, Elixir's first-argument surface, operator sections, and pipeline execution in the kernel term loop.

D169 — Curried pipeline stages

Decision. A pipeline stage may be a word followed by whitespace-separated arguments. `x |> f a` and `x |> f(a)` both elaborate to `Call(f, [a; x])` `x |> f a b` elaborates to `Call(f, [a; b; x])`. The parser reads stage arguments only on the right side of a pipeline. A bare `f a` outside a pipeline is still not accepted.

Why. F# pipeline code commonly writes `xs |> List.map f`, where the stage is a partial application completed by the piped value. Without stage arguments, a 1:1 port would require rewriting every such stage to `List.map(f)`, which is not the same surface. Reading the stage arguments and appending the value gives the F# meaning without adding general currying.

Boundary. This is a pipeline grammar rule, not a general application rule. It does not create first-class partial functions, does not change `Call`/`Lambda` semantics outside a pipeline, and does not add `||>`. Stage arguments stop at a pipe, comma, period, closing parenthesis, `when`, `then`, `otherwise`, or `close`.

Status: landed (D169). The parser and test additions are implemented. The suite is 1371 example checks (1329 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. General currying, partial application outside a pipeline, `||>`, tuple pipes, operator sections, and kernel changes.

D170 — Explicit partial application with holes

Decision. A call argument may be `_`. A call containing at least one hole becomes a `Partial` expression; supplied arguments are evaluated once at creation, and holes bind when the value is applied. `subtract(_ 10)` is a one-argument closure; `f(_ a _)` is a two-hole closure. A call with no holes remains `Call`.

The parser recognizes `_` only in call arguments. The `Partial` node is preserved in the portable expression document. Lowering generates a lambda whose parameters are the holes and whose body calls the target, with the supplied arguments bound in the enclosing frame so they are evaluated once.

Why. This gives explicit partial application without inventing automatic currying or an arity inference rule. The number of holes is the number of arguments the partial expects, so the transform is sound without the P4 signature table. It also reads better than a point-free `f a` in prose: the hole marks exactly what is missing.

Boundary. `_` outside call arguments is still a word; it is not a general wildcard in expressions. A bare `f a` outside a pipeline is still refused. The target must be a word or lambda in the current lowered subset. General currying, `||>`, tuple pipes, and automatic argument insertion remain out.

Status: landed (D170). The parser, portable document, lowering, and test batch are implemented. The suite is 1374 example checks (1332 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Automatic currying, first-class word values, tuple pipes, named-hole syntax beyond _, and kernel changes.

D171 — The reverse pipe

Decision. Support `<|` as `ReversePipeline` for F# package compatibility. It is the reverse of `|>`: `f <| x` means `f(x)` `f a <| x` means `f(a, x)` and it is right-associative, so `f <| g <| x` means `f(g(x))`. The parser gives it the lowest precedence. The node is preserved in the portable expression document, and lowering appends the value to the left call.

Why. Some F# packages use `<|` heavily. A 1:1 port must accept it even though it is not the preferred house style. It also gives the parser a single place to explain the direction, rather than treating `<|` as two unrelated tokens.

Boundary. `<|` is discouraged in new Scale code; `|>` or a direct call is preferred. It does not introduce automatic currying, a second argument-position style, or Elixir semantics. The left side is a call or a word; a target that is not callable is the normal lowering refusal.

Status: landed (D171). The parser, portable document, lowering, and test batch are implemented. The suite is 1377 example checks (1335 examples, 42 properties), 33 granular cases (11 snapshots, 13 integrations, 9 writer checks), and 91 discovered tests.

Non-goals. Automatic currying, first-class word values, tuple pipes, and treating `<|` as a preferred pipeline.

D172 — Function values, currying, and recursion classification

Decision. Rename `let` to `logic`. Use whitespace application: `logic f x = ... close`, `function x -> ... close`. Use `close` as the body terminator for now; indentation may replace it later.

Functions are first-class and typed. A function value carries arity, parameter and result types, effects, closure state, and the number of supplied leading arguments. Application is left-associative; fewer arguments return a function, complete arguments run the body, extra arguments apply the result. Nested unary closures implement this; the signature/effect table is the authority.

`PartialApplication` is a call with missing arguments. `ArgumentHole` is an omitted call argument; `ParameterHole` is reserved for a future declaration parameter hole and is not implemented. Holes are call-argument only.

Functions are single-result. Multi-result words remain concatenative stack effects and cannot be curried; F# and Elixir both return one term, and guessing a tuple from the stack is rejected.

Recursion is explicit at the Scale definition site: `logic f x = ...` is an ordinary definition, `cycle f x = ...` is self-recursive, and `cycle f x = ... and g y = ...` is mutual. The plain name is the recursive reference; there is no `recurse` word. A `logic` self-call is refused. Value recursion is a separate initialization check.

When calling F# packages, the package boundary has two tiers that emit one classification record. Tier A reads the F# compiler classification (`let`, `let rec`, `let rec … and …`) automatically; no manifest. Tier B uses a version-controlled package manifest for per-binding overrides, but still runs F# read-off and compares. Agreement is silent; disagreement is diagnostic; disagreement with a reason is accepted and tracked. The record has no source-tier field. Package modes are pure automatic, mixed, and strict explicit. Overrides without reasons are build errors.

Why. The F# Prelude is function-heavy; application and currying are prerequisites for its higher-order code. Typed currying plus explicit holes serves prose, F#, and Elixir without an implicit first-argument or last-argument guess. Reading F# binding classification from FCS preserves the package's real recursion semantics; a manifest is only a checked override, not a second source of truth.

Boundary. This is the design; implementation and the signature table are next. It does not add automatic currying to every word, parameter holes, tuple multi-results, or a package loader. Package interop remains a separate host lane.

Status: design recorded. `docs/function-values.typ` is the detailed contract. No source or test change yet.

D173 — P4a first-order type constraint engine

Decision. P4 type inference is a host-side, first-order type constraint and unification engine, separate from the image-resident term unifier. It owns type variables, primitives, sorts, functions with latent effects, schemes, constraints, substitutions, an occurs-check, generalization, instantiation, and named diagnostics. It does not reuse D15/D59′s term unifier and does not implement higher-order type unification.

Why. Types are static host data; terms are runtime image data. The term unifier binds image variables and implements the Miller-pattern fragment for Scale terms. P4 must resolve schemes and word signatures at elaboration time and must remain correct without an image. Conflating the two would make type inference depend on runtime substitution and would erase the distinction between a type variable and a term variable.

Boundary. P4a is the core only: `components/typing/type-inference.fs`. It is not yet wired into the parser, lowering, or the signature table; that is P4b/P4c. Higher-order type unification is reserved for a future type-level lambda or higher-kinded feature, not ordinary higher-order programming. Effect rows and subtype constraints are future work; P4a uses exact effect sets.

Status: applied. The core and nine regression checks are in place. P4b adds expression inference and eight checks; P4c adds located diagnostics and five checks; the internal primitive registry adds four checks; the located prose path adds two; child call ranges add three. The suite is 1408 checks (1366 examples, 42 properties). `docs/type-inference.typ` is the contract.

Non-goals. Full Huet unification, dependent types, type-level lambdas, implicit arguments, runtime inference from observed values, and trusting inferred hints as portable semantics.

D174 — One built-in word index, checked against the dispatcher

Decision. `PreludeSignatures.classification` is the single list of built-in words. Each entry is `Expression` (a P4 callable signature) or `NotInferred` (a word the expression elaborator does not type, with the reason). `primitiveTable` is derived from that list rather than maintained beside it. `tools/consistency-check.fsx` (`just docs-check`) parses the explicit string cases in `Evaluator.executeBuiltinWord` and fails if the two sets differ, if a word is classified twice, or if the dispatcher gains a word without a classification.

Why. The dispatcher and the P4 signature table were two hand-maintained lists. The dispatcher named roughly 150 words while the signature table named seven, so a word could execute and silently remain untypeable, and a new word could enter either list without the other. The execution authority and the inference-coverage authority remain distinct, but the gate now keeps them equal.

Boundary. The list classifies words; it does not unify the interpreter and the image VM, and it does not make every word typeable. A word is `Expression` only when the expression elaborator can call it with a real signature; the rest are `NotInferred` with a reason.

Status: implemented. No runtime behavior change. The suite is unchanged: 1408 example checks (1366 examples, 42 properties), 33 granular cases, 91 discovered tests. The dispatcher's 137 explicit word cases match the 137 classification entries.

D175 — The ABI is reset to 1 with independent handles

Decision. The ABI is reset to version 1 because no host has used version 2. `scale_init` now allocates an independent handle; the kernel keeps a thread-safe registry of per-handle callback tables and validated images. Multiple handles coexist in one process, each `scale_set_callbacks` updates only its own handle, and `scale_free` removes only that handle. A stale handle is invalid and freeing it is a no-op. The handle is an opaque integer identity, not a host pointer.

Why. The previous implementation kept one callback pointer, one handle, and one loaded image in module-level mutable state, so a second `scale_init` silently replaced the first. The C ABI looked like a handle API but was process-global. Resetting the version is safe because no adapter has shipped and the version scheme says equality is a deployment rule, not a compatibility negotiation. Independent handles remove the last hidden process-global constraint from the boundary.

Boundary. The registry is thread-safe for create, look-up, replace, and remove. It does not serialize calls that share a handle: the host must serialize calls if its callback table is not thread-safe. The header states that rule, the per-handle lifetime, and the stale-handle behavior.

Status: applied. The header, `abi.fs`, the C adapter header and README, the conformance fake kernel, and the tests all target version 1. A regression checks that two live handles coexist. The suite is 1409 checks (1367 examples, 42 properties), 33 granular cases, 91 discovered tests.

D176 — The test suite is a separate JIT-only deployable

Decision. The product base no longer references `development/scale`. The suite lives in `projects/scale-tests`, an `Exe` deployable that references `development/scale` and the CLI component. The product binary handles the product modes and refuses the suite modes by name with exit code 2. The NativeAOT artifact is the product binary; the suite is JIT-only and is not published. `just test`, `just test-all`, `just test-granular`, and `just check` target `scale-tests`; `just aot-test` and `just release` publish and smoke the product.

Why. The product artifact was coupled to Expecto, Hedgehog, Verify, and the 13,000-line suite because the development library was a project reference (D23). Splitting makes the product smaller, removes the test packages from the publish graph, and keeps the suite as the acceptance gate in CI before a release. The one cost is that the AOT binary can no longer run `--tests`; the suite is already JIT-only in practice because granular and Verify are not AOT-compatible.

Boundary. `scale-tests` is a JIT test deployable, not a language artifact. It carries the suite's content files (wire vectors, term vectors, demo fixtures, witnesses, snapshots). The product keeps its product content (demo fixtures and witnesses) and its product modes.

Status: applied. The product session loads without `development/scale`; `scale-tests` runs the 1409 checks (1367 examples, 42 properties), 33 granular cases, and 91 discovered tests. `just poly-check` recognizes the third project; `just docs-check` checks its compile items.

D177 — Prose logic bindings and bound closure calls

Decision. The prose surface accepts block-scoped definitions: `logic name = value.` and `logic name p1 p2 = body.`. The definition scopes over the sentences that follow. A value definition parses to `Let(name, value, rest)`; a function definition parses to `Let(name, Lambda(params, body), rest)`. The binding is non-recursive. The keyword is `logic`; `=` is a token; `let` and `in` are not keywords. A call `name(args)` where `name` is lexically bound lowers to `PushSlot slot; CallClosure`; a non-closure bound value is a named lowered failure. This supersedes D122′s `CallTargetBound` refusal for bound calls.

Why. The F# Prelude is let-heavy and uses named functions; the surface needs F#-style definitions, not a separate `in` expression. Reusing `Let`/`Lambda` keeps the IR unchanged; the only semantic addition is the bound-name call path, which the existing closure machinery already supports.

Boundary. The body is one expression; the rest of the sentence sequence is the scope. `function`, `close`-terminated bodies, `in`, automatic currying, and recursion remain D172 design. A missing `=` or a missing name is `ExpectedEquals` or `ExpectedBindingName`; a definition with no following sentence is `EmptyProgram`. A logic definition is a sentence, not a general expression, in this slice.

Status: applied. Three regression checks cover a value, a two-parameter function called by name, and the missing-`=` refusal. The suite is 1412 checks (1370 examples, 42 properties), 36 prose-surface checks, 33 granular cases, and 91 discovered tests.

D178 — Event history is opt-in and uses a persistent step chain

Decision. The always-on event stream is replaced by an opt-in recorder: normal evaluation records no steps, `history-on` enables recording, and `history-off` disables it. `bookmark` records its own step explicitly. When enabled, steps form a persistent linked chain through a final `previous-step` field, so appending is O(1); `history` and rollback walk the chain. When recording is on, `history` excludes its own step; when it is off, `history` reports the steps that were recorded (usually a bookmark), and rollback of an absent step is `InvalidHistoryStep`.

Why. The previous recorder wrote a step for every token: the token, the whole stack, and the whole remaining program, then copied the entire history vector on append. Evaluation was $O(n^2)$ in program length and the store grew unbounded. Small tests could not see the ceiling. The event-stream data model stays; the recording policy and the chain representation changed.

Boundary. This deliberately reverses D29′s always-on behavior. A future compact encoding of the program suffix is a separate decision; the linked chain fixes the append copy but does not by itself make the encoded step constant-size.

Status: applied. Step tensors are now eight fields (the final field is the previous step, −1 for none); `history-on`/`history-off` opt in and out, and the default is off. D178 adds one regression check; the suite is 1413 checks (1371 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D179 — Split the evaluator state from the scheduler (staged)

Decision. The target model is `EvaluationState` without `SchedulerState`; a `RuntimeState` carries the evaluator state and the scheduler beside it. An `Actor` holds only the evaluator state, and the scheduler owns the actor map. Stage 1 isolates the one canonical root construction and requires every actor switch to re-thread the store and scheduler through one helper; stages 2 and 3 move fields and callers. The target is a focused actor record with no embedded scheduler.

Why. An `Actor` currently contains a full `EvaluationState`, which contains the `SchedulerState`, which contains every actor. The live state is canonical only because `run` threads it; a snapshot can carry a stale scheduler, and `initial` builds a dummy scheduler to break the cycle. Tests exercise one scheduler shape and cannot see the hazard.

Boundary. Stage 1 does not change the record shape or the public entry points; it creates the seam and the regression checks. Stages 2 and 3 are separate work items with their own suite runs and docs updates.

Status: stage 1 applied. `Scheduler.initializeRoot` is now the single construction of the canonical root scheduler; `Evaluator.initial` no longer builds the actor/handle/mailbox snapshot by hand. Stages 2 and 3 (moving the field and callers, then removing the dummy scheduler) remain. The suite is 1413 checks (1371 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D180 — Kernel changes require a NativeAOT build and C conformance

Decision. A kernel change is verified with `just kernel-aot linux-x64`: publish the kernel as a NativeAOT shared library (`Scale.Kernel.so`) and run the C conformance harness against it. This is the one NativeAOT step allowed in alpha; the product AOT publish remains release-only. CI has a path-filtered Kernel AOT workflow for `components/kernel/**` and `components/adapter-c/**`; the release workflow runs the same gate before it publishes. The kernel project explicitly references `FSharp.Core` so the ILC publish graph includes it, and the conformance harness accepts the prelude dispatch with no result buffers.

Why. The kernel is the binary boundary; JIT tests cannot prove that the exported functions survive NativeAOT, static initialization, trimming, or the C callback layout. The first real run found two latent bugs: the kernel publish graph omitted `FSharp.Core`, and the C harness rejected the prelude notification because `scale_init` passes null result buffers. Both are fixed.

Boundary. The gate proves the kernel shared library and the C adapter. It does not publish the product; that remains `just publish`/`just release`. The NativeAOT build and the C harness are Linux-gated in CI; other RIDs publish the kernel but skip the harness.

Status: applied. The kernel AOT publish succeeds and the C conformance harness passes all eleven checks. The suite is 1412 checks (1370 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D181 — Nu integration and Gaia-on-ImSim plan

Decision. The target is `bases/nu-game` for ImSim games, `bases/gaia-im-sim` for a clean-room editor, `projects/game` for the AOT game, and `projects/gaia` for the JIT desktop editor. `components/nu-host` remains the Scale bridge. The Nu engine remains the external node `components/nu-engine`, and the original `Nu.Gaia` remains there as a read-only behavioral oracle; the new editor does not reference the `Nu.Gaia` assembly. The plan is `docs/nu-gaia-imsim.typ`, structured as Phase 0 build lane through Phase 6 Scale-image bridge.

Boundary. The current Polylith rule permits only `nu-host` to reference the external node. ImSim application code inherits Nu dispatcher types, so the plan prefers an explicit host-app exception for exactly `bases/nu-game` and `bases/gaia-im-sim`. That exception requires a `tools/polylith-check.fsx` change and tests; this record does not apply it. If the exception is rejected, all ImSim app code must live inside `nu-host`.

License. Nu and `Nu.Gaia` are under the Nu Game Engine Noncommercial License. The engine stays external with its notices. The new editor is a clean-room implementation against ImSim, not a copy of Gaia. Whether the editor may be distributed under Scale's licenses is an owner/legal decision; no editor distribution until it is recorded. This plan does not relicense or copy Gaia.

Execution. The Nu lane is opt-in (`just nu-build`, `just nu-smoke`) and does not enter the default fast gate. The editor is JIT; the game target is AOT; the kernel seam remains the C ABI. The host graph must pin one F# Core version, because Nu pins 10.1.204 and Scale uses 10.1.401.

Status: plan recorded; no implementation. No Nu base, project, host reference, workspace edge, or build recipe is implemented by this record. The suite is 1413 checks (1371 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D182 — Isolate Scale-only work in a sparse jj workspace

Decision. The repository is colocated with Jujutsu 0.46.0. The `scale-code` workspace at `/home/shalok/Projects/Scale-code` is created with `jj workspace add --name scale-code -r @ --sparse-patterns empty` and an explicit sparse set of Scale-owned paths. It excludes `components/nu-engine`. The main `Scale` workspace at `/home/shalok/Projects/Scale` remains the full checkout. Scale-only Serena/F# work targets `scale-code`; Nu work targets the main workspace. The usage contract is `docs/jj-workspaces.typ`.

Why. `fsautocomplete` runs with `--project-graph-enabled` and `workspacePath` set to the repository root. In the full checkout the graph contains 37 projects, including 21 vendored Nu projects; the Nu graph build fails on missing `Nu.Math.dll` and `Nu.dll`, and the Scale projects are then absent from `LoadedProjects`. `.ignore` and Serena's `ignored_paths` affect search, not the F# server's project discovery. Removing Nu from the LSP root fixes the failure without changing the code or the Polylith graph.

Boundary. A jj workspace is a working-copy arrangement over the same repository history, not a Polylith node and not a second build graph. `components/nu-engine` remains the external, noncommercial node. Sparse membership changes what is present, not what is committed. Adding a new top-level Scale path requires a `jj sparse set --add`; `bases` and `projects` already cover future host bases and projects. Move work between workspaces with `jj rebase` or bookmarks, never by copying files.

Status: applied. Created on 2026-11-04 with the Scale-only sparse set in `Scale-code`; `components/nu-engine` is absent; Serena symbol lookup on `components/prelude/scheduler.fs` succeeds there. `just docs-check` passes in both workspaces; the external Nu package-policy check is conditional on the Nu project being present. The suite is unchanged at 1413 checks (1371 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D183 — Host words and the first shell effects

Decision. The prose shell's `run` and `capture` commands are host words, not Prelude primitives. `EvaluationState` carries an optional `HostWordAdapter`; when the dispatcher does not handle a token, it offers the token to the adapter before reporting an unknown word. Lowering maps a shell call to the host-word token `$shell.<name>#<arity>`; the adapter pops exactly that many text values. `run` forwards the child's standard output and standard error and pushes the exit status as a number. `capture` forwards standard error and pushes standard output as text. Both record the last command status; the script's exit code is that status, or zero when the script contains no command. A new `components/shell-host` owns the OS effects with `System.Diagnostics.Process`, `UseShellExecute`

= false, an explicit argument list, and redirected standard output and error. The base attaches the adapter when it runs a .prose file.

Why. The command surface is variadic, while the lowered instruction set has no arity-carrying call and the image schema is frozen at version 1. A host-word token carries the arity without a kernel or image-schema change, keeps process authority out of the kernel and Prelude, and leaves the interpreter path untouched when no host is attached. The adapter seam is also where a later capability sandbox plugs in.

Non-goals. The with connective, pipelines, file redirection, variables, stdin values, the REPL, and the capability sandbox do not land in this record. --prose-typecheck does not infer host words yet; host words have no primary signature.

Status: implemented. run and capture execute one command through the host-word adapter; the script's exit code is the last command status. The suite covers lowering, the adapter contract, and the real process host. The suite is 1432 checks (1390 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D184 — Books and spells are packages

Decision. A **book** is a PackageAst.PackageDocument built from a .book file; a **spell** is a book whose exports include main. The classification is derived, never declared. The .book source contains logic definitions only: a definition is a package unit whose root is its body or its lambda, and every definition is an export. The file stem is the package name, the version is 1, and the source hash is the SHA-256 of the file. Prose no longer accepts logic; it refuses it as LogicOutsideBook and casts books instead. This supersedes D177 for the prose surface.

A prose cast is a host word: lowering emits $book.cast#<arity>, and the new components/book-host loads the package, binds the definitions in a fresh frame, and calls the requested closure (or returns a value definition's bound value). The book runs with the caller's store, vocabulary, host-word adapters, and base directory; the result returns to the caller's stack. A relative path resolves against the calling .prose file's directory, or the book's directory for a nested cast. HostWordAdapter becomes a list so the shell and book adapters compose in one state; this amends D183's optional single adapter.

The CLI adds --book <path> (classify, list exports, run main when a spell) and --cast <path> <word> [args...] (call a named export). Stage A keeps main zero-argument and does not import other books; Stage B adds spell arguments and imports.

Why. PackageAst already carries units, exports, imports, capabilities, kinds, predicates, and a source hash. Reusing it gives books one module format, one loader, and a direct path to hot reload and capabilities. A book/spell declaration would duplicate the classification that the presence of main already encodes; deriving it keeps the child's mental model to one rule.

Non-goals. Recursion, mutual references, spell command-line arguments, cross-book imports, per-definition source hashes, spell executables, and capability grants do not land in Stage A.

Status: implemented in Stage A. The .book parser, package classification, --book, --cast, the prose cast host word, and the LogicOutsideBook refusal are covered by the suite. Details and the user-facing contract are in docs/books-and-spells.typ. The suite is 1455 checks (1413 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D185 — Abstractions, logic, and type retrieval

Decision. The canonical vocabulary changes in the documentation. An IR is an **abstraction**: an implementation of a **logic**. The canonical names are `SymbolicAbstraction` (current `TermIr`), `LoweredAbstraction` (current `LoweredInstructions` plus the runner), `ExpressionLogic` (current `ExpressionAst`), `PatternLogic` (current `PatternAst`), `PackageLogic` (current `PackageAst`), and `TypeRetrieval` (current `TypeInference`). `TypeRetrieval` is not a type checker; checking is a later layer.

Everything is an implementation of some logic: tests implement test logic, expression trees implement expression logic, patterns implement pattern logic, decision tables implement decision logic, SMT obligations implement formal logic, the kernel implements the lowered abstraction, and the Prelude implements the symbolic abstraction. The project does not rename every implementation to "logic"; the canonical names are the teaching frame and the documentation vocabulary.

The canonical verification target is one `verification` node with `formal/` (Z3, the refiner, diagnostics, and decision tables), `tests/` (the example, property, granular, and differential suites), and `typing/` (type checking). The current code remains in `components/verification`, `development/scale`, and `components/typing`; no move is made in this record.

Decision tables. A table is an abstraction of decision logic. Its source is the table document (`table <name>`, optional `policy:`, Basic English transition rows, `end`); the host supplies the domain's field getters and the outcome body. It is not inferred from the prose specification in this slice. The demo's table text is a hand-written example; the product path is the parsed document.

Why. The names should say what the thing is. "IR" hides that an intermediate representation is an implementation of a logic; "Ast" hides that an expression tree is logic too. "Type retrieval" says what the elaborator does and leaves "type checking" free for the later layer that rejects ill-typed programs.

Scope. This is a documentation reframing. No code symbol or file is renamed and no test or verification source moves in this record. A later rename changes the code names to the canonical ones after the usual tests and specification update.

Status: applied in the docs. `docs/irs.typ` is the canonical index and the graph; `docs/decision-models.typ` records the table source; `docs/type-inference.typ` records the `TypeRetrieval` name; `docs/now.typ` and `docs/README.typ` point at the canonical vocabulary. Code names are unchanged.

D186 — Basic English is the historical name for the Prose sentence grammar

Decision. The current documents and code comments use **Prose sentence grammar** for the controlled sentence surface historically called Scale Basic English. The two documents move to the Prose grouping: `docs/scale-basic-english.typ` becomes `docs/prose-grammar.typ`, and `docs/basic-english.typ` becomes `docs/prose-foundations.typ` (the historical foundations draft). Ogden's Basic English keeps its own name as the earlier inspiration, not as the current language.

The implementation remains `components/core/tokenizer.fs` and `components/core/parser.fs`, with the vocabulary in `components/core`. It is a distinct sentence grammar from the expression front-end in `components/core/prose-parser.fs`. It is **not** merged into `prose-parser.fs` in this pass: the sentence grammar serves specification sentences and decision-table rows, while the expression parser serves prose programs, books, and casts. A merge would either weaken the sentence grammar or duplicate the expression lowering.

Why. The old term describes a historical inspiration, not the current language. The current name says what the surface is: prose sentences that compile into the core. The two grammars remain separate because they have different jobs and different callers; sharing a parser would be a false unification.

Scope. Documentation rename and file move only. No code symbol, module, diagnostic string, or test is renamed; the code still uses "Basic English" in comments and diagnostics in Stage A. The canonical mapping is in `docs/irs.typ` and the moved docs.

Status: applied in the docs. The two files moved, `docs/README.typ` links the new paths, and the current docs use Prose sentence grammar. The suite is 1455 checks (1413 examples, 42 properties), 33 granular cases, and 91 discovered tests; no code changed.

D187 — The `with` connective

Decision. `with` is a reserved keyword. A sentence-level command is `run` or `capture` followed by a program expression and zero or more `with` expressions: `run "echo" with "hello".` and `capture "echo" with "hello".`. It parses to the same `Call(Word "run" | Word "capture", program :: arguments)` as the parenthesized spelling, so lowering emits the same `$shell.<name>#<arity>` host word. There is no second semantics.

Why. The target surface is prose, not a function-call language. The call spelling stays as the interim form; both must share one tree and one lowering so the surface can grow without a semantic fork.

Boundary. The command form is sentence-level only. It does not add pipelines, file redirection, variables, or a REPL; Stage 2 and later own those. `with` is reserved so it cannot become a name. `run "echo".` is the one-argument form; `run.` remains a bare word.

Status: implemented. The parser accepts both spellings, the lowering is identical, and the process host captures the target form. The suite is 1455 checks (1413 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D188 — Book call-order validation

Decision. A book's definitions are validated at load time, before the package is built. A call or a word reference to the definition itself or to a later definition is a named refusal: `recursive or forward book word <name> in <definition>; a book definition may call only earlier definitions`. A call to an earlier definition is allowed.

Why. The ordered, non-recursive book contract (D184) was not enforced. A self, forward, or mutual call lowered to a bare `CallWord`; at runtime the book frame contained the definition, so the evaluator pushed the closure as a value and the cast exited zero with no result. A silent success is worse than a refusal. The validation makes the existing contract honest and gives the recursion work a safe boundary.

Boundary. This is the first slice of the recursion plan, not the recursion implementation. The durable callable/call semantics (backlog item R0) will distinguish a reference from a call and replace the refusal with a real recursive call or the explicit `define` markers. The plan is `docs/recursion.typ`; the dependency graph is `docs/backlog-graph.typ`.

Status: implemented. The loader walks each definition body and refuses self, forward, and mutual calls; an earlier-definition call still works. The suite is 1455 checks (1413 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D189 — The strict callable contract in the lowered path

Decision. The lowered/compiled path uses `Evaluator.executeWordStrict`. A `CallWord` resolves control words, word metaobjects, primitives, and ordinary words; if it reaches a runtime lexical binding, it is `DynamicCall`: "a lowered word call cannot resolve a runtime lexical binding: ". The interpreter keeps its dynamic `let`/`define` behavior. The book runner calls a defined function by pushing the bound closure and using `Apply`, not by a `CallWord` of a lexical name.

Why. `CallWord` served both a word call and a name reference. A call to a name outside the static environment lowered to `CallWord`; if the frame held that name at runtime, `executeWord` pushed the value. A function calling itself or a later definition returned its closure with exit code zero. D188 refuses that shape in books; this contract also protects the lowered path when a dynamic binding reaches a word call, and it makes the book target call explicit.

Boundary. This is R0′s first slice, not the unified `Callable`/`Invoke` model. The interpreter is unchanged; dynamic `let`/`define` still work there. The durable model will replace the static/dynamic split with one resolver and one invocation operation. Recursion markers and automatic book recursion (R1/R2) build on this boundary.

Status: implemented. A normal word call through the lowered bridge still works; a lowered call that reaches a runtime lexical binding is a named refusal; earlier book calls still work. The suite is 1455 checks (1413 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D190 — The lowered instruction set is `LoweredInstructions`

Decision. The module and file that define the lowered instruction set are `LoweredInstructions` (`components/kernel/lowered-instructions.fs`). The retired `LoweredIr` name is gone. The execution abstraction remains `LoweredAbstraction`; it opens the instruction set. New code uses `LoweredAbstraction` for execution and `LoweredInstructions` for the instructions. No behavior changes.

Why. D185 makes an IR an abstraction and defines the canonical name `LoweredAbstraction`. `LoweredIr` kept the retired "IR" term in new code and made the benchmark add another reference to it. The instruction set is the part of the abstraction that the VM executes, so `LoweredInstructions` says what it is. This is the scheduled code rename D185 deferred.

Boundary. Only the module and file name change: types, constructors, functions, and semantics are unchanged. `TermIr`, `ExpressionAst`, `PatternAst`, `PackageAst`, and `TypeInference` keep their code names; D185′s canonical documentation vocabulary is unchanged.

Status: implemented. All code, tests, generated architecture data, and current docs use `LoweredInstructions`. The suite is 1456 checks (1414 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D191 — One execution-root registry

Decision. The execution roots are enumerated once by `RootKind` and `rootKinds` in `components/core/runtime-types.fs`. `getRoot` reads one and `setRoot` reattaches one. The vocabulary root is mandatory; removing it is the named refusal `the vocabulary root cannot be absent`. `ChannelsRoot` is in the registry even though it lives on `SchedulerState`. Current and saved frames are control state, not registry roots.

Why. R5 exists because a new root could be added to the store and silently omitted from a persistence or reclamation call site. The registry makes the omission a coverage-test failure:

reflection compares every `RootKind` case with `rootKinds`, so adding a case without updating the list fails. Persistence still serializes the store only, so the registry does not make execution state durable; it makes reattachment explicit and complete.

Boundary. The registry is an enumeration and a get/set pair, not a new serialized record. The store's image format, the no-execution-state rule, and the current/saved-frame exclusions are unchanged. Adding a user-facing root accessor is out of scope.

Status: implemented. The registry, the set/get round-trip, and the store round-trip are in the suite. The suite is 1488 checks (1417 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D192 — Image and package schema 2 carry unit groups

Decision. The image and package documents move to schema version 2 and `SCALE_ABI_VERSION` moves to 2. A unit carries explicit `Arity` and `Captures`; a `UnitGroup` carries a name, its unit names, `InitOrder`, and one `BindingClassification` per unit. The image and package readers require the new fields and refuse a schema-1 document by name. The group runtime, book recursion, and `define` markers are later slices; this slice makes only the schema and version contract.

Why. Recursion is a property of a definition group, not a special opcode. The executable records and the package records must carry the group, arity, capture slots, initialization order, and classification before any runtime or book slice can use them. Changing the schema without a version bump would make an old image look like a new one and fail in a reader, not at the deployment boundary.

Boundary. The schema records are data; the current lowered VM still does not resolve groups, and D188′s book refusal remains until Slice 3. The current and saved frames are not serialized. The package groups are empty for books in this slice; the group runtime and automatic book grouping follow. A version mismatch is refused, never negotiated.

Status: implemented. Units and groups round-trip in both schemas; a schema-1 image and a schema-1 package are refused; ABI and image versions are both 2. The suite is 1488 checks (1424 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D193 — Group call frames and the call/closure split

Decision. A `CallWord` to an image unit resolves through the unit's group and pushes an argument/capture frame. The frame is the unit's popped arguments followed by its declared captures; `PushCapture` reads it. A closure invocation (`CallClosure`) reuses the frame the closure already established and does not apply the target unit's arity a second time. Function groups are resolved and called; the depth bound remains `MaxCallDepth` (256), and a short call is `UnitArityMismatch`.

Why. Slice 1 made arity and captures data; without a frame a unit with arguments cannot run, and without a separate closure callback the VM would either ignore arity or double-frame a closure. The split preserves the callable contract: a word call resolves to a unit or word, while a closure runs the frame it owns.

Boundary. Value-unit initialization and book recursion remain Slice 3; the loader validates `InitOrder`, and Slice 2 uses group membership for resolution. `EntryUnitArity` refuses an exported unit that needs arguments, because `scale_run` has none. Captures are indices into the caller's current frame; a missing capture is `UnitCaptureMissing`.

Status: implemented. A grouped unit call with one argument returns 42; an inconsistent `InitOrder` and a short call are named refusals; closure and dynamic-word calls still work. The suite is 1488 checks (1424 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D194 — Automatic book recursion and optional markers

Decision. A book is one recursive unit group. The loader lowers each definition to an image unit with explicit arity and no lexical captures, puts all definitions in one group, and runs the requested definition through the group runtime. Self, forward, and mutual calls are ordinary calls; D188′s refusal is retired. The markers are `logic` for an ordinary definition and `cycle` for a recursive definition; `and` groups cycle definitions F#-style. A `logic` body that calls its own name is refused, and a `cycle` definition that calls no book word is refused.

Why. D188 made the old non-recursive contract honest, but it prevented the feature the book is for. The group runtime now executes the call shape, so the book surface can be recursive through `cycle`. The markers are a check on intent, not a second execution model.

Boundary. `logic`/`cycle` definition words are Slice 4; the F# package classification is Slice 5. A book value definition is a unit and is run when called; the loader no longer eagerly runs the whole definition chain. Cross-book imports remain later work.

Status: implemented. A forward call returns 42; a runaway book is depth-bounded; marker disagreements are refused. The suite is 1488 checks (1430 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D195 — Logic and cycle definitions

Decision. The core definition words are `logic` and `cycle`. They read a name and a quotation body from the program and store the body in the user-word table. `logic name { body }` is non-recursive: its own name is not in scope in the body, and a self-call is a named refusal. `cycle name { body }` is recursive and uses the plain name. `cycle first { body } and second { body }` stores a mutual group, with every group name in scope in every body. There is no `recurse` word. The old stack form `body "name" define` remains supported for now.

Why. The surface reads as English prose: `logic` defines, `cycle` declares recursion, and `and` joins a group the way F# does. The plain word name is the reference, so there is no separate self-reference spelling and no hyphenated marker. The marker is checked when the definition is stored, so a bad definition is a named diagnostic before the word can run.

Boundary. The definition words are core-surface parsing words; the prose surface defines books with the same `logic`, `cycle`, and `and` markers. A `cycle` group is stored atomically and is not a new image record beyond the existing unit group. F# classification remains Slice 5.

Status: implemented. A `logic` word returns 6; a `cycle` factorial returns 120; a two-word `cycle` group returns 2; a `logic` self-call is refused. The suite is 1530 checks (1488 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D196 — F# abstraction and manifest classification

Decision. Add a versioned F# subset abstraction and a checked classification path. `FSharpAbstraction` carries modules, bindings, parameters, expression bodies, plain-name calls, source ranges, and the F# recursion kind read from the source layer. A manifest may declare `automatic`, `mixed`, or `strict` mode and per-binding overrides; an override must agree with the F# kind or carry a `DeclaredReason`. The classifier emits `BindingClassification`, tracks accepted overrides, maps F# diagnostics to Scale diagnostics, and emits `(name, kind, version)` metadata.

The abstraction lowers to the same `PackageUnitGroup`/`PackageUnit` substrate as books and `logic`/`cycle` words.

Why. The package boundary must not infer F#'s recursion kind from a call edge, and a manifest must not become an unchecked second source of truth. The abstraction keeps the source classification explicit; the manifest is only a checked override, and every override is auditable. The runtime consumes the classification and groups, never FCS.

Boundary. FCS remains a build-time dependency; the product and AOT runtime do not link it. The core abstraction, classifier, FCS read-off adapter, and the supported expression-body translation are implemented. An F# body that uses a form outside the supported subset is a named refusal, never a guessed translation. Diagnostics preserve their Scale source range; `Unknown` is never a pass.

Status: implemented. The abstraction, manifest parser, automatic and strict classification, explained overrides, diagnostic mapping, metadata, and package lowering are tested. The suite is 1530 checks (1488 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D197 — Classification metadata wire

Decision. Add `ClassificationWire` in `components/ipc` as the closed metadata payload between a classifier and a caller. It serializes `(name, kind, version)` per binding with schema version `1`; `version` is the classification metadata version, not the package or ABI version. Decode rejects a missing or unknown field, an unknown kind, a duplicate name, a version skew, and any `body` field. Encode never writes a body.

Why. The caller needs the recursion kind to accept or refuse a binding, but it must never receive a function body. A closed payload makes the no-body rule executable rather than a convention. The metadata version has its own refusal path, so a classification schema change cannot masquerade as a package change.

Boundary. This is a metadata payload, not a new actor envelope and not a second call path. Invocation still uses the fixed call contract. The payload does not carry a package version or an ABI version; those remain separate versioned artifacts.

Status: implemented. A metadata round-trip, a refused body, a refused version skew, and the absence of an FCS reference in the runtime abstraction are tested. The suite is 1530 checks (1488 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D198 — Book package contract

Decision. Loading a book is lazy: the source is parsed and packaged, but no definition is evaluated until called. Each package unit carries its own stable source hash and the book words it calls as dependencies. A call checks the definition's arity before lowering. A spell's `main` must have arity zero; the CLI supplies no arguments. Package version is `1`, the file stem is the package name, and a relative path resolves from the book's directory.

Why. A package is a description of units, not a side-effecting import. Per-unit hashes and dependencies make a unit identifiable and keep hot reload from treating the whole book as one opaque artifact. The arity check turns a stack mismatch into a named book diagnostic. A parameterized `main` is not a spell entry the CLI can call.

Boundary. Opened books populate `Imports` and `Capabilities`; a general external package import surface is still separate. The book cache is keyed by resolved path and source hash; `clearCache` resets it. `MaxCallDepth` remains the depth policy.

Status: implemented. Per-unit hashes and dependencies, lazy loading, main arity, call arity, and the package contract are tested. The suite is 1530 checks (1488 examples, 42 properties), 33 granular cases, and 91 discovered tests.

D199 — Open a book

Decision. A `.book` file imports another book with a leading `open name` line. It has no quotes and no `.book` suffix: `open math` resolves to `math.book` beside the including file. The opened book's exports become callable words in the importer, its package capabilities are unioned into the importer, and its package name and version are recorded in `Imports`. The opened definitions are made available before the importer's own definitions.

Why. The surface reads as prose and names the module directly. The file extension and directory are the loader's concern, not the writer's. The package fields make the dependency and capability surface explicit instead of hiding a second call path.

Boundary. Missing opens, duplicate opens, open cycles, and conflicting definition names are named refusals. A book with opens is rebuilt when loaded so a changed opened book cannot leave a stale parent. Nested opens work recursively.

Index. The ordered open block at the top of the main book is the project index. There is no separate index file: each file appears once, in dependency order, and a dependency must appear before its user. A duplicate, missing, out-of-order, or cyclic entry is refused. This is the F# project-file model placed in the book itself; a separate solution file is only needed for multiple roots.

Status: implemented. Open parsing, import recording, callable opened exports, ordered project index, and cycle refusal are tested. The suite is 1530 checks (1488 examples, 42 properties), 33 granular cases, and 91 discovered tests.